M05 — Prácticas integrales: DDD y CQRS con .NET 8

Autor del curso: JAIME GALLO

Cómo utilizar el manual

Continúa exactamente la etapa **M04 4.5**, que contiene catorce proyectos y 118 pruebas. Construye cada etapa en su propia carpeta a partir de la anterior. Los bloques contienen el código completo que se crea o modifica; sigue los pasos para reconstruir el resultado. Los 120 pasos conservan los catálogos, validadores, documentación y pruebas del recorrido e incorporan un contexto de Producción ejecutable.

Requisitos: Visual Studio con desarrollo ASP.NET, SDK .NET 8 y SQL Server Express o LocalDB. Las pruebas de integración usan el proveedor SQL Server real; las pruebas unitarias de lógica pueden emplear dobles de sus puertos. Ejecuta los comandos desde la raíz de la etapa indicada salvo cuando se indique la raíz del curso. En Bash sustituye las barras inversas de las rutas Windows por barras normales.

SQL Server para las pruebas. Conserva `tests/Compartido/SqlServerPruebas.cs` al copiar la etapa anterior. `BaseSqlServer.Crear()` toma la conexión de `ACERIA_TEST_SQLSERVER`, conecta a `master` y crea una base con nombre único por prueba; la cuenta necesita permiso para crear y eliminar esas bases. Al finalizar elimina solo la base que creó. Configura la variable antes de ejecutar `dotnet test`. Ejemplo PowerShell con LocalDB: `$env:ACERIA_TEST_SQLSERVER = "Server=(localdb)\MSSQLLocalDB;Database=master;Integrated Security=True;TrustServerCertificate=True"`. Para Express sustituye el servidor por `.\SQLEXPRESS` o el nombre de tu instancia. Si la variable no está definida, el helper necesita Docker y arranca SQL Server 2022 Express mediante `Testcontainers.MsSql` 3.10.0 y la imagen fija `mcr.microsoft.com/mssql/server:2022-CU14-ubuntu-22.04`. Una conexión errónea o un motor no disponible hacen fallar las pruebas; no se omiten ni se cambia a un proveedor en memoria.

Capas y frontera: `AceriaApp.Api` compone los adaptadores de entrada. Los proyectos hexagonales de M03–M04 conservan sus contratos. `AceriaApp.Produccion.Dominio` expresa el modelo propio del contexto Producción; desde 5.3 se incorporan `AceriaApp.Produccion.Aplicacion` e `AceriaApp.Produccion.Infraestructura`. Los otros contextos de los catálogos son modelos de análisis, no servicios desplegados. Un nombre del catálogo no registra un manejador ni crea una entidad del dominio.

Convención de código: cada bloque identifica archivo y operación. Crear exige un archivo nuevo; reemplazar exige un archivo existente; insertar exige una única coincidencia del texto indicado. Los fragmentos de los retos se aplican únicamente en una copia optativa. Las explicaciones numeran las líneas del bloque anterior e incluyen las líneas vacías.

Preparar Docker Desktop en Windows para los tests SQL (opcional)

La **API se ejecuta en Windows con SQL Server Express LocalDB**. Docker solo es necesario si eliges la variante de pruebas SQL Server Express mediante Testcontainers; **no se despliega aquí la aplicación entera**. Trabaja con PowerShell y realiza estas comprobaciones antes del primer test que necesite SQL Server:

1. Instala Docker Desktop para Windows usando el motor **WSL 2** y contenedores **Linux**; abre Docker Desktop y espera a que indique que el motor está iniciado. Si ya utilizas LocalDB, puedes omitir

Docker. La instalación, las redes y los despliegues completos de contenedores se estudiarán por separado.

2. En PowerShell ejecuta `docker version` y `docker compose version`. Comprueba el tipo del servidor con `docker info --format '{{.OSType}}'`: debe devolver `linux`. Si da un error de conexión, inicia Docker Desktop; no continúes.
3. Comprueba el motor con `docker run --rm hello-world`. La primera vez debe descargarse una imagen; no es todavía la prueba de SQL Server. Verifica espacio disponible y memoria suficiente para SQL Server Express.
4. **Opción Docker/Testcontainers:** desde la raíz de una etapa **ya completada**, ejecuta `Remove-Item Env:\ACERIA_TEST_SQLSERVER -ErrorAction SilentlyContinue`, seguido de `dotnet test .\AceriaApp.sln -c Release` (en la primera etapa de M02, usa el `.csproj` de tests si aún no existe la solución). El helper crea un contenedor temporal SQL Server Express 2022, bases aisladas y las elimina. La imagen de SQL Server se descarga automáticamente; **no hace falta un `compose.yml` de SQL**.
5. **Alternativa LocalDB sin Docker:** ejecuta `sqllocaldb start MSSQLLocalDB`, configura `$env:ACERIA_TEST_SQLSERVER = 'Server=(localdb)\MSSQLLocalDB;Database=master;Integrated Security=True;TrustServerCertificate=True'` y ejecuta de nuevo los tests. Ambas vías usan SQL Server real; no se permite sustituirlo por SQLite o InMemory.
6. **Diagnóstico:** si falla Docker, comprueba que Docker Desktop está abierto y en modo Linux; si falla SQL, revisa la salida del test, la cadena de conexión a `master` y los permisos de creación de bases. No consideres una compilación como prueba de persistencia ni ignores pruebas fallidas.

Resultado exigido: `dotnet test` debe terminar con todas las pruebas de la etapa en estado correcto. El comando `docker ps -a` sirve para observar los contenedores temporales durante la ejecución, no para atribuir automáticamente un PASS. Esta preparación no altera el proyecto AceriaApp ni exige Linux como sistema operativo del alumno.

Resultados de comprobación

Punto	Tests de catálogo nuevos	Tests funcionales nuevos	Tests de la etapa	Proyectos
Base M04 4.5	—	—	118	14
5.1	11	5	134	16
5.2	12	7	153	17
5.3	12	7	172	20
5.4	12	6	190	21
5.5	10	8	208	22

Cada reto añade una prueba al proyecto de su punto y a la solución completa, únicamente en su copia optativa. Los retos no se acumulan en las etapas obligatorias.

Punto 5.1 — Conceptos básicos de Domain-Driven Design

Construir un lenguaje común y delimitar el contexto Producción

En AceriaApp la palabra «orden» puede significar cosas diferentes para Producción, Inventario o Clientes. Domain-Driven Design comienza por reconocer esos límites y acordar el vocabulario con el que describiremos el negocio. Un contexto delimitado no se crea al ponerle nombre a una clase; sus reglas deben poder sostenerse dentro de una frontera coherente y sus contratos deben explicar qué información comparte con otras capacidades.

El primer punto conserva el sistema acumulado desde M04 y añade tanto un catálogo docente de contextos como una implementación ejecutable del núcleo de Producción. La orden de producción tendrá identidad y transiciones de estado propias; no reutilizaremos la entidad simple del primer módulo como si todos los contextos necesitaran el mismo modelo. Los tests deben diferenciar la validez de una descripción del catálogo de las invariantes reales del dominio. Al finalizar podremos explicar qué se implementó, qué sigue siendo conceptual y cómo el comportamiento se refleja en el código.

Carpeta de trabajo: `M05_DDD_CQRS/PROYECTO/5.1-conceptos-ddd`.

Objetivo: completar los veinticuatro pasos, conservar los documentos heredados y comprobar el modelo con código y tests ejecutables. La etapa suma 134 pruebas en 16 proyectos.

Ejercicio guiado. Añadir al punto 4.5 un documento de conceptos básicos de DDD en `docs/`, una clase de validación de contextos delimitados en el proyecto API y un test unitario que verifique la validación. El punto 5.1 es el primer punto del módulo 5 y parte del punto 4.5, que es el último punto del módulo 4. Se crea en la carpeta `M05_DDD_CQRS\PROYECTO`. La implementación del paso 16 añade comportamiento ejecutable del contexto Producción al ejercicio de catálogo.

PASO 5.1.1 — Crear la estructura del módulo 5 y del punto 5.1

La copia del punto 4.5 conserva los patrones y los endpoints que ya estaban probados. En 5.1 comenzamos a modelar Producción como una frontera propia; trabajar sobre otra carpeta permite comparar el estado heredado con el nuevo sin sobrescribir el proyecto anterior.

1. Abre la terminal integrada y localiza `M04_PATRONES_ARQUITECTURA/PROYECTO/4.5-inyeccion-dependencias`; esa es la etapa completa que vas a continuar.
2. Crea `M05_DDD_CQRS/PROYECTO/5.1-conceptos-ddd` y copia en ella el contenido de `4.5-inyeccion-dependencias`; en Windows puedes usar `xcopy` con `/E /I /H /Y`.
3. Excluye `bin`, `obj` y `TestResults`; abre únicamente la copia de `5.1-conceptos-ddd` para conservar intacto el punto anterior.
4. Ejecuta `dir` dentro de `5.1-conceptos-ddd` y verifica `AceriaApp.sln`, `README.md`, `src`, `tests`, `docs` y `global.json`.

Verificación concreta. La copia contiene la solución y los 14 proyectos anteriores; todavía no contiene los archivos de 5.1.

Diagnóstico si falla.

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.
- Copiar desde el punto 4.4 en lugar del 4.5: se pierde el contenido del último punto del módulo 4.

PASO 5.1.2 — Abrir la solución copiada en Visual Studio con soporte de .NET 8

Abrir la solución copiada permite trabajar en el punto 5.1 sobre la base del punto 4.5. La compilación verifica que la copia es correcta.

1. Abre `AceriaApp.sln` dentro de `5.1-conceptos-ddd` con Visual Studio; en Windows puedes ejecutar `start AceriaApp.sln`.
2. Ejecuta `dotnet sln AceriaApp.sln list` y comprueba los 14 proyectos heredados; todas sus rutas deben existir en esta etapa.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release` para comprobar el punto de partida.
4. Si hay DLL bloqueadas por una API en ejecución, detén esa aplicación y repite la compilación. Continúa cuando no haya errores.

Verificación concreta. La solución muestra 14 proyectos y `dotnet build AceriaApp.sln -c Release` termina con `Build succeeded.`.

Diagnóstico si falla.

- Abrir la solución del punto 4.5 en lugar de la del punto 5.1: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener la API del punto 4.5 en ejecución: el puerto queda ocupado y la compilación falla.

PASO 5.1.3 — Verificar que la carpeta `docs` contiene los documentos del punto 4.5

La nueva etapa preserva 18 documentos y añade su ficha temática y su ADR. Los cambios de integración se aplican únicamente en la copia actual.

1. Expande `docs` dentro de la copia de `5.1-conceptos-ddd`.
2. Comprueba los 18 documentos heredados: ADR-010 a ADR-018 y fichas de los puntos anteriores.
3. Compara sus nombres y contenido con `docs` de `4.5-inyeccion-dependencias`; no modifiques ni elimines documentos anteriores.
4. Confirma que todavía no existen `docs/conceptos-ddd.md` ni `docs/ADR-019-conceptos-ddd.md`.

Verificación concreta. Existen 18 documentos heredados idénticos y no existen todavía los dos de este punto.

Diagnóstico si falla.

- Modificar alguno de los ADR o documentos existentes: rompe la conservación de la etapa anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 4.5.
- Crear la carpeta `docs` si ya existe: `mkdir` falla.

PASO 5.1.4 — Crear el documento de conceptos básicos de DDD en `docs`

El documento de conceptos básicos de DDD es el artefacto de `docs` específico del punto 5.1. No modifica los 18 documentos existentes del punto 4.5.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `conceptos-ddd.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que el archivo aparece en la carpeta `docs` sin subrayados rojos.

Archivo: `docs/conceptos-ddd.md`. **Operación:** Crear.

```
# Conceptos básicos de Domain-Driven Design — Punto 5.1

## Definición

Enfoque que sitúa el dominio del negocio en el centro del proceso de desarrollo.

## Dominio y subdominios

- Núcleo: aporta valor diferencial.
- Soporte: apoya al núcleo.
- Genérico: se resuelve con soluciones existentes.

## Contexto delimitado

Frontera dentro de la cual un modelo es consistente y un término tiene un significado preciso.

## Lenguaje ubicuo

Vocabulario común entre negocio y desarrollo. Se refleja en el código.

## Modelo del dominio

- Entidades: objetos con identidad propia.
- Value objects: objetos sin identidad propia.
- Agregados: grupos de objetos con una raíz.
- Servicios de dominio: operaciones sin estado.

## DDD estratégico y táctico

- Estratégico: contextos delimitados y lenguaje ubicuo.
- Táctico: entidades, value objects, agregados, repositorios.

## DDD y microservicios

Un contexto delimita un modelo y un lenguaje. Puede implementarse como módulo del mismo proceso o evolucionar
hacia un servicio autónomo; DDD no exige desplegar un microservicio por contexto.

## Beneficios

Comunicación, alineación, mantenibilidad y evolución.

## Costes

Inversión inicial, colaboración continua, complejidad adicional y dificultad en dominios poco comprendidos.

## Contextos delimitados del proyecto

| Contexto | Descripción |
|----------|-------------|
| Producción | Órdenes de fabricación |
| Aprovisionamiento | Órdenes de compra |
| Clientes | Registro de clientes |
| Identidad | Usuarios, roles y permisos |

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Contextos de análisis | Producción, aprovisionamiento, clientes e identidad |
| Lenguaje ubicuo | Términos del negocio |
| Implementación ejecutable | Contexto Producción en un ensamblado de dominio independiente |
| Validación | Test de coherencia |

## Correspondencia ejecutable

`ContextoDelimitado`, `CatalogoContextos` y `ValidadorContextos` representan datos del ejercicio. Sus
indicadores no prueban la arquitectura por sí solos. El contexto implementado es Producción, con
`OrdenProduccion` en un proyecto independiente y transiciones controladas. Las pruebas funcionales comprueban
construcción, reglas y frontera de dependencias. La relación entre contextos no requiere compartir las
entidades del modelo.
```

Verificación concreta. El archivo `docs/conceptos-ddd.md` existe; comprueba que sus títulos, contenido y enlaces coinciden con todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: rompe la conservación de la etapa anterior.
- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

PASO 5.1.5 — Crear la carpeta `Ddd` en el proyecto API

La carpeta `Ddd` albergará las clases de validación de contextos delimitados. Se crea como carpeta nueva sin tocar las carpetas existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.Api`.
2. Seleccionar `Add` → `New Folder` y nombrar la carpeta `Ddd`.
3. Verificar que la carpeta `Ddd` aparece dentro de `AceriaApp.Api` sin afectar a las carpetas existentes.
4. Verificar que no se ha modificado ningún archivo existente del punto 4.5.

Verificación concreta. la carpeta `Ddd` existe dentro de `AceriaApp.Api` y las carpetas existentes permanecen intactas.

Diagnóstico si falla.

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: rompe la conservación de la etapa anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

PASO 5.1.6 — Crear la clase `ContextoDelimitado`

`ContextoDelimitado` describe un límite y su lenguaje para discutir el mapa del negocio. La clase no crea una base de datos ni un microservicio: el dominio ejecutable de Producción se introduce por separado.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `Ddd`.
2. Seleccionar `Add` → `Class` y nombrar la clase `ContextoDelimitado.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/Ddd/ContextoDelimitado.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Ddd;

public class ContextoDelimitado
{
    public string Nombre { get; init; } = string.Empty;
    public string TipoDeSubdominio { get; init; } = string.Empty;
    public string LenguajeUbicuo { get; init; } = string.Empty;
    public bool TieneModeloPropio { get; init; }
    public bool ComparteModeloConOtro { get; init; }
}
```

Verificación concreta. la clase `ContextoDelimitado` compila sin errores y expone cinco propiedades.

Diagnóstico si falla.

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar las propiedades de tipo `string` con `string.Empty`: el compilador emite advertencias.
- Usar `set` en lugar de `init`: permite modificar la instancia después de crearla.

PASO 5.1.7 — Crear la clase `ValidadorContextos`

`ValidadorContextos` contrasta las reglas del catálogo con los datos declarados. Sus tests pueden señalar una descripción incoherente, pero no decidir automáticamente si un bounded context refleja el negocio real.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `Ddd`.
2. Seleccionar `Add` → `Class` y nombrar la clase `ValidadorContextos.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Alcance del validador. Comprueba los indicadores de los ejemplos del catálogo. No analiza por reflexión los tipos de producción ni garantiza por sí solo la arquitectura. Las reglas del modelo ejecutable y sus límites se comprueban en las pruebas funcionales del paso 16.

Archivo: `src/AceriaApp.Api/Ddd/ValidadorContextos.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Ddd;

public class ValidadorContextos
{
    private static readonly string[] TiposPermitidos = { "Nucleo", "Soporte", "Generico" };

    public ResultadoContextos Validar(IEnumerable<ContextoDelimitado> contextos)
    {
        var errores = new List<string>();
        var lista = contextos.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay contextos declarados.");
        }

        foreach (var contexto in lista)
        {
            if (string.IsNullOrWhiteSpace(contexto.Nombre))
            {
                errores.Add("Un contexto no declara el nombre.");
            }

            if (!TiposPermitidos.Contains(contexto.TipoDeSubdominio))
            {
                errores.Add($"Tipo de subdominio no permitido en '{contexto.Nombre}':
'{contexto.TipoDeSubdominio}'.");
            }

            if (string.IsNullOrWhiteSpace(contexto.LenguajeUbicuo))
            {
                errores.Add($"El contexto '{contexto.Nombre}' no declara lenguaje ubicuo.");
            }

            if (!contexto.TieneModeloPropio)
            {
                errores.Add($"El contexto '{contexto.Nombre}' no tiene modelo propio.");
            }

            if (contexto.ComparteModeloConOtro)
            {
                errores.Add($"El contexto '{contexto.Nombre}' comparte modelo con otro contexto.");
            }
        }

        return new ResultadoContextos(errores.Count == 0, errores);
    }
}

public record ResultadoContextos(bool EsValido, IReadOnlyList<string> Errores);
```

Verificación concreta. la clase `ValidadorContextos` compila sin errores y expone un método y un `record`.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<string>` e `IReadOnlyList<string>` no se resuelven.
- Olvidar el `using System.Linq;`: `Contains` y `ToList` no se resuelven.
- No comprobar el caso de lista vacía: el validador devuelve válido sin contextos.

PASO 5.1.8 — Añadir los `using` necesarios en `ValidadorContextos`

Los tipos `List<string>` e `IReadOnlyList<string>` pertenecen a `System.Collections.Generic`. Los métodos `Contains` y `ToList` pertenecen a `System.Linq`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. En el panel Solution Explorer, hacer doble clic en `ValidadorContextos.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; el texto de inserción debe aparecer exactamente una vez.
3. Verificar que no hay subrayados rojos en `List<string>`, `Contains` ni `ToList`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/Ddd/ValidadorContextos.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.Ddd;
```

```
using System.Collections.Generic;
using System.Linq;
```

Verificación concreta. `ValidadorContextos.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir solo `System.Collections.Generic` y olvidar `System.Linq`: `Contains` y `ToList` no se resuelven.
- Mezclar directivas using con declaraciones de tipos: coloca las directivas antes del namespace para mantener el orden del archivo.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 5.1.9 — Crear la clase `CatalogoContextos`

`CatalogoContextos` reúne descripciones para compararlas en el ejercicio. Es importante recordar que Inventario y Clientes en el mapa no significan que existan sus endpoints, modelos persistentes o servicios ejecutándose.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `Ddd`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoContextos.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/Ddd/CatalogoContextos.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Ddd;

public class CatalogoContextos
{
    public static List<ContextoDelimitado> ContextosPorDefecto()
    {
```

```
        return new List<ContextoDelimitado>
        {
            new()
            {
                Nombre = "Produccion",
                TipoDeSubdominio = "Nucleo",
                LenguajeUbicuo = "orden de produccion, material, cantidad solicitada",
                TieneModeloPropio = true,
                ComparteModeloConOtro = false
            },
            new()
            {
                Nombre = "Aprovisionamiento",
                TipoDeSubdominio = "Soporte",
                LenguajeUbicuo = "orden de compra, proveedor, fecha de entrega",
                TieneModeloPropio = true,
                ComparteModeloConOtro = false
            },
            new()
            {
                Nombre = "Clientes",
                TipoDeSubdominio = "Soporte",
                LenguajeUbicuo = "cliente, razon social, contacto",
                TieneModeloPropio = true,
                ComparteModeloConOtro = false
            },
            new()
            {
                Nombre = "Identidad",
                TipoDeSubdominio = "Generico",
                LenguajeUbicuo = "usuario, rol, permiso",
                TieneModeloPropio = true,
                ComparteModeloConOtro = false
            }
        };
    }
}
```

Verificación concreta. la clase `CatalogoContextos` compila sin errores y expone un método estático.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<T>` no se resuelve.
- No inicializar las propiedades de los objetos: el compilador emite advertencias.
- Reutilizar una lista estática mutable: cada llamada del catálogo debe construir una lista independiente.

PASO 5.1.10 — Añadir el `using` necesario en `CatalogoContextos`

El tipo `List<T>` pertenece a `System.Collections.Generic`. En proyectos con `ImplicitUsings` desactivado, hay que añadir el using explícito.

1. En el panel Solution Explorer, hacer doble clic en `CatalogoContextos.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; el texto de inserción debe aparecer exactamente una vez.
3. Verificar que no hay subrayados rojos en `List<T>`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/Ddd/CatalogoContextos.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.Ddd;
```

```
using System.Collections.Generic;
```

Verificación concreta. `CatalogoContextos.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir `using System.Linq;` en lugar de `System.Collections.Generic`: no resuelve `List<T>`.
- Colocar el using después de una clase: las directivas deben preceder las declaraciones de tipos.
- Olvidar el using y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 5.1.11 — Crear el proyecto de tests del punto 5.1

El punto 5.1 añade un proyecto de tests específico para las clases de DDD. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

1. Crea `tests/AceriaApp.Ddd.Tests` y su archivo `AceriaApp.Ddd.Tests.csproj` con el XML completo. Si utilizas la plantilla xUnit de Visual Studio, ajusta el archivo generado.
2. Elimina `UnitTest1.cs` si lo creó la plantilla; las pruebas completas se añaden en los pasos 14 y 15.
3. Ejecuta `dotnet sln AceriaApp.sln add tests/AceriaApp.Ddd.Tests/AceriaApp.Ddd.Tests.csproj` incluso si la solución ya existe.
4. Ejecuta `dotnet sln AceriaApp.sln list` y `dotnet restore tests/AceriaApp.Ddd.Tests/AceriaApp.Ddd.Tests.csproj`.

Archivo: `tests/AceriaApp.Ddd.Tests/AceriaApp.Ddd.Tests.csproj`. **Operación:** Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. La solución muestra 15 proyectos, incluido `AceriaApp.Ddd.Tests`; el csproj fija .NET 8, SDK de tests 17.12.0, xUnit 2.9.2 y runner 2.8.2.

Diagnóstico si falla.

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 4.5: rompe la conservación de la etapa anterior.
- Usar `dotnet new mstest`: el curso usa xUnit.

PASO 5.1.12 — Añadir la referencia del proyecto de tests a la API

La referencia permite que el proyecto de tests acceda a las clases de `Ddd` del proyecto API.

1. Ejecuta `dotnet add tests/AceriaApp.Ddd.Tests/AceriaApp.Ddd.Tests.csproj reference src/AceriaApp.Api/AceriaApp.Api.csproj` o deja el archivo como el XML completo.
2. Comprueba que la referencia va del proyecto de tests a la API; la API no debe depender de los tests.
3. Abre `tests/AceriaApp.Ddd.Tests/AceriaApp.Ddd.Tests.csproj` y confirma la ruta `../../src/AceriaApp.Api/AceriaApp.Api.csproj`.
4. Ejecuta `dotnet build tests/AceriaApp.Ddd.Tests/AceriaApp.Ddd.Tests.csproj -c Release` y verifica que resuelve la API.

Archivo: `tests/AceriaApp.Ddd.Tests/AceriaApp.Ddd.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. el archivo `AceriaApp.Ddd.Tests.csproj` contiene un `ProjectReference` al proyecto API.

Diagnóstico si falla.

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

PASO 5.1.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Ejecuta `dotnet add tests/AceriaApp.Ddd.Tests/AceriaApp.Ddd.Tests.csproj package FluentAssertions --version 6.12.2` o escribe el XML completo.
2. Ejecuta `dotnet list tests/AceriaApp.Ddd.Tests/AceriaApp.Ddd.Tests.csproj package` y localiza `FluentAssertions 6.12.2`.
3. Conserva la referencia a la API y las versiones fijadas de xUnit.
4. Ejecuta `dotnet build tests/AceriaApp.Ddd.Tests/AceriaApp.Ddd.Tests.csproj -c Release`; las aserciones `Should()` estarán disponibles.

Archivo: `tests/AceriaApp.Ddd.Tests/AceriaApp.Ddd.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. El archivo y `dotnet list package` muestran FluentAssertions 6.12.2; la compilación termina correctamente.

Diagnóstico si falla.

- Instalar una versión preliminar: el curso usa la versión fijada en el archivo de proyecto.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

PASO 5.1.14 — Crear el archivo `ValidadorContextosTests.cs`

Los tests verifican que el validador acepta el catálogo por defecto y rechaza los contextos que violan las reglas de DDD.

1. En el panel Solution Explorer, expandir `tests` → `AceriaApp.Ddd.Tests`.
2. Crear el archivo `ValidadorContextosTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.Ddd.Tests/ValidadorContextosTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.Ddd;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Ddd.Tests;

public class ValidadorContextosTests
{
    private readonly ValidadorContextos _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoContextos.ContextosPorDefecto());

        resultado.EsValido.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinContextos_DevuelveError()
    {
        var resultado = _validador.Validar(new List<ContextoDelimitado>());

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay contextos"));
    }

    [Fact]
    public void Validar_TipoNoPermitido_DevuelveError()
    {
        var contextos = new List<ContextoDelimitado>
        {
            new() { Nombre = "ContextoX", TipoDeSubdominio = "TipoInventado", LenguajeUbicuo = "lenguaje",
TieneModeloPropio = true }
        };

        var resultado = _validador.Validar(contextos);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Tipo de subdominio no permitido"));
    }

    [Fact]
    public void Validar_SinLenguajeUbicuo_DevuelveError()
    {
        var contextos = new List<ContextoDelimitado>
        {
            new() { Nombre = "ContextoX", TipoDeSubdominio = "Nucleo", LenguajeUbicuo = string.Empty,
TieneModeloPropio = true }
        };
```

```
        var resultado = _validador.Validar(contextos);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara lenguaje ubicuo"));
    }

    [Fact]
    public void Validar_SinModeloPropio_DevuelveError()
    {
        var contextos = new List<ContextoDelimitado>
        {
            new() { Nombre = "ContextoX", TipoDeSubdominio = "Nucleo", LenguajeUbicuo = "lenguaje",
TieneModeloPropio = false }
        };

        var resultado = _validador.Validar(contextos);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene modelo propio"));
    }

    [Fact]
    public void Validar_ModeloCompartido_DevuelveError()
    {
        var contextos = new List<ContextoDelimitado>
        {
            new() { Nombre = "ContextoX", TipoDeSubdominio = "Nucleo", LenguajeUbicuo = "lenguaje",
TieneModeloPropio = true, ComparteModeloConOtro = true }
        };

        var resultado = _validador.Validar(contextos);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("comparte modelo"));
    }
}
```

Verificación concreta. el archivo `ValidadorContextosTests.cs` compila y contiene seis métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.Ddd;`: las clases no se resuelven.
- Usar `Assert.True` en lugar de `Should().BeTrue()`: el curso usa FluentAssertions.
- No cubrir el caso de modelo compartido: la regla específica no se verifica.

PASO 5.1.15 — Crear el archivo `CatalogoContextosTests.cs`

Los tests verifican que el catálogo por defecto contiene los contextos esperados y la distribución correcta por tipo de subdominio.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.Ddd.Tests`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoContextosTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.Ddd.Tests/CatalogoContextosTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.Ddd;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Ddd.Tests;

public class CatalogoContextosTests
{
    [Fact]
    public void ContextosPorDefecto_ContieneCuatroContextos()
    {
        var contextos = CatalogoContextos.ContextosPorDefecto();
```

```
        contextos.Should().HaveCount(4);
    }

    [Fact]
    public void ContextosPorDefecto_ContieneUnNucleo()
    {
        var contextos = CatalogoContextos.ContextosPorDefecto();

        contextos.Count(c => c.TipoDeSubdominio == "Nucleo").Should().Be(1);
    }

    [Fact]
    public void ContextosPorDefecto_ContieneDosSoportes()
    {
        var contextos = CatalogoContextos.ContextosPorDefecto();

        contextos.Count(c => c.TipoDeSubdominio == "Soporte").Should().Be(2);
    }

    [Fact]
    public void ContextosPorDefecto_ContieneUnGenerico()
    {
        var contextos = CatalogoContextos.ContextosPorDefecto();

        contextos.Count(c => c.TipoDeSubdominio == "Generico").Should().Be(1);
    }

    [Fact]
    public void ContextosPorDefecto_NingunoComparteModelo()
    {
        var contextos = CatalogoContextos.ContextosPorDefecto();

        contextos.Should().OnlyContain(c => !c.ComparteModeloConOtro);
    }
}
```

Verificación concreta. el archivo `CatalogoContextosTests.cs` compila y contiene cinco métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.Ddd;`: las clases no se resuelven.
- Olvidar el `using System.Linq;`: `Count` y `OnlyContain` no se resuelven.
- No verificar la distribución por tipo: el catálogo podría estar mal balanceado.

PASO 5.1.16 — Construir la frontera del contexto Producción y compilar

Aquí aparece una frontera de dominio auténtica: `OrdenProduccion` controla la creación, la cantidad y las transiciones Borrador, Planificada y Finalizada, y `ReglaProduccionException` identifica incumplimientos conocidos. Comprueba que el proyecto de dominio no referencia EF Core ni ASP.NET Core y que los tests ejercitan las reglas, no solo su nombre.

1. Lee cada archivo de integración de este paso y localiza su responsabilidad en las capas del contexto Producción.
2. Crea o modifica los archivos exactamente como se indica; sigue la lectura numerada para conectar contratos, reglas y pruebas.
3. Añade los nuevos proyectos a la solución: `dotnet sln AceriaApp.sln add src/AceriaApp.Produccion.Dominio/AceriaApp.Produccion.Dominio.csproj`.
4. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`; `dotnet sln AceriaApp.sln list` debe mostrar 16 proyectos.

Integración funcional. Configura las referencias y versiones de este proyecto manteniendo los proyectos y catálogos heredados.

Archivo: `src/AceriaApp.Produccion.Dominio/AceriaApp.Produccion.Dominio.csproj`. **Operación:** Crear.

```
<Project Sdk="Microsoft.NET.Sdk"><PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</
Nullable><ImplicitUsings>enable</ImplicitUsings></PropertyGroup></Project>
```

Compila el modelo para .NET 8. No incluye referencias de proyecto ni paquetes externos: la frontera del dominio se comprueba también con tests.

Integración funcional. Modela la raíz de Producción con identidad estable, transiciones explícitas y reglas de capacidad; desde 5.2 controla todas las mutaciones de sus líneas.

Archivo: `src/AceriaApp.Produccion.Dominio/OrdenProduccion.cs`. **Operación:** Crear.

```
namespace AceriaApp.Produccion.Dominio;

// Producción tiene identidad y ciclo de vida propios; no reutiliza Orden de M2/M4.
public sealed class OrdenProduccion : IEquatable<OrdenProduccion>
{
    public const int CapacidadMaxima = 500;
    public Guid Id { get; private set; }
    public string Cliente { get; private set; }
    public string Material { get; private set; }
    public int Cantidad { get; private set; }
    public EstadoOrdenProduccion Estado { get; private set; }

    public OrdenProduccion(string cliente, string material, int cantidad)
    {
        if (string.IsNullOrWhiteSpace(cliente) || cliente.Trim().Length > 100)
            throw new ReglaProduccionException("cliente.invalido", "El cliente debe tener entre 1 y 100
caracteres.");
        if (string.IsNullOrWhiteSpace(material) || material.Trim().Length > 100)
            throw new ReglaProduccionException("material.invalido", "El material debe tener entre 1 y 100
caracteres.");
        if (cantidad is < 1 or > CapacidadMaxima)
            throw new ReglaProduccionException("cantidad.invalida", "La cantidad debe estar entre 1 y 500
unidades.");
        Id = Guid.NewGuid();
        Cliente = cliente.Trim();
        Material = material.Trim();
        Cantidad = cantidad;
        Estado = EstadoOrdenProduccion.Borrador;
    }

    public void Planificar()
    {
        if (Estado != EstadoOrdenProduccion.Borrador)
            throw new ReglaProduccionException("estado.invalido", "Solo se puede planificar una orden en
borrador.");
        Estado = EstadoOrdenProduccion.Planificada;
    }

    public void Finalizar()
    {
        if (Estado != EstadoOrdenProduccion.Planificada)
            throw new ReglaProduccionException("estado.invalido", "Solo se puede finalizar una orden
planificada.");
        Estado = EstadoOrdenProduccion.Finalizada;
    }

    public bool Equals(OrdenProduccion? otra) => otra is not null && Id == otra.Id;
    public override bool Equals(object? otra) => otra is OrdenProduccion orden && Equals(orden);
    public override int GetHashCode() => Id.GetHashCode();
}
```

Fija la capacidad didáctica por orden, no una cuota global entre órdenes. Valida el cliente normalizado antes de crear un agregado o llegar a la persistencia. Impide finalizar directamente desde Borrador o volver a finalizar una orden.

Compara entidades por identidad estable; los datos iguales no convierten dos órdenes distintas en una sola.

Integración funcional. Define el error de negocio y los estados Borrador, Planificada y Finalizada sin dependencias técnicas.

Archivo: `src/AceriaApp.Produccion.Dominio/ReglaProduccionException.cs`. **Operación:** Crear.

```
namespace AceriaApp.Produccion.Dominio;
public sealed class ReglaProduccionException(string codigo, string mensaje) : Exception(mensaje)
{
    public string Codigo { get; } = codigo;
}
public enum EstadoOrdenProduccion { Borrador, Planificada, Finalizada }
```

Distingue una regla del modelo incumplida de un fallo técnico. La aplicación puede traducir este tipo concreto sin ocultar otras excepciones. Conserva un código estable que permite identificar la regla sin depender del texto del mensaje. Expresa los estados del ciclo de vida; los métodos de la raíz deciden qué transiciones están permitidas.

Integración funcional. Configura las referencias y versiones de este proyecto manteniendo los proyectos y catálogos heredados.

Archivo: `tests/AceriaApp.Ddd.Tests/AceriaApp.Ddd.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsPackable>false</IsPackable><IsTestProject>true</IsTestProject></PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <ProjectReference Include="../../src/AceriaApp.Produccion.Dominio/AceriaApp.Produccion.Dominio.csproj" />

  </ItemGroup>
</Project>
```

Fija la versión del paquete usado por esta capa o por sus pruebas para hacer reproducible la compilación. Fija la versión del paquete usado por esta capa o por sus pruebas para hacer reproducible la compilación. Fija la versión del paquete usado por esta capa o por sus pruebas para hacer reproducible la compilación.

Fija la versión del paquete usado por esta capa o por sus pruebas para hacer reproducible la compilación. Declara una dependencia de proyecto que el compilador necesita; no registra servicios en el contenedor. Declara una dependencia de proyecto que el compilador necesita; no registra servicios en el contenedor.

Integración funcional. Añade 5 pruebas funcionales del punto 5.1, además del catálogo original; verifica comportamiento real y regresiones heredadas.

Archivo: `tests/AceriaApp.Ddd.Tests/DddFuncionalesTests.cs`. **Operación:** Crear.

```
using AceriaApp.Produccion.Dominio;
using Xunit;
namespace AceriaApp.Ddd.Tests;
public sealed class DddFuncionalesTests
{
    [Fact]
    public void Crear_una_orden_asigna_identidad_y_normaliza_el_lenguaje_del_negocio()
    {
        var orden = new OrdenProduccion("  Axia  ", " Acero ", 20);
        Assert.NotEqual(Guid.Empty, orden.Id);
        Assert.Equal("Axia", orden.Cliente);
        Assert.Equal("Acero", orden.Material);
        Assert.Equal(20, orden.Cantidad);
```

```
        Assert.Equal(EstadoOrdenProduccion.Borrador, orden.Estado);
        Assert.NotEqual(orden, new OrdenProduccion("Axia", "Acero", 20));
    }
    [Fact]
    public void No_se_crean_ordenes_sin_cliente_o_material()
    {
        Assert.Throws<ReglaProduccionException>(() => new OrdenProduccion(" ", "Acero", 20));
        Assert.Throws<ReglaProduccionException>(() => new OrdenProduccion("Axia", " ", 20));
        Assert.Throws<ReglaProduccionException>(() => new OrdenProduccion(new string('C', 101), "Acero",
20));
        Assert.Throws<ReglaProduccionException>(() => new OrdenProduccion("Axia", new string('M', 101), 20));
        var limite = new OrdenProduccion(new string('C', 100), new string('M', 100), 20);
        Assert.Equal(100, limite.Cliente.Length);
        Assert.Equal(100, limite.Material.Length);
    }
    [Fact]
    public void Cantidad_respeta_los_limites_de_la_capacidad_didactica()
    {
        Assert.Throws<ReglaProduccionException>(() => new OrdenProduccion("Axia", "Acero", 0));
        Assert.Throws<ReglaProduccionException>(() => new OrdenProduccion("Axia", "Acero", 501));
        Assert.Equal(500, new OrdenProduccion("Axia", "Acero", 500).Cantidad);
    }
    [Fact]
    public void Planificar_y_finalizar_conserva_la_identidad()
    {
        var orden = new OrdenProduccion("Axia", "Acero", 20);
        var id = orden.Id;
        orden.Planificar();
        Assert.Equal(EstadoOrdenProduccion.Planificada, orden.Estado);
        orden.Finalizar();
        Assert.Equal(EstadoOrdenProduccion.Finalizada, orden.Estado);
        Assert.Equal(id, orden.Id);
        Assert.Throws<ReglaProduccionException>(orden.Planificar);
        Assert.Throws<ReglaProduccionException>(orden.Finalizar);
    }
    [Fact]
    public void Finalizar_sin_planificar_rechaza_la_transicion_sin_mutar()
    {
        var orden = new OrdenProduccion("Axia", "Acero", 20);
        var error = Assert.Throws<ReglaProduccionException>(orden.Finalizar);
        Assert.Equal("estado.invalido", error.Codigo);
        Assert.Equal(EstadoOrdenProduccion.Borrador, orden.Estado);
    }
}
```

Comprueba el comportamiento observable: crear una orden asigna identidad y normaliza el lenguaje del negocio. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos. Comprueba el comportamiento observable: no se crean ordenes sin cliente o material. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos. Comprueba el comportamiento observable: cantidad respeta los limites de la capacidad didactica. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos.

Comprueba el comportamiento observable: planificar y finalizar conserva la identidad. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos. Comprueba el comportamiento observable: finalizar sin planificar rechaza la transicion sin mutar. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos.

Verificación concreta. La solución contiene 16 proyectos, los archivos de integración coinciden con los bloques y la compilación completa termina con cero errores.

Diagnóstico si falla.

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.
- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

PASO 5.1.17 — Ejecutar todos los tests del punto 5.1

Las pruebas de catálogo verifican los datos declarados; las pruebas funcionales ejecutan las reglas y los adaptadores reales del contexto. Ambas evidencias son necesarias.

1. Ejecuta `dotnet test tests/AceriaApp.Ddd.Tests/AceriaApp.Ddd.Tests.csproj -c Release` desde la raíz de la etapa.
2. Comprueba 16 pruebas aprobadas: 11 de catálogo y 5 funcionales.
3. Exige cero fallos y cero omisiones; revisa los nombres de las pruebas para identificar el contrato comprobado.
4. Si falla una prueba, lee su aserción y corrige el archivo del paso correspondiente antes de continuar.

Verificación concreta. El proyecto ejecuta 16 tests: `Failed: 0, Passed: 16, Skipped: 0, Total: 16`.

Diagnóstico si falla.

- Ejecutar `dotnet test` sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

PASO 5.1.18 — Ejecutar todos los tests de la solución

Ejecutar todos los tests de la solución verifica que los tests de los puntos anteriores y del punto 5.1 pasan conjuntamente.

1. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build` después de la compilación Release.
2. Comprueba que se ejecutan los 11 proyectos de tests, incluidos los heredados de M04.
3. Suma los resultados por proyecto: deben ser 134 pruebas aprobadas, cero fallos y cero omisiones.
4. Conserva la salida como evidencia; si cambias código vuelve a compilar antes de utilizar `--no-build`.

Verificación concreta. Los 11 proyectos ejecutan conjuntamente 134 tests aprobados, cero fallos y cero omisiones.

Diagnóstico si falla.

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Omitir proyectos heredados de la solución: se pierde la comprobación de regresión.
- Eliminar o modificar pruebas anteriores sin una instrucción explícita: se pierde la cobertura heredada.

PASO 5.1.19 — Actualizar el README de la copia 5.1

El README del punto 5.1 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 4.5.

1. Abre el archivo `README.md` copiado de `4.5-inyeccion-dependencias` dentro de `5.1-conceptos-ddd`.
2. Reemplaza su contenido completo por el bloque de este paso; el README de la carpeta anterior permanece intacto.
3. Comprueba el objetivo, el estado, la ejecución, los añadidos, la documentación y la verificación ejecutable.
4. Guarda el archivo y abre su vista previa para comprobar títulos, enlaces, rutas y cifras.

Archivo: `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 5.1 — Conceptos básicos de Domain-Driven Design
```

```
## Objetivo
Comprender los conceptos básicos de Domain-Driven Design y añadir al proyecto las clases de validación de
contextos delimitados.

Construir la frontera y las transiciones del contexto Producción.

## Estado del proyecto
- Solución `AceriaApp.sln` con 16 proyectos.
- Documento `docs/conceptos-ddd.md` con la definición y los contextos.
- Clases `ContextoDelimitado`, `ValidadorContextos` y `CatalogoContextos` en `src/AceriaApp.Api/Ddd/`.
- 11 tests de catálogo y 5 funcionales nuevos en `tests/AceriaApp.Ddd.Tests`.
- Los tests acumulados suman 134 pruebas.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio con soporte de .NET 8.
2. Configurar SQL Server para los adaptadores heredados; las pruebas proporcionan su persistencia de prueba.
3. Configurar las conexiones `AceriaApp` y `AceriaHexagonal` en el entorno local si se ejecuta la API con SQL
Server.
4. Ejecutar `dotnet run --project src/AceriaApp.Api/AceriaApp.Api.csproj --urls http://localhost:5000` desde
la raíz de la etapa.
5. Navegar a `http://localhost:5000/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/conceptos-ddd.md`.
- Carpeta `Ddd` en el proyecto API con tres clases nuevas.
- Proyecto `tests/AceriaApp.Ddd.Tests` con 11 tests de catálogo y 5 funcionales nuevos.
- Se conserva intacta la carpeta del punto 4.5.

- Construir la frontera y las transiciones del contexto Producción. Los bloques completos están en el paso 16
del manual.

## Documentación
- [ADR-010](docs/ADR-010-capas-aplicacion.md) a [ADR-018](docs/ADR-018-inyeccion-dependencias.md)
(heredados).
- [Conceptos básicos de DDD](docs/conceptos-ddd.md).

## Verificación ejecutable

El catálogo describe ejemplos de diseño y sus indicadores; no inspecciona automáticamente las clases reales.
Los archivos funcionales del paso 16 y sus tests comprueban el comportamiento ejecutable. Ejecuta `dotnet
test AceriaApp.sln -c Release` y exige cero fallos y cero omisiones.
```

Verificación concreta. El archivo `README.md` existe; comprueba que sus títulos, contenido y enlaces coinciden con todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Modificar el README del punto 4.5: rompe la conservación de la etapa anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 4.5: rompe la separación.

PASO 5.1.20 — Crear el ADR-019 del punto 5.1

El ADR-019 documenta la decisión de adoptar DDD. Es un artefacto nuevo del punto 5.1 que no modifica los ADR existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `ADR-019-conceptos-ddd.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que los ADR-010 a ADR-018 no se han modificado.

Archivo: `docs/ADR-019-conceptos-ddd.md`. **Operación:** Crear.

```
# ADR-019: Adopción de Domain-Driven Design

## Estado
Aceptado
```

```
## Contexto
El proyecto AceriaApp ha cerrado el módulo 4 con los patrones de arquitectura. Es necesario definir cómo se organiza el dominio del negocio y cómo se alinean los límites de los servicios con los límites del negocio.

## Decisión
Se adopta DDD con fronteras explícitas de modelo. El catálogo de análisis describe Producción, Aprovisionamiento, Clientes e Identidad y valida sus indicadores. La implementación comienza únicamente con el contexto Producción en `AceriaApp.Produccion.Dominio`: `OrdenProduccion` usa el lenguaje del negocio y protege las transiciones Borrador, Planificada y Finalizada. El dominio no depende de EF Core, HTTP ni de los proyectos hexagonales heredados. Los otros contextos continúan siendo ejemplos de análisis; no se implementan aquí servicios autónomos para ellos. Las pruebas funcionales ejecutan el modelo propio y comprueban la frontera del ensamblado.

## Alternativas consideradas
- Organización exclusivamente por capas técnicas sin fronteras de dominio: descartada porque mezcla modelos del negocio.
- Modelo único para todo el dominio: descartado por mezclar significados.
- Sin lenguaje ubicuo: descartado por dificultar la comunicación.

## Consecuencias
- Positivas: alineación con el negocio, lenguaje compartido, límites claros.
- Negativas: requiere inversión inicial y colaboración continua.
- Mitigación: la validación automática y la revisión periódica de los contextos.

## Correspondencia ejecutable

`ContextoDelimitado`, `CatalogoContextos` y `ValidadorContextos` representan datos del ejercicio. Sus indicadores no prueban la arquitectura por sí solos. El contexto implementado es Producción, con `OrdenProduccion` en un proyecto independiente y transiciones controladas. Las pruebas funcionales comprueban construcción, reglas y frontera de dependencias. La relación entre contextos no requiere compartir las entidades del modelo.
```

Verificación concreta. El archivo `docs/ADR-019-conceptos-ddd.md` existe; comprueba que sus títulos, contenido y enlaces coinciden con todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Modificar los ADR anteriores: rompe la conservación de la etapa anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-019`: rompe la convención.

PASO 5.1.21 — Comparar los documentos heredados con 4.5-inyeccion-dependencias

La comparación comprueba los documentos heredados byte a byte. Las nuevas clases y los cambios explícitos de integración pertenecen a la copia actual; la carpeta del punto anterior permanece intacta.

1. En la terminal, escribir `cd C:\cursos\AceriaApp` y pulsar Enter.
2. Escribir `for %f in (M04_PATRONES_ARQUITECTURA\PROYECTO\4.5-inyeccion-dependencias\docs\*.md) do fc /L "%f" "M05_DDD_CQRS\PROYECTO\5.1-conceptos-ddd\docs\%~nxf"` y pulsar Enter.
3. Verificar que la salida muestra `FC: no differences encountered` para cada archivo.
4. Verificar que el número de comparaciones coincide con el número de documentos del punto 4.5. Si algún archivo muestra diferencias, restaurarlo desde el punto 4.5.

Verificación concreta. Los 18 documentos heredados coinciden con la etapa anterior, sin omisiones ni modificaciones.

Diagnóstico si falla.

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.
- Omitir `/L` cuando se desea una comparación de texto: indica el modo explícitamente y revisa cada resultado.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

PASO 5.1.22 — Verificar la estructura del punto 5.1

La verificación estructural confirma que el punto 5.1 añade archivos nuevos conservando los documentos heredados.

1. Escribir `tree /F M05_DDD_CQRS\PROYECTO\5.1-conceptos-ddd` y pulsar Enter.
2. Verificar que la salida muestra `AceriaApp.sln`, `README.md`, `src\AceriaApp.Api\Ddd` con las tres clases nuevas, `tests\AceriaApp.Ddd.Tests` con los dos archivos de tests de catálogo y las pruebas funcionales y `docs` con el documento de conceptos y el ADR-019. Comprueba también los proyectos y archivos del contexto Producción indicados en el paso 16.
3. Verificar que las carpetas existentes del punto 4.5 permanecen intactas.
4. Verificar que no existe ninguna carpeta `labs` ni ningún archivo `metaprompt.md` ni `verificacion.md`.

Verificación concreta. la estructura contiene los archivos nuevos y conserva los del punto 4.5.

Diagnóstico si falla.

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 4.5: viola la regla de no modificar.
- Omitir la carpeta `Ddd`: los tests no encuentran las clases.

PASO 5.1.23 — Verificar el `.gitignore` del punto 5.1

El `.gitignore` se hereda del punto 4.5 sin modificaciones. La verificación confirma que no se ha tocado.

1. Verificar que el archivo `.gitignore` existe en la carpeta del punto 5.1 (copiado del punto 4.5).
2. Abrir el archivo `.gitignore` con Visual Studio con soporte de .NET 8.
3. Verificar que contiene las líneas `**/bin/`, `**/obj/`, `.vs/`, `*.user`, `TestResults/` y `*.trx`.
4. Verificar que no se ha modificado respecto al punto 4.5. Pulsar `Ctrl+S` si no hay cambios y cerrar el archivo.

Verificación concreta. el `.gitignore` existe y contiene las seis líneas sin modificaciones.

Diagnóstico si falla.

- Modificar el `.gitignore` copiado del punto 4.5: rompe la conservación de la etapa anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el `.gitignore` con otro nombre: el archivo no se reconoce.

PASO 5.1.24 — Hacer el commit del punto 5.1 y verificar el estado final

El commit registra la etapa completa. Si existe uno por punto desde 1.1, el recorrido suma 19 commits de puntos; otros commits legítimos pueden aumentar el historial.

1. Ejecuta `git diff --stat` y revisa los archivos y entregables de esta etapa.
2. Ejecuta `git add .` después de comprobar que no se incluyen binarios, resultados temporales ni credenciales.
3. Ejecuta `git commit -m "Punto 5.1: Conceptos básicos de Domain-Driven Design"` con `user.name` y `user.email` configurados.
4. Comprueba `git log --oneline` y `git status`; deben registrar este punto y no dejar cambios pendientes de esta actividad.

Verificación concreta. El historial incluye el commit de 5.1 y `git status` no muestra cambios pendientes de esta actividad; no se impone una longitud rígida al historial.

Diagnóstico si falla.

- No tener configurado `user.name` y `user.email`: `git commit` falla.
- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

Reto de ampliación (optativo)

Trabaja en una copia independiente de la etapa obligatoria, por ejemplo `5.1-conceptos-ddd-reto`. Conserva intacta la carpeta obligatoria y sus documentos. Los tres fragmentos añaden un contrato, un cálculo y una prueba al proyecto del punto.

Reto: añadir un método `Resumen` a `ValidadorContextos` que devuelva el número total de contextos, el número de núcleos, el número de soportes y el número de genéricos, y un test que verifique el cálculo con el catálogo por defecto.

Paso 1: En `ValidadorContextos.cs`, añadir el siguiente `record` al final del archivo:

Archivo: `src/AceriaApp.Api/Ddd/ValidadorContextos.cs`. **Operación:** Modificar.

Punto de inserción: después de este texto exacto, que debe aparecer una sola vez:

```
public record ResultadoContextos(bool EsValido, IReadOnlyList<string> Errores);
```

```
public record ResumenContextos(
    int Total,
    int Nucleos,
    int Soportes,
    int Genericos);
```

Paso 2: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public record ResumenContextos(` declara un contrato inmutable. Número total de contextos. Número de contextos núcleo. Número de contextos de soporte. Número de contextos genéricos.

Paso 3: En `ValidadorContextos.cs`, añadir el siguiente método estático dentro de la clase:

Archivo: `src/AceriaApp.Api/Ddd/ValidadorContextos.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}

public record ResultadoContextos(bool EsValido, IReadOnlyList<string> Errores);
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase indicado por el texto de inserción.

```
public static ResumenContextos Resumen(IEnumerable<ContextoDelimitado> contextos)
{
    var lista = contextos.ToList();
    var nucleos = lista.Count(c => c.TipoDeSubdominio == "Nucleo");
    var soportes = lista.Count(c => c.TipoDeSubdominio == "Soporte");
    var genericos = lista.Count(c => c.TipoDeSubdominio == "Generico");
    return new ResumenContextos(lista.Count, nucleos, soportes, genericos);
}
```

Paso 4: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public static ResumenContextos Resumen(IEnumerable<ContextoDelimitado> c` declara el método. Materializa la lista. Cuenta los núcleos. Cuenta los soportes. Cuenta los genéricos. Devuelve el resumen.

Paso 5: Guardar el archivo con `Ctrl+S`.

Paso 6: En `ValidadorContextosTests.cs`, añadir el siguiente test:

Archivo: `tests/AceriaApp.Ddd.Tests/ValidadorContextosTests.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase indicado por el texto de inserción.

```
[Fact]
public void Resumen_ConCatalogoPorDefecto_CuentaPorTipo()
{
    var resumen = ValidadorContextos.Resumen(CatalogoContextos.ContextosPorDefecto());

    resumen.Total.Should().Be(4);
    resumen.Nucleos.Should().Be(1);
    resumen.Soportes.Should().Be(2);
    resumen.Genericos.Should().Be(1);
}
```

Paso 7: Explicar línea por línea:

Marca el método como test de xUnit. Declara el test. Invoca el método. Verifica el total. Verifica los núcleos. Verifica los soportes. Verifica los genéricos.

Paso 8: Ejecuta `dotnet test tests/AceriaApp.Ddd.Tests/AceriaApp.Ddd.Tests.csproj -c Release` en la copia optativa. Deben pasar 17 tests del proyecto, incluido el nuevo resumen. Ejecuta después `dotnet test AceriaApp.sln -c Release` y exige 135 pruebas aprobadas.

Paso 9: Comprueba que solo cambian `ValidadorContextos.cs` y `ValidadorContextosTests.cs` en la copia optativa; la etapa obligatoria `5.1-conceptos-ddd` y el punto anterior permanecen idénticos.

Paso 10: Hacer commit con `git commit -m "Reto 5.1: método Resumen en ValidadorContextos"`.

Verificación: `Passed: 17` en el proyecto del punto y 135 pruebas aprobadas en la solución de la copia, cero fallos y cero omisiones.

Resultado verificable

Al finalizar este punto, el alumno debe tener:

- La carpeta `M05_DDD_CQRS/PROYECTO/5.1-conceptos-ddd/` con la solución `AceriaApp.sln` que contiene 16 proyectos.
- El documento `docs/conceptos-ddd.md` con la definición y los contextos.
- El documento `docs/ADR-019-conceptos-ddd.md` con la decisión sobre DDD.
- Las clases `ContextoDelimitado`, `ValidadorContextos` y `CatalogoContextos` en `src/AceriaApp.Api/Ddd/`.
- El proyecto `tests/AceriaApp.Ddd.Tests` añade 11 tests de catálogo y 5 funcionales; el reto añade uno más.
- `dotnet test AceriaApp.sln -c Release` ejecuta 134 pruebas en la etapa obligatoria y 135 en una copia con su reto; ninguna falla ni se omite.

- Los ADR-010 a ADR-018 del punto 4.5 sin modificar.
- Un commit revisado para 5.1, con el historial previo conservado.

Continuidad del aprendizaje

El punto 5.1 ha establecido los fundamentos de Domain-Driven Design: el dominio como centro del diseño, los contextos delimitados como fronteras del modelo, el lenguaje ubicuo como vocabulario compartido y la alineación entre contextos y servicios. El proyecto AceriaApp dispone ahora de un validador automático de contextos delimitados y 11 tests de catálogo más 5 funcionales nuevos. Esta base permite avanzar hacia el punto 5.2, donde se estudiarán las entidades y los value objects como elementos tácticos del modelo del dominio.

Punto 5.2 — Entidades y value objects

Proteger identidad, valor e invariantes dentro del agregado

Una entidad sigue siendo la misma aunque cambien algunos atributos; un value object se compara por los valores que lo definen. Esa diferencia tiene consecuencias prácticas cuando una orden cambia de estado, agrega líneas o se recupera de una base de datos. En la etapa 5.2 estudiaremos qué operaciones tienen sentido para el agregado de Producción y qué estados deben rechazarse antes de persistir.

La práctica añade elementos del catálogo didáctico, pero su núcleo es el modelo ejecutable: la raíz controla las modificaciones y evita que una operación externa salte sus invariantes mediante setters indiscriminados. La igualdad de entidades y la igualdad de valores necesitan pruebas distintas. Utilizaremos el mismo principio para distinguir una representación válida de un modelo que realmente impide cantidades o transiciones incorrectas. Las pruebas de dominio son importantes, aunque no reemplazan un recorrido SQL real.

Carpeta de trabajo: `M05_DDD_CQRS/PROYECTO/5.2-entidades-value-objects`.

Objetivo: completar los veinticuatro pasos, conservar los documentos heredados y comprobar el modelo con código y tests ejecutables. La etapa suma 153 pruebas en 17 proyectos.

Ejercicio guiado. Añadir al punto 5.1 un documento de entidades y value objects en `docs/`, una clase de validación de entidades y value objects en el proyecto API y un test unitario que verifique la validación. El punto 5.2 parte del contenido del punto 5.1 conservando la carpeta anterior y los documentos heredados; los cambios de integración se indican explícitamente en esta copia. La implementación del paso 16 añade comportamiento ejecutable del contexto Producción al ejercicio de catálogo.

PASO 5.2.1 — Crear la carpeta del punto 5.2 a partir del punto 5.1

La copia conserva una etapa autónoma. 5.1-conceptos-ddd aporta 134 tests aprobados y 16 proyectos antes de incorporar este punto.

1. Abre la terminal integrada y localiza `M05_DDD_CQRS/PROYECTO/5.1-conceptos-ddd`; esa es la etapa completa que vas a continuar.
2. Crea `M05_DDD_CQRS/PROYECTO/5.2-entidades-value-objects` y copia en ella el contenido de `5.1-conceptos-ddd`; en Windows puedes usar `xcopy` con `/E /I /H /Y`.
3. Excluye `bin`, `obj` y `TestResults`; abre únicamente la copia de `5.2-entidades-value-objects` para conservar intacto el punto anterior.

4. Ejecuta `dir` dentro de `5.2-entidades-value-objects` y verifica `AceriaApp.sln`, `README.md`, `src`, `tests`, `docs` y `global.json`.

Verificación concreta. La copia contiene la solución y los 16 proyectos anteriores; todavía no contiene los archivos de 5.2.

Diagnóstico si falla.

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.

PASO 5.2.2 — Abrir la solución copiada en Visual Studio con soporte de .NET 8

Abrir la solución copiada permite trabajar en el punto 5.2 sobre la base del punto 5.1. La compilación verifica que la copia es correcta.

1. Abre `AceriaApp.sln` dentro de `5.2-entidades-value-objects` con Visual Studio; en Windows puedes ejecutar `start AceriaApp.sln`.
2. Ejecuta `dotnet sln AceriaApp.sln list` y comprueba los 16 proyectos heredados; todas sus rutas deben existir en esta etapa.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release` para comprobar el punto de partida.
4. Si hay DLL bloqueadas por una API en ejecución, detén esa aplicación y repite la compilación. Continúa cuando no haya errores.

Verificación concreta. La solución muestra 16 proyectos y `dotnet build AceriaApp.sln -c Release` termina con `Build succeeded.`.

Diagnóstico si falla.

- Abrir la solución del punto 5.1 en lugar de la del punto 5.2: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener la API del punto 5.1 en ejecución: el puerto queda ocupado y la compilación falla.

PASO 5.2.3 — Verificar que la carpeta `docs` contiene los documentos del punto 5.1

La nueva etapa preserva 20 documentos y añade su ficha temática y su ADR. Los cambios de integración se aplican únicamente en la copia actual.

1. Expande `docs` dentro de la copia de `5.2-entidades-value-objects`.
2. Comprueba los 20 documentos heredados: ADR-010 a ADR-019 y fichas de los puntos anteriores.
3. Compara sus nombres y contenido con `docs` de `5.1-conceptos-ddd`; no modifiques ni elimines documentos anteriores.
4. Confirma que todavía no existen `docs/entidades-value-objects.md` ni `docs/ADR-020-entidades-value-objects.md`.

Verificación concreta. Existen 20 documentos heredados idénticos y no existen todavía los dos de este punto.

Diagnóstico si falla.

- Modificar alguno de los ADR o documentos existentes: rompe la conservación de la etapa anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 5.1.

- Crear la carpeta `docs` si ya existe: `mkdir` falla.

PASO 5.2.4 — Crear el documento de entidades y value objects en `docs`

El documento de entidades y value objects es el artefacto de `docs` específico del punto 5.2. No modifica los 20 documentos existentes del punto 5.1.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `entidades-value-objects.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que el archivo aparece en la carpeta `docs` sin subrayados rojos.

Archivo: `docs/entidades-value-objects.md`. **Operación:** Crear.

```
# Entidades y Value Objects — Punto 5.2

## Entidad

Objeto del dominio que se distingue por su identidad, no por sus atributos.

## Identidad

- Natural: número de bastidor, DNI, ISBN.
- Asignada: número de factura, código de pedido.
- Generada: Guid, ULID, Snowflake ID.

## Value object

Objeto del dominio que se compara por sus atributos, sin identidad propia. Es inmutable y se sustituye por
otro valor cuando cambia el concepto representado.

## Comparación

| Aspecto | Entidad | Value object |
|---------|---------|--------------|
| Identidad | Propia | No tiene |
| Mutabilidad | Depende del modelo; no define la identidad | Inmutable |
| Ciclo de vida | Sí | No |
| Igualdad | Por identidad | Por atributos |

## Invariantes

Condiciones que deben mantenerse en todo estado observable. Se validan al construir el objeto y en cada
operación que cambia su estado.

## Composición

Los value objects se componen para formar estructuras más complejas.

## Persistencia

- Entidades: clave primaria.
- Value objects: mapeados como parte de la entidad.

## Criterios de decisión

- Identidad → entidad.
- La mutabilidad puede aparecer en entidades, pero no las define.
- Ciclo de vida → entidad.
- Sin identidad → value object.
- Sin ciclo de vida → value object.

## Errores comunes

- Entidad para concepto sin identidad.
- Value object para concepto con identidad.
- Value object mutable.
- Rechazar una entidad solo por ser inmutable: la identidad determina su categoría.
- Invariantes sin validar.
```

```
## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Entidades ejecutables | OrdenProduccion y LineaDeOrden |
| Value objects ejecutables | NombreMaterial y CantidadProduccion |
| Identidad | Guid en las entidades |
| Inmutabilidad | Valores sin setters públicos y construcción controlada |
| Invariantes | Construcción, capacidad total y operaciones del agregado |
| Validación | Test de coherencia |

## Alcance del catálogo y del modelo

El catálogo conserva los ejemplos Orden, Material, Cliente, Dinero, Direccion y Cantidad para practicar la
clasificación. No implica que esos seis tipos estén implementados como entidades o valores de Producción.
`ValidadorModeloDominio` exige identidad para las entidades e inmutabilidad para los value objects; no
rechaza una entidad por ser inmutable. El código ejecutable usa `OrdenProduccion`, `LineaDeOrden`,
`NombreMaterial` y `CantidadProduccion`, cuya identidad, igualdad y capacidad se verifican con pruebas
funcionales. La capacidad de 500 pertenece a este ejercicio y no representa una regla universal del sector.
```

Verificación concreta. El archivo `docs/entidades-value-objects.md` existe; comprueba que sus títulos, contenido y enlaces coinciden con todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: rompe la conservación de la etapa anterior.
- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

PASO 5.2.5 — Crear la carpeta `ModeloDominio` en el proyecto API

La carpeta `ModeloDominio` albergará las clases de validación de entidades y value objects. Se crea como carpeta nueva sin tocar las carpetas existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.Api`.
2. Seleccionar `Add` → `New Folder` y nombrar la carpeta `ModeloDominio`.
3. Verificar que la carpeta `ModeloDominio` aparece dentro de `AceriaApp.Api` sin afectar a las carpetas existentes.
4. Verificar que no se ha modificado ningún archivo existente del punto 5.1.

Verificación concreta. la carpeta `ModeloDominio` existe dentro de `AceriaApp.Api` y las carpetas existentes permanecen intactas.

Diagnóstico si falla.

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: rompe la conservación de la etapa anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

PASO 5.2.6 — Crear la clase `ElementoModelo`

`ElementoModelo` permite clasificar propuestas como entidad o value object. Su nombre y sus propiedades se utilizan en el catálogo; la igualdad y los invariantes del agregado se prueban con las clases propias de Producción.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `ModeloDominio`.
2. Seleccionar `Add` → `Class` y nombrar la clase `ElementoModelo.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/ModeloDominio/ElementoModelo.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.ModeloDominio;

public class ElementoModelo
{
    public string Nombre { get; init; } = string.Empty;
    public string Tipo { get; init; } = string.Empty;
    public bool TieneIdentidad { get; init; }
    public bool EsInmutable { get; init; }
    public bool ValidaInvariantes { get; init; }
}
```

Verificación concreta. la clase `ElementoModelo` compila sin errores y expone cinco propiedades.

Diagnóstico si falla.

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar las propiedades de tipo `string` con `string.Empty`: el compilador emite advertencias.
- Usar `set` en lugar de `init`: permite modificar la instancia después de crearla.

PASO 5.2.7 — Crear la clase `ValidadorModeloDominio`

`ValidadorModeloDominio` comprueba categorías y propiedades declaradas, pero un objeto aceptado no garantiza que la raíz de Producción impida una transición inválida. Mantén separadas ambas clases de pruebas.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `ModeloDominio`.
2. Seleccionar `Add` → `Class` y nombrar la clase `ValidadorModeloDominio.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Alcance del validador. Comprueba los indicadores de los ejemplos del catálogo. No analiza por reflexión los tipos de producción ni garantiza por sí solo la arquitectura. Las reglas del modelo ejecutable y sus límites se comprueban en las pruebas funcionales del paso 16.

Archivo: `src/AceriaApp.Api/ModeloDominio/ValidadorModeloDominio.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.ModeloDominio;

public class ValidadorModeloDominio
{
    private static readonly string[] TiposPermitidos = { "Entidad", "ValueObject" };

    public ResultadoModelo Validar(IEnumerable<ElementoModelo> elementos)
    {
        var errores = new List<string>();
        var lista = elementos.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay elementos declarados.");
        }

        foreach (var elemento in lista)
        {
            if (string.IsNullOrWhiteSpace(elemento.Nombre))
            {
                errores.Add("Un elemento no declara el nombre.");
            }

            if (!TiposPermitidos.Contains(elemento.Tipo))
            {
                errores.Add($"Tipo no permitido en '{elemento.Nombre}': '{elemento.Tipo}'.");
            }
```

```
            if (elemento.Tipo == "Entidad" && !elemento.TieneIdentidad)
            {
                errores.Add($"La entidad '{elemento.Nombre}' no tiene identidad.");
            }

            if (elemento.Tipo == "ValueObject" && elemento.TieneIdentidad)
            {
                errores.Add($"El value object '{elemento.Nombre}' no debería tener identidad.");
            }

            if (elemento.Tipo == "ValueObject" && !elemento.EsInmutable)
            {
                errores.Add($"El value object '{elemento.Nombre}' no es inmutable.");
            }

            if (!elemento.ValidaInvariantes)
            {
                errores.Add($"El elemento '{elemento.Nombre}' no valida invariantes.");
            }
        }

        return new ResultadoModelo(errores.Count == 0, errores);
    }
}

public record ResultadoModelo(bool EsValido, IReadOnlyList<string> Errores);
```

Verificación concreta. la clase `ValidadorModeloDominio` compila sin errores y expone un método y un `record`.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<string>` e `IReadOnlyList<string>` no se resuelven.
- Olvidar el `using System.Linq;`: `Contains` y `ToList` no se resuelven.
- No comprobar el caso de lista vacía: el validador devuelve válido sin elementos.

PASO 5.2.8 — Añadir los `using` necesarios en `ValidadorModeloDominio`

Los tipos `List<string>` e `IReadOnlyList<string>` pertenecen a `System.Collections.Generic`. Los métodos `Contains` y `ToList` pertenecen a `System.Linq`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. En el panel Solution Explorer, hacer doble clic en `ValidadorModeloDominio.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; el texto de inserción debe aparecer exactamente una vez.
3. Verificar que no hay subrayados rojos en `List<string>`, `Contains` ni `ToList`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/ModeloDominio/ValidadorModeloDominio.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.ModeloDominio;
```

```
using System.Collections.Generic;
using System.Linq;
```

Verificación concreta. `ValidadorModeloDominio.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir solo `System.Collections.Generic` y olvidar `System.Linq`: `Contains` y `ToList` no se resuelven.

- Mezclar directivas using con declaraciones de tipos: coloca las directivas antes del namespace para mantener el orden del archivo.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 5.2.9 — Crear la clase `CatalogoModelo`

`CatalogoModelo` facilita comparar elementos descritos en el ejercicio. Cuando examines una línea de producción real, la prueba decisiva será que no permite cantidades negativas y que el agregado mantiene consistencia al persistirse.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `ModeloDominio`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoModelo.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/ModeloDominio/CatalogoModelo.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.ModeloDominio;

public class CatalogoModelo
{
    public static List<ElementoModelo> ElementosPorDefecto()
    {
        return new List<ElementoModelo>
        {
            new() { Nombre = "Orden", Tipo = "Entidad", TieneIdentidad = true, EsInmutable = false,
ValidaInvariantes = true },
            new() { Nombre = "Material", Tipo = "Entidad", TieneIdentidad = true, EsInmutable = false,
ValidaInvariantes = true },
            new() { Nombre = "Cliente", Tipo = "Entidad", TieneIdentidad = true, EsInmutable = false,
ValidaInvariantes = true },
            new() { Nombre = "Dinero", Tipo = "ValueObject", TieneIdentidad = false, EsInmutable = true,
ValidaInvariantes = true },
            new() { Nombre = "Direccion", Tipo = "ValueObject", TieneIdentidad = false, EsInmutable = true,
ValidaInvariantes = true },
            new() { Nombre = "Cantidad", Tipo = "ValueObject", TieneIdentidad = false, EsInmutable = true,
ValidaInvariantes = true }
        };
    }
}
```

Verificación concreta. la clase `CatalogoModelo` compila sin errores y expone un método estático.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<T>` no se resuelve.
- No inicializar las propiedades de los objetos: el compilador emite advertencias.
- Reutilizar una lista estática mutable: cada llamada del catálogo debe construir una lista independiente.

PASO 5.2.10 — Añadir el `using` necesario en `CatalogoModelo`

El tipo `List<T>` pertenece a `System.Collections.Generic`. En proyectos con `ImplicitUsings` desactivado, hay que añadir el using explícito.

1. En el panel Solution Explorer, hacer doble clic en `CatalogoModelo.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; el texto de inserción debe aparecer exactamente una vez.
3. Verificar que no hay subrayados rojos en `List<T>`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/ModeloDominio/CatalogoModelo.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.ModeloDominio;
```

```
using System.Collections.Generic;
```

Verificación concreta. `CatalogoModelo.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir `using System.Linq;` en lugar de `System.Collections.Generic`: no resuelve `List<T>`.
- Colocar el using después de una clase: las directivas deben preceder las declaraciones de tipos.
- Olvidar el using y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 5.2.11 — Crear el proyecto de tests del punto 5.2

El punto 5.2 añade un proyecto de tests específico para las clases del modelo del dominio. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

1. Crea `tests/AceriaApp.ModeloDominio.Tests` y su archivo `AceriaApp.ModeloDominio.Tests.csproj` con el XML completo. Si utilizas la plantilla xUnit de Visual Studio, ajusta el archivo generado.
2. Elimina `UnitTest1.cs` si lo creó la plantilla; las pruebas completas se añaden en los pasos 14 y 15.
3. Ejecuta `dotnet sln AceriaApp.sln add tests/AceriaApp.ModeloDominio.Tests/AceriaApp.ModeloDominio.Tests.csproj` incluso si la solución ya existe.
4. Ejecuta `dotnet sln AceriaApp.sln list` y `dotnet restore tests/AceriaApp.ModeloDominio.Tests/AceriaApp.ModeloDominio.Tests.csproj`.

Archivo: `tests/AceriaApp.ModeloDominio.Tests/AceriaApp.ModeloDominio.Tests.csproj`. **Operación:** Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. La solución muestra 17 proyectos, incluido `AceriaApp.ModeloDominio.Tests`; el csproj fija .NET 8, SDK de tests 17.12.0, xUnit 2.9.2 y runner 2.8.2.

Diagnóstico si falla.

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 5.1: rompe la conservación de la etapa anterior.
- Usar `dotnet new mstest`: el curso usa xUnit.

PASO 5.2.12 — Añadir la referencia del proyecto de tests a la API

La referencia permite que el proyecto de tests acceda a las clases de `ModeloDominio` del proyecto API.

1. Ejecuta `dotnet add tests/AceriaApp.ModeloDominio.Tests/AceriaApp.ModeloDominio.Tests.csproj reference src/AceriaApp.Api/AceriaApp.Api.csproj` o deja el archivo como el XML completo.

2. Comprueba que la referencia va del proyecto de tests a la API; la API no debe depender de los tests.
3. Abre `tests/AceriaApp.ModeloDominio.Tests/AceriaApp.ModeloDominio.Tests.csproj` y confirma la ruta `../../src/AceriaApp.Api/AceriaApp.Api.csproj`.
4. Ejecuta `dotnet build tests/AceriaApp.ModeloDominio.Tests/AceriaApp.ModeloDominio.Tests.csproj -c Release` y verifica que resuelve la API.

Archivo: `tests/AceriaApp.ModeloDominio.Tests/AceriaApp.ModeloDominio.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. el archivo `AceriaApp.ModeloDominio.Tests.csproj` contiene un `ProjectReference` al proyecto API.

Diagnóstico si falla.

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

PASO 5.2.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Ejecuta `dotnet add tests/AceriaApp.ModeloDominio.Tests/AceriaApp.ModeloDominio.Tests.csproj package FluentAssertions --version 6.12.2` o escribe el XML completo.
2. Ejecuta `dotnet list tests/AceriaApp.ModeloDominio.Tests/AceriaApp.ModeloDominio.Tests.csproj package` y localiza `FluentAssertions 6.12.2`.
3. Conserva la referencia a la API y las versiones fijadas de xUnit.
4. Ejecuta `dotnet build tests/AceriaApp.ModeloDominio.Tests/AceriaApp.ModeloDominio.Tests.csproj -c Release`; las aserciones `Should()` estarán disponibles.

Archivo: `tests/AceriaApp.ModeloDominio.Tests/AceriaApp.ModeloDominio.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
```

```
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. El archivo y `dotnet list package` muestran FluentAssertions 6.12.2; la compilación termina correctamente.

Diagnóstico si falla.

- Instalar una versión preliminar: el curso usa la versión fijada en el archivo de proyecto.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

PASO 5.2.14 — Crear el archivo `ValidadorModeloDominioTests.cs`

Los tests verifican que el validador acepta el catálogo por defecto y rechaza los elementos que violan las reglas del modelo del dominio.

1. En el panel Solution Explorer, expandir `tests` → `AceriaApp.ModeloDominio.Tests`.
2. Crear el archivo `ValidadorModeloDominioTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.ModeloDominio.Tests/ValidadorModeloDominioTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.ModeloDominio;
using FluentAssertions;
using Xunit;

namespace AceriaApp.ModeloDominio.Tests;

public class ValidadorModeloDominioTests
{
    private readonly ValidadorModeloDominio _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoModelo.ElementosPorDefecto());

        resultado.EsValido.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinElementos_DevuelveError()
    {
        var resultado = _validador.Validar(new List<ElementoModelo>());

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay elementos"));
    }

    [Fact]
    public void Validar_TipoNoPermitido_DevuelveError()
    {
        var elementos = new List<ElementoModelo>
        {
            new() { Nombre = "X", Tipo = "TipoInventado", TieneIdentidad = true }
        };

        var resultado = _validador.Validar(elementos);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Tipo no permitido"));
    }

    [Fact]
```

```
    public void Validar_EntidadSinIdentidad_DevuelveError()
    {
        var elementos = new List<ElementoModelo>
        {
            new() { Nombre = "X", Tipo = "Entidad", TieneIdentidad = false, ValidaInvariantes = true }
        };

        var resultado = _validador.Validar(elementos);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene identidad"));
    }

    [Fact]
    public void Validar_ValueObjectConIdentidad_DevuelveError()
    {
        var elementos = new List<ElementoModelo>
        {
            new() { Nombre = "X", Tipo = "ValueObject", TieneIdentidad = true, EsInmutable = true,
ValidaInvariantes = true }
        };

        var resultado = _validador.Validar(elementos);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no debería tener identidad"));
    }

    [Fact]
    public void Validar_ValueObjectMutable_DevuelveError()
    {
        var elementos = new List<ElementoModelo>
        {
            new() { Nombre = "X", Tipo = "ValueObject", TieneIdentidad = false, EsInmutable = false,
ValidaInvariantes = true }
        };

        var resultado = _validador.Validar(elementos);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no es inmutable"));
    }

    [Fact]
    public void Validar_SinValidarInvariantes_DevuelveError()
    {
        var elementos = new List<ElementoModelo>
        {
            new() { Nombre = "X", Tipo = "Entidad", TieneIdentidad = true, ValidaInvariantes = false }
        };

        var resultado = _validador.Validar(elementos);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no valida invariantes"));
    }
}
```

Verificación concreta. el archivo `ValidadorModeloDominioTests.cs` compila y contiene siete métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.ModeloDominio;`: las clases no se resuelven.
- Usar `Assert.True` en lugar de `Should().BeTrue()`: el curso usa FluentAssertions.
- No cubrir el caso de value object mutable: la regla específica no se verifica.

PASO 5.2.15 — Crear el archivo `CatalogoModeloTests.cs`

Los tests verifican que el catálogo por defecto contiene los elementos esperados y la distribución correcta por tipo.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.ModeloDominio.Tests`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoModeloTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.ModeloDominio.Tests/CatalogoModeloTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.ModeloDominio;
using FluentAssertions;
using Xunit;

namespace AceriaApp.ModeloDominio.Tests;

public class CatalogoModeloTests
{
    [Fact]
    public void ElementosPorDefecto_ContieneSeisElementos()
    {
        var elementos = CatalogoModelo.ElementosPorDefecto();

        elementos.Should().HaveCount(6);
    }

    [Fact]
    public void ElementosPorDefecto_ContieneTresEntidades()
    {
        var elementos = CatalogoModelo.ElementosPorDefecto();

        elementos.Count(e => e.Tipo == "Entidad").Should().Be(3);
    }

    [Fact]
    public void ElementosPorDefecto_ContieneTresValueObjects()
    {
        var elementos = CatalogoModelo.ElementosPorDefecto();

        elementos.Count(e => e.Tipo == "ValueObject").Should().Be(3);
    }

    [Fact]
    public void ElementosPorDefecto_TodasLasEntidadesTienenIdentidad()
    {
        var elementos = CatalogoModelo.ElementosPorDefecto();

        elementos.Where(e => e.Tipo == "Entidad").Should().OnlyContain(e => e.TieneIdentidad);
    }

    [Fact]
    public void ElementosPorDefecto_TodosLosValueObjectsSonInmutables()
    {
        var elementos = CatalogoModelo.ElementosPorDefecto();

        elementos.Where(e => e.Tipo == "ValueObject").Should().OnlyContain(e => e.EsInmutable);
    }
}
```

Verificación concreta. el archivo `CatalogoModeloTests.cs` compila y contiene cinco métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.ModeloDominio;`: las clases no se resuelven.
- Olvidar el `using System.Linq;`: `Count`, `Where` y `OnlyContain` no se resuelven.
- No verificar la inmutabilidad de los value objects: la regla no se comprueba.

PASO 5.2.16 — Implementar identidad, igualdad por valor e invariantes

Las modificaciones de dominio deben pasar por operaciones que protejan las invariantes. Comprueba la identidad de una orden tras cambios permitidos, la igualdad por valor de los objetos apropiados y el rechazo de cantidades o estados incompatibles; una entidad sin setters públicos no basta si sus métodos dejan escapar estados inválidos.

1. Lee cada archivo de integración de este paso y localiza su responsabilidad en las capas del contexto Producción.
2. Crea o modifica los archivos exactamente como se indica; sigue la lectura numerada para conectar contratos, reglas y pruebas.
3. Conserva todos los proyectos de la solución y comprueba las referencias indicadas en cada archivo de proyecto.
4. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`; `dotnet sln AceriaApp.sln list` debe mostrar 17 proyectos.

Integración funcional. Implementa la entidad hija con identidad propia; su creación y sus cambios quedan bajo control de la raíz.

Archivo: `src/AceriaApp.Produccion.Dominio/LineaDeOrden.cs`. **Operación:** Crear.

```
namespace AceriaApp.Produccion.Dominio;

public sealed class LineaDeOrden : IEquatable<LineaDeOrden>
{
    public Guid Id { get; private set; }
    public NombreMaterial Material { get; private set; } = null!;
    public CantidadProduccion Cantidad { get; private set; } = null!;

    private LineaDeOrden() { } // Materialización; ningún consumidor crea una línea sin la raíz.
    internal LineaDeOrden(NombreMaterial material, CantidadProduccion cantidad)
    {
        Id = Guid.NewGuid();
        Material = material;
        Cantidad = cantidad;
    }
    internal void CambiarCantidad(CantidadProduccion cantidad) => Cantidad = cantidad;
    public bool Equals(LineaDeOrden? otra) => otra is not null && Id == otra.Id;
    public override bool Equals(object? otra) => otra is LineaDeOrden linea && Equals(linea);
    public override int GetHashCode() => Id.GetHashCode();
}
```

Impide crear entidades hijas desde la capa de aplicación: la raíz controla su incorporación. La raíz llama a esta operación solo después de validar el total y el estado. Dos líneas con el mismo material y cantidad siguen siendo entidades distintas.

Integración funcional. Modela la raíz de Producción con identidad estable, transiciones explícitas y reglas de capacidad; desde 5.2 controla todas las mutaciones de sus líneas.

Archivo: `src/AceriaApp.Produccion.Dominio/OrdenProduccion.cs`. **Operación:** Reemplazar el contenido completo.

```
namespace AceriaApp.Produccion.Dominio;

public sealed class OrdenProduccion : IEquatable<OrdenProduccion>
{
    public const int CapacidadMaxima = 500;
    private readonly List<LineaDeOrden> _lineas = new();
    public Guid Id { get; private set; }
    public string Cliente { get; private set; } = "";
    public EstadoOrdenProduccion Estado { get; private set; }
    public IReadOnlyList<LineaDeOrden> Lineas => _lineas.AsReadOnly();
    public int CantidadTotal => _lineas.Sum(x => x.Cantidad.Valor);
    // Compatibilidad con el primer ejemplo de 5.1; la raíz calcula el total.
    public int Cantidad => CantidadTotal;
```

```
    public string Material => _lineas[0].Material.Valor;

    private OrdenProduccion() { }
    public OrdenProduccion(string cliente, string material, int cantidad)
    {
        if (string.IsNullOrWhiteSpace(cliente) || cliente.Trim().Length > 100)
            throw new ReglaProduccionException("cliente.invalido", "El cliente debe tener entre 1 y 100
caracteres.");
        Id = Guid.NewGuid();
        Cliente = cliente.Trim();
        Estado = EstadoOrdenProduccion.Borrador;
        AgregarLinea(material, cantidad);
    }

    public void AgregarLinea(string material, int cantidad)
    {
        ExigirBorrador();
        var nombre = new NombreMaterial(material);
        var valor = new CantidadProduccion(cantidad);
        ExigirCapacidad(CantidadTotal + valor.Valor);
        _lineas.Add(new LineaDeOrden(nombre, valor));
    }

    public void CambiarCantidadLinea(Guid idLinea, int cantidad)
    {
        ExigirBorrador();
        var linea = _lineas.SingleOrDefault(x => x.Id == idLinea)
            ?? throw new ReglaProduccionException("linea.inexistente", "La línea no pertenece a la orden.");
        var nueva = new CantidadProduccion(cantidad);
        ExigirCapacidad(CantidadTotal - linea.Cantidad.Valor + nueva.Valor);
        linea.CambiarCantidad(nueva);
    }

    public void Planificar()
    {
        ExigirBorrador();
        Estado = EstadoOrdenProduccion.Planificada;
    }
    public void Finalizar()
    {
        if (Estado != EstadoOrdenProduccion.Planificada)
            throw new ReglaProduccionException("estado.invalido", "Solo se puede finalizar una orden
planificada.");
        Estado = EstadoOrdenProduccion.Finalizada;
    }
    private void ExigirBorrador()
    {
        if (Estado != EstadoOrdenProduccion.Borrador)
            throw new ReglaProduccionException("estado.invalido", "La operación requiere una orden en
borrador.");
    }
    private static void ExigirCapacidad(int total)
    {
        if (total > CapacidadMaxima)
            throw new ReglaProduccionException("capacidad.excedida", "El total de la orden no puede superar
500 unidades.");
    }
    public bool Equals(OrdenProduccion? otra) => otra is not null && Id == otra.Id;
    public override bool Equals(object? otra) => otra is OrdenProduccion orden && Equals(orden);
    public override int GetHashCode() => Id.GetHashCode();
}
```

Fija la capacidad didáctica por orden, no una cuota global entre órdenes. Devuelve una vista no modificable: ningún consumidor puede insertar o borrar líneas fuera de la raíz. Deriva el total de todas las líneas para evitar un contador independiente que pueda quedar desactualizado.

Valida el cliente normalizado antes de crear un agregado o llegar a la persistencia. Valida la suma antes de añadir la línea; un rechazo no cambia la colección. Comprueba el nuevo total antes de sustituir la cantidad de una línea existente.

Impide finalizar directamente desde Borrador o volver a finalizar una orden. Compara entidades por identidad estable; los datos iguales no convierten dos órdenes distintas en una sola.

Integración funcional. Implementa value objects inmutables, con normalización, validación e igualdad por valor; las clases selladas evitan default(struct) inválidos.

Archivo: `src/AceriaApp.Produccion.Dominio/ValoresProduccion.cs`. **Operación:** Crear.

```
namespace AceriaApp.Produccion.Dominio;

// Clases record selladas: igualdad por valor, sin default(struct) inválido ni init público.
public sealed record NombreMaterial
{
    public string Valor { get; }
    public NombreMaterial(string valor)
    {
        if (string.IsNullOrWhiteSpace(valor) || valor.Trim().Length > 100)
            throw new ReglaProduccionException("material.invalido", "El material debe tener entre 1 y 100
caracteres.");
        Valor = valor.Trim();
    }
}

public sealed record CantidadProduccion
{
    public int Valor { get; }
    public CantidadProduccion(int valor)
    {
        if (valor is < 1 or > OrdenProduccion.CapacidadMaxima)
            throw new ReglaProduccionException("cantidad.invalida", "La cantidad debe estar entre 1 y 500
unidades.");
        Valor = valor;
    }
}
```

El tipo sellado obtiene igualdad por valor sin abrir una jerarquía que cambie esa semántica. El getter carece de setter e init público: tampoco un with puede introducir un valor inválido. Rechaza valores vacíos o demasiado largos antes de normalizar.

Usa una clase en lugar de struct para evitar una instancia default con cero sin pasar por validación. Valida la cantidad individual; la raíz valida además la suma del agregado.

Integración funcional. Configura las referencias y versiones de este proyecto manteniendo los proyectos y catálogos heredados.

Archivo: `tests/AceriaApp.ModeloDominio.Tests/AceriaApp.ModeloDominio.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsPackable>false</IsPackable><IsTestProject>true</IsTestProject></PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <ProjectReference Include="../../src/AceriaApp.Produccion.Dominio/AceriaApp.Produccion.Dominio.csproj" />

  </ItemGroup>
</Project>
```

Fija la versión del paquete usado por esta capa o por sus pruebas para hacer reproducible la compilación. Fija la versión del paquete usado por esta capa o por sus pruebas para hacer reproducible la compilación. Fija la versión del paquete usado por esta capa o por sus pruebas para hacer reproducible la compilación.

Fija la versión del paquete usado por esta capa o por sus pruebas para hacer reproducible la compilación. Declara una dependencia de proyecto que el compilador necesita; no registra servicios en el contenedor. Declara una dependencia de proyecto que el compilador necesita; no registra servicios en el contenedor.

Integración funcional. Añade 7 pruebas funcionales del punto 5.2, además del catálogo original; verifica comportamiento real y regresiones heredadas.

Archivo: `tests/AceriaApp.ModeloDominio.Tests/ModeloDominioFuncionalesTests.cs`. **Operación:** Crear.

```
using AceriaApp.Produccion.Dominio;
using Xunit;
namespace AceriaApp.ModeloDominio.Tests;
public sealed class ModeloDominioFuncionalesTests
{
    [Fact]
    public void Value_objects_tienen_igualdad_por_valor_normalizado()
    {
        var uno = new NombreMaterial(" Acero ");
        var otro = new NombreMaterial("Acero");
        Assert.Equal(uno, otro);
        Assert.Equal(uno.GetHashCode(), otro.GetHashCode());
        Assert.Single(new HashSet<NombreMaterial> { uno, otro });
        Assert.Equal(new CantidadProduccion(25), new CantidadProduccion(25));
        Assert.NotEqual(new CantidadProduccion(25), new CantidadProduccion(26));
    }
    [Fact]
    public void Value_objects_no_permiten_valores_invalidos()
    {
        Assert.Throws<ReglaProduccionException>(() => new NombreMaterial(null!));
        Assert.Throws<ReglaProduccionException>(() => new NombreMaterial(new string('A', 101)));
        Assert.Throws<ReglaProduccionException>(() => new CantidadProduccion(0));
        Assert.Throws<ReglaProduccionException>(() => new CantidadProduccion(501));
        Assert.Equal(100, new NombreMaterial(new string('A', 100)).Valor.Length);
    }
    [Fact]
    public void Lineas_con_igual_material_y_cantidad_conservan_identidades_distintas()
    {
        var orden = new OrdenProduccion("Axia", "Acero", 20);
        orden.AgregarLinea("Acero", 20);
        Assert.Equal(2, orden.Lineas.Count);
        Assert.NotEqual(orden.Lineas[0].Id, orden.Lineas[1].Id);
        Assert.NotEqual(orden.Lineas[0], orden.Lineas[1]);
        Assert.Equal(orden.Lineas[0].Material, orden.Lineas[1].Material);
    }
    [Fact]
    public void Capacidad_se_evalua_sobre_todo_el_agregado_sin_cambios_parciales()
    {
        var orden = new OrdenProduccion("Axia", "Acero", 300);
        orden.AgregarLinea("Acero inoxidable", 200);
        var error = Assert.Throws<ReglaProduccionException>(() => orden.AgregarLinea("Acero", 1));
        Assert.Equal("capacidad.excedida", error.Codigo);
        Assert.Equal(500, orden.CantidadTotal);
        Assert.Equal(2, orden.Lineas.Count);
    }
    [Fact]
    public void Coleccion_publica_no_permite_saltar_las_reglas_de_la_raiz()
    {
        var orden = new OrdenProduccion("Axia", "Acero", 20);
        var vista = Assert.IsAssignableFrom<ICollection<LineaDeOrden>>(orden.Lineas);
        Assert.True(vista.IsReadOnly);
        Assert.Throws<NotSupportedException>(vista.Clear);
        Assert.Single(orden.Lineas);
    }
    [Fact]
    public void Cambiar_cantidad_preserva_identidad_y_rechaza_total_excesivo()
    {
        var orden = new OrdenProduccion("Axia", "Acero", 200);
        orden.AgregarLinea("Acero inoxidable", 200);
        var idLinea = orden.Lineas[0].Id;
        orden.CambiarCantidadLinea(idLinea, 300);
        Assert.Equal(idLinea, orden.Lineas[0].Id);
        Assert.Equal(500, orden.CantidadTotal);
```

```
        Assert.Throws<ReglaProduccionException>(() => orden.CambiarCantidadLinea(idLinea, 301));
        Assert.Equal(300, orden.Lineas[0].Cantidad.Valor);
        Assert.Throws<ReglaProduccionException>(() => orden.CambiarCantidadLinea(Guid.NewGuid(), 1));
        Assert.Equal(500, orden.CantidadTotal);
    }
    [Fact]
    public void Una_orden_planificada_cierra_la_edicion_de_sus_lineas()
    {
        var orden = new OrdenProduccion("Axia", "Acero", 20);
        orden.Planificar();
        Assert.Throws<ReglaProduccionException>(() => orden.AgregarLinea("Acero", 10));
        Assert.Throws<ReglaProduccionException>(() => orden.CambiarCantidadLinea(orden.Lineas[0].Id, 30));
        orden.Finalizar();
        Assert.Throws<ReglaProduccionException>(() => orden.AgregarLinea("Acero", 10));
        Assert.Equal(20, orden.CantidadTotal);
    }
}
```

Comprueba el comportamiento observable: value objects tienen igualdad por valor normalizado. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos. Comprueba el comportamiento observable: value objects no permiten valores invalidos. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos. Comprueba el comportamiento observable: lineas con igual material y cantidad conservan identidades distintas. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos.

Comprueba el comportamiento observable: capacidad se evalua sobre todo el agregado sin cambios parciales. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos. Comprueba el comportamiento observable: coleccion publica no permite saltar las reglas de la raiz. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos. Comprueba el comportamiento observable: cambiar cantidad preserva identidad y rechaza total excesivo. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos.

Comprueba el comportamiento observable: una orden planificada cierra la edicion de sus lineas. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos.

Verificación concreta. La solución contiene 17 proyectos, los archivos de integración coinciden con los bloques y la compilación completa termina con cero errores.

Diagnóstico si falla.

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.
- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

PASO 5.2.17 — Ejecutar todos los tests del punto 5.2

Las pruebas de catálogo verifican los datos declarados; las pruebas funcionales ejecutan las reglas y los adaptadores reales del contexto. Ambas evidencias son necesarias.

1. Ejecuta `dotnet test tests/AceriaApp.ModeloDominio.Tests/AceriaApp.ModeloDominio.Tests.csproj -c Release` desde la raíz de la etapa.
2. Comprueba 19 pruebas aprobadas: 12 de catálogo y 7 funcionales.
3. Exige cero fallos y cero omisiones; revisa los nombres de las pruebas para identificar el contrato comprobado.
4. Si falla una prueba, lee su aserción y corrige el archivo del paso correspondiente antes de continuar.

Verificación concreta. El proyecto ejecuta 19 tests: `Failed: 0, Passed: 19, Skipped: 0, Total: 19`.

Diagnóstico si falla.

- Ejecutar `dotnet test` sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

PASO 5.2.18 — Ejecutar todos los tests de la solución

Ejecutar todos los tests de la solución verifica que los tests de los puntos anteriores y del punto 5.2 pasan conjuntamente.

1. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build` después de la compilación Release.
2. Comprueba que se ejecutan los 12 proyectos de tests, incluidos los heredados de M04.
3. Suma los resultados por proyecto: deben ser 153 pruebas aprobadas, cero fallos y cero omisiones.
4. Conserva la salida como evidencia; si cambias código vuelve a compilar antes de utilizar `--no-build`.

Verificación concreta. Los 12 proyectos ejecutan conjuntamente 153 tests aprobados, cero fallos y cero omisiones.

Diagnóstico si falla.

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Omitir proyectos heredados de la solución: se pierde la comprobación de regresión.
- Eliminar o modificar pruebas anteriores sin una instrucción explícita: se pierde la cobertura heredada.

PASO 5.2.19 — Actualizar el README de la copia 5.2

El README del punto 5.2 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 5.1.

1. Abre el archivo `README.md` copiado de `5.1-conceptos-ddd` dentro de `5.2-entidades-value-objects`.
2. Reemplaza su contenido completo por el bloque de este paso; el README de la carpeta anterior permanece intacto.
3. Comprueba el objetivo, el estado, la ejecución, los añadidos, la documentación y la verificación ejecutable.
4. Guarda el archivo y abre su vista previa para comprobar títulos, enlaces, rutas y cifras.

Archivo: `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 5.2 — Entidades y Value Objects

## Objetivo
Comprender las entidades y los value objects como elementos tácticos del modelo del dominio y añadir al
proyecto las clases de validación del modelo.

Añadir valores inmutables, identidad y líneas encapsuladas al agregado de Producción.

## Estado del proyecto
- Solución `AceriaApp.sln` con 17 proyectos.
- Documento `docs/entidades-value-objects.md` con la definición y las reglas.
- Clases `ElementoModelo`, `ValidadorModeloDominio` y `CatalogoModelo` en `src/AceriaApp.Api/ModeloDominio/`.
- 12 tests de catálogo y 7 funcionales nuevos en `tests/AceriaApp.ModeloDominio.Tests`.
- Los tests acumulados suman 153 pruebas.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio con soporte de .NET 8.
2. Configurar SQL Server para los adaptadores heredados; las pruebas proporcionan su persistencia de prueba.
3. Configurar las conexiones `AceriaApp` y `AceriaHexagonal` en el entorno local si se ejecuta la API con SQL
Server.
4. Ejecutar `dotnet run --project src/AceriaApp.Api/AceriaApp.Api.csproj --urls http://localhost:5000` desde
```

```
la raíz de la etapa.
5. Navegar a `http://localhost:5000/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/entidades-value-objects.md`.
- Carpeta `ModeloDominio` en el proyecto API con tres clases nuevas.
- Proyecto `tests/AceriaApp.ModeloDominio.Tests` con 12 tests de catálogo y 7 funcionales nuevos.
- Se conserva intacta la carpeta del punto 5.1.

- Añadir valores inmutables, identidad y líneas encapsuladas al agregado de Producción. Los bloques completos
están en el paso 16 del manual.

## Documentación
- [ADR-010](docs/ADR-010-capas-aplicacion.md) a [ADR-019](docs/ADR-019-conceptos-ddd.md) (heredados).
- [Entidades y Value Objects](docs/entidades-value-objects.md).

## Verificación ejecutable

El catálogo describe ejemplos de diseño y sus indicadores; no inspecciona automáticamente las clases reales.
Los archivos funcionales del paso 16 y sus tests comprueban el comportamiento ejecutable. Ejecuta `dotnet
test AceriaApp.sln -c Release` y exige cero fallos y cero omisiones.
```

Verificación concreta. El archivo `README.md` existe; comprueba que sus títulos, contenido y enlaces coinciden con todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Modificar el README del punto 5.1: rompe la conservación de la etapa anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 5.1: rompe la separación.

PASO 5.2.20 — Crear el ADR-020 del punto 5.2

El ADR-020 documenta la decisión de adoptar la distinción entre entidades y value objects. Es un artefacto nuevo del punto 5.2 que no modifica los ADR existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `ADR-020-entidades-value-objects.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que los ADR-010 a ADR-019 no se han modificado.

Archivo: `docs/ADR-020-entidades-value-objects.md`. **Operación:** Crear.

```
# ADR-020: Entidades y Value Objects

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado Domain-Driven Design y ha definido contextos delimitados. Es necesario
definir los elementos tácticos del modelo del dominio.

## Decisión
Se distingue identidad de igualdad por valor. `OrdenProduccion` y `LineaDeOrden` son entidades con identidad;
`NombreMaterial` y `CantidadProduccion` son valores inmutables. La raíz controla las líneas, los cambios de
estado y una capacidad total máxima de 500 unidades del ejercicio. Las invariantes se comprueban al construir
y al cambiar estado. Ser mutable no es requisito universal de una entidad. El catálogo utiliza ejemplos de
entidades mutables y valores inmutables; el validador comprueba identidad e invariantes sin exigir
mutabilidad a las entidades. Las pruebas funcionales comprueban identidad, igualdad, validación y
encapsulación del agregado.

## Alternativas consideradas
- Todo entidades: descartado por añadir identidad innecesaria.
- Todo value objects: descartado por no modelar el ciclo de vida.
- Modelo plano sin distinción: descartado por falta de expresividad.

## Consecuencias
```

```
- Positivas: modelo expresivo, validación clara, persistencia adecuada.
- Negativas: la distinción requiere análisis por cada concepto.
- Mitigación: la validación automática y la revisión del modelo.

## Alcance del catálogo y del modelo

El catálogo conserva los ejemplos Orden, Material, Cliente, Dinero, Direccion y Cantidad para practicar la
clasificación. No implica que esos seis tipos estén implementados como entidades o valores de Producción.
`ValidadorModeloDominio` exige identidad para las entidades e inmutabilidad para los value objects; no
rechaza una entidad por ser inmutable. El código ejecutable usa `OrdenProduccion`, `LineaDeOrden`,
`NombreMaterial` y `CantidadProduccion`, cuya identidad, igualdad y capacidad se verifican con pruebas
funcionales. La capacidad de 500 pertenece a este ejercicio y no representa una regla universal del sector.
```

Verificación concreta. El archivo `docs/ADR-020-entidades-value-objects.md` existe; comprueba que sus títulos, contenido y enlaces coinciden con todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Modificar los ADR anteriores: rompe la conservación de la etapa anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-020`: rompe la convención.

PASO 5.2.21 — Comparar los documentos heredados con 5.1-conceptos-ddd

La comparación comprueba los documentos heredados byte a byte. Las nuevas clases y los cambios explícitos de integración pertenecen a la copia actual; la carpeta del punto anterior permanece intacta.

1. En la terminal, escribir `cd C:\cursos\AceriaApp` y pulsar Enter.
2. Escribir `for %f in (M05_DDD_CQRS\PROYECTO\5.1-conceptos-ddd\docs\*.md) do fc /L "%f" "M05_DDD_CQRS\PROYECTO\5.2-entidades-value-objects\docs\%~nxf"` y pulsar Enter.
3. Verificar que la salida muestra `FC: no differences encountered` para cada archivo.
4. Verificar que el número de comparaciones coincide con el número de documentos del punto 5.1. Si algún archivo muestra diferencias, restaurarlo desde el punto 5.1.

Verificación concreta. Los 20 documentos heredados coinciden con la etapa anterior, sin omisiones ni modificaciones.

Diagnóstico si falla.

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.
- Omitir `/L` cuando se desea una comparación de texto: indica el modo explícitamente y revisa cada resultado.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

PASO 5.2.22 — Verificar la estructura del punto 5.2

La verificación estructural confirma que el punto 5.2 añade archivos nuevos conservando los documentos heredados.

1. Escribir `tree /F M05_DDD_CQRS\PROYECTO\5.2-entidades-value-objects` y pulsar Enter.
2. Verificar que la salida muestra `AceriaApp.sln`, `README.md`, `src\AceriaApp.Api\ModeloDominio` con las tres clases nuevas, `tests\AceriaApp.ModeloDominio.Tests` con los dos archivos de tests de catálogo y las pruebas funcionales y `docs` con el documento de entidades y el ADR-020. Comprueba también los proyectos y archivos del contexto Producción indicados en el paso 16.
3. Verificar que las carpetas existentes del punto 5.1 permanecen intactas.
4. Verificar que no existe ninguna carpeta `labs` ni ningún archivo `metaprompt.md` ni `verificacion.md`.

Verificación concreta. la estructura contiene los archivos nuevos y conserva los del punto 5.1.

Diagnóstico si falla.

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 5.1: viola la regla de no modificar.
- Omitir la carpeta `ModeloDominio`: los tests no encuentran las clases.

PASO 5.2.23 — Verificar el `.gitignore` del punto 5.2

El `.gitignore` se hereda del punto 5.1 sin modificaciones. La verificación confirma que no se ha tocado.

1. Verificar que el archivo `.gitignore` existe en la carpeta del punto 5.2 (copiado del punto 5.1).
2. Abrir el archivo `.gitignore` con Visual Studio con soporte de .NET 8.
3. Verificar que contiene las líneas `**/bin/`, `**/obj/`, `.vs/`, `*.user`, `TestResults/` y `*.trx`.
4. Verificar que no se ha modificado respecto al punto 5.1. Pulsar `Ctrl+S` si no hay cambios y cerrar el archivo.

Verificación concreta. el `.gitignore` existe y contiene las seis líneas sin modificaciones.

Diagnóstico si falla.

- Modificar el `.gitignore` copiado del punto 5.1: rompe la conservación de la etapa anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el `.gitignore` con otro nombre: el archivo no se reconoce.

PASO 5.2.24 — Hacer el commit del punto 5.2 y verificar el estado final

El commit registra la etapa completa. Si existe uno por punto desde 1.1, el recorrido suma 20 commits de puntos; otros commits legítimos pueden aumentar el historial.

1. Ejecuta `git diff --stat` y revisa los archivos y entregables de esta etapa.
2. Ejecuta `git add .` después de comprobar que no se incluyen binarios, resultados temporales ni credenciales.
3. Ejecuta `git commit -m "Punto 5.2: Entidades y value objects"` con `user.name` y `user.email` configurados.
4. Comprueba `git log --oneline` y `git status`; deben registrar este punto y no dejar cambios pendientes de esta actividad.

Verificación concreta. El historial incluye el commit de 5.2 y `git status` no muestra cambios pendientes de esta actividad; no se impone una longitud rígida al historial.

Diagnóstico si falla.

- No tener configurado `user.name` y `user.email`: `git commit` falla.
- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

Reto de ampliación (optativo)

Trabaja en una copia independiente de la etapa obligatoria, por ejemplo `5.2-entidades-value-objects-reto`. Conserva intacta la carpeta obligatoria y sus documentos. Los tres fragmentos añaden un contrato, un cálculo y una prueba al proyecto del punto.

Reto: añadir un método `Resumen` a `ValidadorModeloDominio` que devuelva el número total de elementos, el número de entidades, el número de value objects y el número de elementos que validan invariantes, y un test que verifique el cálculo con el catálogo por defecto.

Paso 1: En `ValidadorModeloDominio.cs`, añadir el siguiente `record` al final del archivo:

Archivo: `src/AceriaApp.Api/ModeloDominio/ValidadorModeloDominio.cs`. **Operación:** Modificar.

Punto de inserción: después de este texto exacto, que debe aparecer una sola vez:

```
public record ResultadoModelo(bool EsValido, IReadOnlyList<string> Errores);
```

```
public record ResumenModelo(
    int Total,
    int Entidades,
    int ValueObjects,
    int ValidanInvariantes);
```

Paso 2: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public record ResumenModelo(` declara un contrato inmutable. Número total de elementos. Número de entidades. Número de value objects. Número de elementos que validan invariantes.

Paso 3: En `ValidadorModeloDominio.cs`, añadir el siguiente método estático dentro de la clase:

Archivo: `src/AceriaApp.Api/ModeloDominio/ValidadorModeloDominio.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}

public record ResultadoModelo(bool EsValido, IReadOnlyList<string> Errores);
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase indicado por el texto de inserción.

```
public static ResumenModelo Resumen(IEnumerable<ElementoModelo> elementos)
{
    var lista = elementos.ToList();
    var entidades = lista.Count(e => e.Tipo == "Entidad");
    var valueObjects = lista.Count(e => e.Tipo == "ValueObject");
    var validan = lista.Count(e => e.ValidaInvariantes);
    return new ResumenModelo(lista.Count, entidades, valueObjects, validan);
}
```

Paso 4: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public static ResumenModelo Resumen(IEnumerable<ElementoModelo> elemento` declara el método. Materializa la lista. Cuenta las entidades. Cuenta los value objects. Cuenta los que validan invariantes. Devuelve el resumen.

Paso 5: Guardar el archivo con `Ctrl+S`.

Paso 6: En `ValidadorModeloDominioTests.cs`, añadir el siguiente test:

Archivo: `tests/AceriaApp.ModeloDominio.Tests/ValidadorModeloDominioTests.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase indicado por el texto de inserción.

```
[Fact]
public void Resumen_ConCatalogoPorDefecto_CalculaTotales()
{
    var resumen = ValidadorModeloDominio.Resumen(CatalogoModelo.ElementosPorDefecto());
```

```
    resumen.Total.Should().Be(6);
    resumen.Entidades.Should().Be(3);
    resumen.ValueObjects.Should().Be(3);
    resumen.ValidanInvariantes.Should().Be(6);
}
```

Paso 7: Explicar línea por línea:

Marca el método como test de xUnit. Declara el test. Invoca el método. Verifica el total. Verifica las entidades. Verifica los value objects. Verifica los que validan invariantes.

Paso 8: Ejecuta `dotnet test tests/AceriaApp.ModeloDominio.Tests/AceriaApp.ModeloDominio.Tests.csproj -c Release` en la copia optativa. Deben pasar 20 tests del proyecto, incluido el nuevo resumen. Ejecuta después `dotnet test AceriaApp.sln -c Release` y exige 154 pruebas aprobadas.

Paso 9: Comprueba que solo cambian `ValidadorModeloDominio.cs` y `ValidadorModeloDominioTests.cs` en la copia optativa; la etapa obligatoria `5.2-entidades-value-objects` y el punto anterior permanecen idénticos.

Paso 10: Hacer commit con `git commit -m "Reto 5.2: método Resumen en ValidadorModeloDominio"`.

Verificación: `Passed: 20` en el proyecto del punto y 154 pruebas aprobadas en la solución de la copia, cero fallos y cero omisiones.

Resultado verificable

Al finalizar este punto, el alumno debe tener:

- La carpeta `M05_DDD_CQRS/PROYECTO/5.2-entidades-value-objects/` con la solución `AceriaApp.sln` que contiene 17 proyectos.
- El documento `docs/entidades-value-objects.md` con la definición y las reglas.
- El documento `docs/ADR-020-entidades-value-objects.md` con la decisión sobre entidades y value objects.
- Las clases `ElementoModelo`, `ValidadorModeloDominio` y `CatalogoModelo` en `src/AceriaApp.Api/ModeloDominio/`.
- El proyecto `tests/AceriaApp.ModeloDominio.Tests` añade 12 tests de catálogo y 7 funcionales; el reto añade uno más.
- `dotnet test AceriaApp.sln -c Release` ejecuta 153 pruebas en la etapa obligatoria y 154 en una copia con su reto; ninguna falla ni se omite.
- Los ADR-010 a ADR-019 del punto 5.1 sin modificar.
- Un commit revisado para 5.2, con el historial previo conservado.

Continuidad del aprendizaje

El punto 5.2 ha definido las entidades y los value objects como elementos tácticos del modelo del dominio. El proyecto AceriaApp dispone ahora de un validador automático del modelo que comprueba la identidad, la mutabilidad y la validación de invariantes, y 12 tests de catálogo más 7 funcionales nuevos. Esta base permite avanzar hacia el punto 5.3, donde se estudiarán los casos de uso como expresión de las operaciones del dominio y su relación con los puertos de entrada del núcleo.

Punto 5.3 — Casos de uso

Separar la intención del caso de uso de las decisiones técnicas

El dominio define las reglas que una orden debe obedecer; la aplicación coordina cuándo ejecutarlas. Un caso de uso no debería abrir directamente una conexión a SQL Server ni decidir cómo se serializa una respuesta HTTP. Por eso la etapa 5.3 incorporará puertos y adaptadores que permitan recuperar y guardar el agregado de Producción manteniendo la dependencia hacia las abstracciones interiores.

Comenzaremos desde la solución del punto 5.2, conservando sus tests de identidad e invariantes. Añadiremos los casos de uso y comprobaremos tanto resultados satisfactorios como rechazos conocidos. En paralelo, el catálogo docente permite describir acciones del negocio, pero no equivale a invocar una implementación concreta. La evidencia funcional deberá mostrar la interacción entre aplicación, dominio y repositorio SQL Server.

Carpeta de trabajo: `M05_DDD_CQRS/PROYECTO/5.3-casos-de-uso`.

Objetivo: completar los veinticuatro pasos, conservar los documentos heredados y comprobar el modelo con código y tests ejecutables. La etapa suma 172 pruebas en 20 proyectos.

Ejercicio guiado. Añadir al punto 5.2 un documento de casos de uso en `docs/`, una clase de validación de casos de uso en el proyecto API y un test unitario que verifique la validación. El punto 5.3 parte del contenido del punto 5.2 conservando la carpeta anterior y los documentos heredados; los cambios de integración se indican explícitamente en esta copia. La implementación del paso 16 añade comportamiento ejecutable del contexto Producción al ejercicio de catálogo.

PASO 5.3.1 — Crear la carpeta del punto 5.3 a partir del punto 5.2

La copia conserva una etapa autónoma. 5.2-entidades-value-objects aporta 153 tests aprobados y 17 proyectos antes de incorporar este punto.

1. Abre la terminal integrada y localiza `M05_DDD_CQRS/PROYECTO/5.2-entidades-value-objects`; esa es la etapa completa que vas a continuar.
2. Crea `M05_DDD_CQRS/PROYECTO/5.3-casos-de-uso` y copia en ella el contenido de `5.2-entidades-value-objects`; en Windows puedes usar `xcopy` con `/E /I /H /Y`.
3. Excluye `bin`, `obj` y `TestResults`; abre únicamente la copia de `5.3-casos-de-uso` para conservar intacto el punto anterior.
4. Ejecuta `dir` dentro de `5.3-casos-de-uso` y verifica `AceriaApp.sln`, `README.md`, `src`, `tests`, `docs` y `global.json`.

Verificación concreta. La copia contiene la solución y los 17 proyectos anteriores; todavía no contiene los archivos de 5.3.

Diagnóstico si falla.

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.

PASO 5.3.2 — Abrir la solución copiada en Visual Studio con soporte de .NET 8

Abrir la solución copiada permite trabajar en el punto 5.3 sobre la base del punto 5.2. La compilación verifica que la copia es correcta.

1. Abre `AceriaApp.sln` dentro de `5.3-casos-de-uso` con Visual Studio; en Windows puedes ejecutar `start AceriaApp.sln`.
2. Ejecuta `dotnet sln AceriaApp.sln list` y comprueba los 17 proyectos heredados; todas sus rutas deben existir en esta etapa.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release` para comprobar el punto de partida.
4. Si hay DLL bloqueadas por una API en ejecución, detén esa aplicación y repite la compilación. Continúa cuando no haya errores.

Verificación concreta. La solución muestra 17 proyectos y `dotnet build AceriaApp.sln -c Release` termina con `Build succeeded.`.

Diagnóstico si falla.

- Abrir la solución del punto 5.2 en lugar de la del punto 5.3: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener la API del punto 5.2 en ejecución: el puerto queda ocupado y la compilación falla.

PASO 5.3.3 — Verificar que la carpeta `docs` contiene los documentos del punto 5.2

La nueva etapa preserva 22 documentos y añade su ficha temática y su ADR. Los cambios de integración se aplican únicamente en la copia actual.

1. Expande `docs` dentro de la copia de `5.3-casos-de-uso`.
2. Comprueba los 22 documentos heredados: ADR-010 a ADR-020 y fichas de los puntos anteriores.
3. Compara sus nombres y contenido con `docs` de `5.2-entidades-value-objects`; no modifiques ni elimines documentos anteriores.
4. Confirma que todavía no existen `docs/casos-uso.md` ni `docs/ADR-021-casos-uso.md`.

Verificación concreta. Existen 22 documentos heredados idénticos y no existen todavía los dos de este punto.

Diagnóstico si falla.

- Modificar alguno de los ADR o documentos existentes: rompe la conservación de la etapa anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 5.2.
- Crear la carpeta `docs` si ya existe: `mkdir` falla.

PASO 5.3.4 — Crear el documento de casos de uso en `docs`

El documento de casos de uso es el artefacto de `docs` específico del punto 5.3. No modifica los 22 documentos existentes del punto 5.2.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `casos-uso.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que el archivo aparece en la carpeta `docs` sin subrayados rojos.

Archivo: `docs/casos-uso.md`. **Operación:** Crear.

```
# Casos de uso — Punto 5.3

## Definición

Operación del sistema que expresa una intención del negocio. Puede exponerse como método de un servicio de
aplicación o como manejador de una solicitud; el contrato refleja su intención.

## Relación con comando

El caso de uso puede invocarse directamente por su contrato. En 5.3 la API invoca los casos de uso; los
mensajes y manejadores MediatR del contexto Producción se incorporan en 5.4.

## Orquestación

El caso de uso orquesta entidades, value objects, repositorios y unidad de trabajo. Mantiene la política de
aplicación y delega las invariantes en el dominio; no reproduce las reglas internas del agregado.

## Coordinación de repositorios

1. Recuperar entidades con repositorios.
2. Modificar entidades del dominio.
3. Persistir cambios con repositorios.
4. Confirmar transacción con la unidad de trabajo.
5. Devolver el resultado.

## Casos de uso de escritura y de lectura

- Escritura: modifican el estado, requieren unidad de trabajo.
- Lectura: no modifican el estado, pueden usar proyecciones.

## Casos de uso y puertos de entrada

Cada puerto agrupa los casos de uso de una misma capacidad de negocio.

## Casos de uso y adaptadores de entrada

La API llama los casos de uso mediante inyección de dependencias. Un mediador es una opción de despacho que
se incorpora en el siguiente punto.

## Validación

- Sintáctica: contrato de la petición.
- Aplicación: comprobaciones de entrada y traducción de errores esperados; no se incorpora FluentValidation a
este flujo.
- Negocio: invariantes del dominio.

## Transacciones

El caso de uso decide cuándo confirmar los cambios del contexto de persistencia mediante la unidad de
trabajo. Una operación de lectura no confirma cambios. No se promete atomicidad entre contextos distintos ni
servicios externos.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Aplicación ejecutable | Casos de uso de crear, obtener, planificar y finalizar órdenes de producción |
| Adaptador de entrada | Endpoints bajo /api/v1/produccion/ordenes |
| Resultados | `ResultadoProduccion<T>` |
| Coordinación | IOrdenProduccionRepositorio e IProduccionUnitOfWork |
| Validación | Test de coherencia |

## Alcance del catálogo

`CatalogoCasosDeUso` contiene ejemplos CrearOrden, CancelarOrden, ObtenerOrden y ListarOrdenes. Sus
indicadores son datos pedagógicos; no constituyen un inventario de métodos implementados en Producción. En
este punto se ejecutan crear, obtener, planificar y finalizar una orden. El nuevo contexto tiene su propia
persistencia y no comparte entidades con el flujo hexagonal heredado. Los tests funcionales comprueban el
orden de las operaciones y la persistencia desde otro contexto de prueba.
```

Verificación concreta. El archivo `docs/casos-uso.md` existe; comprueba que sus títulos, contenido y enlaces coinciden con todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: rompe la conservación de la etapa anterior.
- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

PASO 5.3.5 — Crear la carpeta `CasosDeUso` en el proyecto API

La carpeta `CasosDeUso` albergará las clases de validación de casos de uso. Se crea como carpeta nueva sin tocar las carpetas existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.Api`.
2. Seleccionar `Add` → `New Folder` y nombrar la carpeta `CasosDeUso`.
3. Verificar que la carpeta `CasosDeUso` aparece dentro de `AceriaApp.Api` sin afectar a las carpetas existentes.
4. Verificar que no se ha modificado ningún archivo existente del punto 5.2.

Verificación concreta. la carpeta `CasosDeUso` existe dentro de `AceriaApp.Api` y las carpetas existentes permanecen intactas.

Diagnóstico si falla.

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: rompe la conservación de la etapa anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

PASO 5.3.6 — Crear la clase `CasoDeUso`

`CasoDeUso` es una descripción del catálogo; no es la clase que registra y ejecuta las órdenes del negocio. Analiza qué entrada, salida y dependencia debería tener una operación como PlanificarOrden y dónde comienza el tratamiento técnico.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `CasosDeUso`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CasoDeUso.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/CasosDeUso/CasoDeUso.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.CasosDeUso;

public class CasoDeUso
{
    public string Nombre { get; init; } = string.Empty;
    public string Tipo { get; init; } = string.Empty;
    public bool TieneManejador { get; init; }
    public bool DevuelveResult { get; init; }
    public bool CoordinaUnidadDeTrabajo { get; init; }
    public bool ContieneReglasDeNegocio { get; init; }
}
```

Verificación concreta. la clase `CasoDeUso` compila sin errores y expone seis propiedades.

Diagnóstico si falla.

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar las propiedades de tipo `string` con `string.Empty`: el compilador emite advertencias.
- Usar `set` en lugar de `init`: permite modificar la instancia después de crearla.

PASO 5.3.7 — Crear la clase `ValidadorCasosDeUso`

El validador del catálogo estudia descripciones de casos de uso. Las condiciones reales de creación o planificación deben permanecer dentro del dominio y comprobarse mediante la ejecución de la clase de aplicación correspondiente.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `CasosDeUso`.
2. Seleccionar `Add` → `Class` y nombrar la clase `ValidadorCasosDeUso.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Alcance del validador. Comprueba los indicadores de los ejemplos del catálogo. No analiza por reflexión los tipos de producción ni garantiza por sí solo la arquitectura. Las reglas del modelo ejecutable y sus límites se comprueban en las pruebas funcionales del paso 16.

Archivo: `src/AceriaApp.Api/CasosDeUso/ValidadorCasosDeUso.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.CasosDeUso;

public class ValidadorCasosDeUso
{
    private static readonly string[] TiposPermitidos = { "Escritura", "Lectura" };

    public ResultadoCasosDeUso Validar(IEnumerable<CasoDeUso> casos)
    {
        var errores = new List<string>();
        var lista = casos.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay casos de uso declarados.");
        }

        foreach (var caso in lista)
        {
            if (string.IsNullOrWhiteSpace(caso.Nombre))
            {
                errores.Add("Un caso de uso no declara el nombre.");
            }

            if (!TiposPermitidos.Contains(caso.Tipo))
            {
                errores.Add($"Tipo no permitido en '{caso.Nombre}': '{caso.Tipo}'.");
            }

            if (!caso.TieneManejador)
            {
                errores.Add($"El caso de uso '{caso.Nombre}' no tiene manejador.");
            }

            if (!caso.DevuelveResult)
            {
                errores.Add($"El caso de uso '{caso.Nombre}' no devuelve Result.");
            }

            if (caso.Tipo == "Escritura" && !caso.CoordinaUnidadDeTrabajo)
            {
                errores.Add($"El caso de uso de escritura '{caso.Nombre}' no coordina la unidad de
trabajo.");
            }

            if (caso.ContieneReglasDeNegocio)
            {
                errores.Add($"El caso de uso '{caso.Nombre}' contiene reglas de negocio.");
            }
        }

        return new ResultadoCasosDeUso(errores.Count == 0, errores);
    }
}
```

```
public record ResultadoCasosDeUso(bool EsValido, IReadOnlyList<string> Errores);
```

Verificación concreta. la clase `ValidadorCasosDeUso` compila sin errores y expone un método y un `record`.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<string>` e `IReadOnlyList<string>` no se resuelven.
- Olvidar el `using System.Linq;`: `Contains` y `ToList` no se resuelven.
- No comprobar el caso de lista vacía: el validador devuelve válido sin casos.

PASO 5.3.8 — Añadir los `using` necesarios en `ValidadorCasosDeUso`

Los tipos `List<string>` e `IReadOnlyList<string>` pertenecen a `System.Collections.Generic`. Los métodos `Contains` y `ToList` pertenecen a `System.Linq`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. En el panel Solution Explorer, hacer doble clic en `ValidadorCasosDeUso.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; el texto de inserción debe aparecer exactamente una vez.
3. Verificar que no hay subrayados rojos en `List<string>`, `Contains` ni `ToList`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/CasosDeUso/ValidadorCasosDeUso.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.CasosDeUso;
```

```
using System.Collections.Generic;
using System.Linq;
```

Verificación concreta. `ValidadorCasosDeUso.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir solo `System.Collections.Generic` y olvidar `System.Linq`: `Contains` y `ToList` no se resuelven.
- Mezclar directivas using con declaraciones de tipos: coloca las directivas antes del namespace para mantener el orden del archivo.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 5.3.9 — Crear la clase `CatalogoCasosDeUso`

`CatalogoCasosDeUso` reúne contratos conceptuales, mientras el puerto real expresa las operaciones necesarias para recuperar o guardar órdenes. Su separación evita confundir inventario documental con composición de servicios.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `CasosDeUso`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoCasosDeUso.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/CasosDeUso/CatalogoCasosDeUso.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.CasosDeUso;

public class CatalogoCasosDeUso
```

```
{
    public static List<CasoDeUso> CasosPorDefecto()
    {
        return new List<CasoDeUso>
        {
            new() { Nombre = "CrearOrden", Tipo = "Escritura", TieneManejador = true, DevuelveResult = true,
CoordinaUnidadDeTrabajo = true, ContieneReglasDeNegocio = false },
            new() { Nombre = "CancelarOrden", Tipo = "Escritura", TieneManejador = true, DevuelveResult =
true, CoordinaUnidadDeTrabajo = true, ContieneReglasDeNegocio = false },
            new() { Nombre = "ObtenerOrden", Tipo = "Lectura", TieneManejador = true, DevuelveResult = true,
CoordinaUnidadDeTrabajo = false, ContieneReglasDeNegocio = false },
            new() { Nombre = "ListarOrdenes", Tipo = "Lectura", TieneManejador = true, DevuelveResult = true,
CoordinaUnidadDeTrabajo = false, ContieneReglasDeNegocio = false }
        };
    }
}
```

Verificación concreta. la clase `CatalogoCasosDeUso` compila sin errores y expone un método estático.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<T>` no se resuelve.
- No inicializar las propiedades de los objetos: el compilador emite advertencias.
- Reutilizar una lista estática mutable: cada llamada del catálogo debe construir una lista independiente.

PASO 5.3.10 — Añadir el `using` necesario en `CatalogoCasosDeUso`

El tipo `List<T>` pertenece a `System.Collections.Generic`. En proyectos con `ImplicitUsings` desactivado, hay que añadir el using explícito.

1. En el panel Solution Explorer, hacer doble clic en `CatalogoCasosDeUso.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; el texto de inserción debe aparecer exactamente una vez.
3. Verificar que no hay subrayados rojos en `List<T>`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/CasosDeUso/CatalogoCasosDeUso.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.CasosDeUso;
```

```
using System.Collections.Generic;
```

Verificación concreta. `CatalogoCasosDeUso.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir `using System.Linq;` en lugar de `System.Collections.Generic`: no resuelve `List<T>`.
- Colocar el using después de una clase: las directivas deben preceder las declaraciones de tipos.
- Olvidar el using y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 5.3.11 — Crear el proyecto de tests del punto 5.3

El punto 5.3 añade un proyecto de tests específico para las clases de casos de uso. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

1. Crea `tests/AceriaApp.CasosDeUso.Tests` y su archivo `AceriaApp.CasosDeUso.Tests.csproj` con el XML completo. Si utilizas la plantilla xUnit de Visual Studio, ajusta el archivo generado.
2. Elimina `UnitTest1.cs` si lo creó la plantilla; las pruebas completas se añaden en los pasos 14 y 15.

3. Ejecuta `dotnet sln AceriaApp.sln add tests/AceriaApp.CasosDeUso.Tests/AceriaApp.CasosDeUso.Tests.csproj` incluso si la solución ya existe.
4. Ejecuta `dotnet sln AceriaApp.sln list` y `dotnet restore tests/AceriaApp.CasosDeUso.Tests/AceriaApp.CasosDeUso.Tests.csproj`.

Archivo: `tests/AceriaApp.CasosDeUso.Tests/AceriaApp.CasosDeUso.Tests.csproj`. **Operación:** Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. La solución muestra 18 proyectos, incluido `AceriaApp.CasosDeUso.Tests`; el csproj fija .NET 8, SDK de tests 17.12.0, xUnit 2.9.2 y runner 2.8.2.

Diagnóstico si falla.

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 5.2: rompe la conservación de la etapa anterior.
- Usar `dotnet new mstest`: el curso usa xUnit.

PASO 5.3.12 — Añadir la referencia del proyecto de tests a la API

La referencia permite que el proyecto de tests acceda a las clases de `CasosDeUso` del proyecto API.

1. Ejecuta `dotnet add tests/AceriaApp.CasosDeUso.Tests/AceriaApp.CasosDeUso.Tests.csproj reference src/AceriaApp.Api/AceriaApp.Api.csproj` o deja el archivo como el XML completo.
2. Comprueba que la referencia va del proyecto de tests a la API; la API no debe depender de los tests.
3. Abre `tests/AceriaApp.CasosDeUso.Tests/AceriaApp.CasosDeUso.Tests.csproj` y confirma la ruta `../../src/AceriaApp.Api/AceriaApp.Api.csproj`.
4. Ejecuta `dotnet build tests/AceriaApp.CasosDeUso.Tests/AceriaApp.CasosDeUso.Tests.csproj -c Release` y verifica que resuelve la API.

Archivo: `tests/AceriaApp.CasosDeUso.Tests/AceriaApp.CasosDeUso.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. el archivo `AceriaApp.CasosDeUso.Tests.csproj` contiene un `ProjectReference` al proyecto API.

Diagnóstico si falla.

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

PASO 5.3.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Ejecuta `dotnet add tests/AceriaApp.CasosDeUso.Tests/AceriaApp.CasosDeUso.Tests.csproj package FluentAssertions --version 6.12.2` o escribe el XML completo.
2. Ejecuta `dotnet list tests/AceriaApp.CasosDeUso.Tests/AceriaApp.CasosDeUso.Tests.csproj package` y localiza `FluentAssertions 6.12.2`.
3. Conserva la referencia a la API y las versiones fijadas de xUnit.
4. Ejecuta `dotnet build tests/AceriaApp.CasosDeUso.Tests/AceriaApp.CasosDeUso.Tests.csproj -c Release`; las aserciones `Should()` estarán disponibles.

Archivo: `tests/AceriaApp.CasosDeUso.Tests/AceriaApp.CasosDeUso.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. El archivo y `dotnet list package` muestran FluentAssertions 6.12.2; la compilación termina correctamente.

Diagnóstico si falla.

- Instalar una versión preliminar: el curso usa la versión fijada en el archivo de proyecto.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

PASO 5.3.14 — Crear el archivo `ValidadorCasosDeUsoTests.cs`

Los tests verifican que el validador acepta el catálogo por defecto y rechaza los casos de uso que violan las reglas del patrón.

1. En el panel Solution Explorer, expandir `tests` → `AceriaApp.CasosDeUso.Tests`.
2. Crear el archivo `ValidadorCasosDeUsoTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.CasosDeUso.Tests/ValidadorCasosDeUsoTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.CasosDeUso;
using FluentAssertions;
using Xunit;

namespace AceriaApp.CasosDeUso.Tests;

public class ValidadorCasosDeUsoTests
{
    private readonly ValidadorCasosDeUso _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoCasosDeUso.CasosPorDefecto());

        resultado.EsValido.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinCasos_DevuelveError()
    {
        var resultado = _validador.Validar(new List<CasoDeUso>());

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay casos de uso"));
    }

    [Fact]
    public void Validar_TipoNoPermitido_DevuelveError()
    {
        var casos = new List<CasoDeUso>
        {
            new() { Nombre = "X", Tipo = "TipoInventado", TieneManejador = true, DevuelveResult = true }
        };

        var resultado = _validador.Validar(casos);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Tipo no permitido"));
    }

    [Fact]
    public void Validar_SinManejador_DevuelveError()
    {
        var casos = new List<CasoDeUso>
        {
            new() { Nombre = "X", Tipo = "Lectura", TieneManejador = false, DevuelveResult = true }
        };

        var resultado = _validador.Validar(casos);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene manejador"));
    }

    [Fact]
    public void Validar_SinResult_DevuelveError()
    {
        var casos = new List<CasoDeUso>
        {
            new() { Nombre = "X", Tipo = "Lectura", TieneManejador = true, DevuelveResult = false }
        };

        var resultado = _validador.Validar(casos);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no devuelve Result"));
    }

    [Fact]
    public void Validar_EscrituraSinUnidadDeTrabajo_DevuelveError()
    {
```

```
        var casos = new List<CasoDeUso>
        {
            new() { Nombre = "X", Tipo = "Escritura", TieneManejador = true, DevuelveResult = true,
CoordinaUnidadDeTrabajo = false }
        };

        var resultado = _validador.Validar(casos);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no coordina la unidad de trabajo"));
    }

    [Fact]
    public void Validar_ConReglasDeNegocio_DevuelveError()
    {
        var casos = new List<CasoDeUso>
        {
            new() { Nombre = "X", Tipo = "Lectura", TieneManejador = true, DevuelveResult = true,
ContieneReglasDeNegocio = true }
        };

        var resultado = _validador.Validar(casos);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("contiene reglas de negocio"));
    }
}
```

Verificación concreta. el archivo `ValidadorCasosDeUsoTests.cs` compila y contiene siete métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.CasosDeUso;`: las clases no se resuelven.
- Usar `Assert.True` en lugar de `Should().BeTrue()`: el curso usa FluentAssertions.
- No cubrir el caso de escritura sin unidad de trabajo: la regla específica no se verifica.

PASO 5.3.15 — Crear el archivo `CatalogoCasosDeUsoTests.cs`

Los tests verifican que el catálogo por defecto contiene los casos esperados y la distribución correcta por tipo.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.CasosDeUso.Tests`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoCasosDeUsoTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.CasosDeUso.Tests/CatalogoCasosDeUsoTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.CasosDeUso;
using FluentAssertions;
using Xunit;

namespace AceriaApp.CasosDeUso.Tests;

public class CatalogoCasosDeUsoTests
{
    [Fact]
    public void CasosPorDefecto_ContieneCuatroCasos()
    {
        var casos = CatalogoCasosDeUso.CasosPorDefecto();

        casos.Should().HaveCount(4);
    }

    [Fact]
    public void CasosPorDefecto_ContieneDosEscriturasYDosLecturas()
    {
```

```
        var casos = CatalogoCasosDeUso.CasosPorDefecto();

        casos.Count(c => c.Tipo == "Escritura").Should().Be(2);
        casos.Count(c => c.Tipo == "Lectura").Should().Be(2);
    }

    [Fact]
    public void CasosPorDefecto_TodosDevuelvenResult()
    {
        var casos = CatalogoCasosDeUso.CasosPorDefecto();

        casos.Should().OnlyContain(c => c.DevuelveResult);
    }

    [Fact]
    public void CasosPorDefecto_EscriturasCoordinanUnidadDeTrabajo()
    {
        var casos = CatalogoCasosDeUso.CasosPorDefecto();

        casos.Where(c => c.Tipo == "Escritura").Should().OnlyContain(c => c.CoordinaUnidadDeTrabajo);
    }

    [Fact]
    public void CasosPorDefecto_LecturasNoCoordinanUnidadDeTrabajo()
    {
        var casos = CatalogoCasosDeUso.CasosPorDefecto();

        casos.Where(c => c.Tipo == "Lectura").Should().OnlyContain(c => !c.CoordinaUnidadDeTrabajo);
    }
}
```

Verificación concreta. el archivo `CatalogoCasosDeUsoTests.cs` compila y contiene cinco métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.CasosDeUso;`: las clases no se resuelven.
- Olvidar el `using System.Linq;`: `Count`, `Where` y `OnlyContain` no se resuelven.
- No verificar la coordinación de la unidad de trabajo: la regla no se comprueba.

PASO 5.3.16 — Construir casos de uso con puertos y persistencia

La aplicación coordina los puertos y el agregado: no debe exigir al dominio que conozca DbContext, y las pruebas de integración deben recuperar una orden desde un contexto SQL distinto al que la guardó. De ese modo se observa la persistencia efectiva del caso de uso.

1. Lee cada archivo de integración de este paso y localiza su responsabilidad en las capas del contexto Producción.
2. Crea o modifica los archivos exactamente como se indica; sigue la lectura numerada para conectar contratos, reglas y pruebas.
3. Añade los nuevos proyectos a la solución: `dotnet sln AceriaApp.sln add src/AceriaApp.Produccion.Aplicacion/AceriaApp.Produccion.Aplicacion.csproj`. `dotnet sln AceriaApp.sln add src/AceriaApp.Produccion.Infraestructura/AceriaApp.Produccion.Infraestructura.csproj`.
4. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`; `dotnet sln AceriaApp.sln list` debe mostrar 20 proyectos.

Integración funcional. Configura las referencias y versiones de este proyecto manteniendo los proyectos y catálogos heredados.

Archivo: `src/AceriaApp.Api/AceriaApp.Api.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk.Web"><PropertyGroup><TargetFramework>net8.0</
TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</ImplicitUsings></
PropertyGroup><ItemGroup><PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer"
Version="8.0.21" /><PackageReference Include="FluentValidation.DependencyInjectionExtensions"
```

```
Version="11.11.0" /><PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" /></ItemGroup>
<ItemGroup>
    <ProjectReference Include="../AceriaApp.Hexagonal.Aplicacion/AceriaApp.Hexagonal.Aplicacion.csproj" />
    <ProjectReference Include="../AceriaApp.Hexagonal.Infraestructura/
AceriaApp.Hexagonal.Infraestructura.csproj" />
  </ItemGroup>
  <ItemGroup>
    <ProjectReference Include="../AceriaApp.Produccion.Infraestructura/
AceriaApp.Produccion.Infraestructura.csproj" />
  </ItemGroup>
</Project>
```

Fija la versión del paquete usado por esta capa o por sus pruebas para hacer reproducible la compilación. Declara una dependencia de proyecto que el compilador necesita; no registra servicios en el contenedor. Declara una dependencia de proyecto que el compilador necesita; no registra servicios en el contenedor.

Declara una dependencia de proyecto que el compilador necesita; no registra servicios en el contenedor.

Integración funcional. Expone el contrato HTTP de Producción con 201 y Location al crear, y errores 400, 404 o 409 según su significado.

Archivo: `src/AceriaApp.Api/Produccion/ProduccionEndpoints.cs`. **Operación:** Crear.

```
using AceriaApp.Produccion.Aplicacion;
namespace AceriaApp.Api.Produccion;
public sealed record CrearOrdenProduccionRequest(string Cliente, string Material, int Cantidad);
public static class ProduccionEndpoints
{
    public static IEndpointRouteBuilder MapProduccion(this IEndpointRouteBuilder app)
    {
        var grupo = app.MapGroup("/api/v1/produccion/ordenes").WithTags("Produccion");
        grupo.MapPost("", async (CrearOrdenProduccionRequest request, OrdenesProduccionCasosUso casos,
CancellationToken ct) =>
        {
            var resultado = await casos.CrearAsync(request.Cliente, request.Material, request.Cantidad, ct);
            return resultado.EsExito
                ? Results.Created($"/api/v1/produccion/ordenes/{resultado.Valor}", new { id =
resultado.Valor })
                : ErrorHttp(resultado.Error);
        });
        grupo.MapGet("/{id:guid}", async (Guid id, OrdenesProduccionCasosUso casos, CancellationToken ct) =>
            Responder(await casos.ObtenerAsync(id, ct)));
        grupo.MapPost("/{id:guid}/planificar", async (Guid id, OrdenesProduccionCasosUso casos,
CancellationToken ct) =>
            Responder(await casos.PlanificarAsync(id, ct)));
        grupo.MapPost("/{id:guid}/finalizar", async (Guid id, OrdenesProduccionCasosUso casos,
CancellationToken ct) =>
            Responder(await casos.FinalizarAsync(id, ct)));
        return app;
    }
    private static IResult Responder<T>(ResultadoProduccion<T> resultado) =>
        resultado.EsExito ? Results.Ok(resultado.Valor) : ErrorHttp(resultado.Error);
    private static IResult ErrorHttp(ErrorProduccion error) => Results.Problem(
        title: error.Mensaje,
        statusCode: error.Tipo switch
        {
            TipoErrorProduccion.Validacion => 400,
            TipoErrorProduccion.NoEncontrado => 404,
            TipoErrorProduccion.Conflicto => 409,
            _ => throw new ArgumentOutOfRangeException(nameof(error))
        }, extensions: new Dictionary<string, object?> { ["codigo"] = error.Codigo });
}
```

Mantiene una ruta explícita del contexto nuevo, sin colisionar con las APIs de los módulos anteriores. Devuelve 201 y la URI que permite consultar el recurso recién creado. Traduce una transición incompatible con el estado actual a conflicto HTTP.

Integración funcional. Integra el contexto nuevo en la API conservando los endpoints heredados y limita EnsureCreated al entorno Development.

Archivo: `src/AceriaApp.Api/Program.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Api.Produccion;
using AceriaApp.Produccion.Infraestructura;
using AceriaApp.Api.Patrones;
using AceriaApp.Hexagonal.Infraestructura.Patrones;
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Validacion;
using FluentValidation;
using AceriaApp.Api.Documentacion;

using AceriaApp.Api.Hexagonal;
using AceriaApp.Hexagonal.Infraestructura;
using AceriaApp.Hexagonal.Infraestructura.Datos;
var builder=WebApplication.CreateBuilder(args);
// INICIO PASO 2.1.12: SQL Server Express LocalDB, no SQLite.
builder.Services.AddDbContext<AceriaDbContext>(o=>
    o.UseSqlServer(builder.Configuration.GetConnectionString("AceriaApp")));
// PASO 2.3.14: registrar validadores en el contenedor.
builder.Services.AddValidatorsFromAssemblyContaining<CrearOrdenValidator>(); // PASO 2.3.14
// PASO 2.4.6: registrar generador antes de Build.
builder.Services.AddDocumentacion(); // PASO 2.4.6

builder.Services.AddServiciosHexagonales(builder.Configuration);
builder.Services.AddPatronesOrdenes();
builder.Services.AddProduccion(builder.Configuration);
var app=builder.Build();
// EnsureCreated se utiliza SOLO como ayuda didáctica en Development.
// Para un sistema productivo se utilizarían migraciones controladas.
// PASO 2.1.13: inicializar LocalDB solo en Development.
if(app.Environment.IsDevelopment()&&!app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope=app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<AceriaDbContext>().Database.EnsureCreatedAsync();
}

// INICIO PASO 2.1.14: colección de recursos, consulta real con EF Core.
app.MapGet("/api/v1/ordenes",async (AceriaDbContext db,CancellationToken ct)=>
{
    // PASO 2.1.15: lectura sin tracking y proyeccion a DTO.
    var ordenes=await db.Ordenes.AsNoTracking().OrderBy(x=>x.FechaCreacion).ToListAsync(ct);
    return Results.Ok(ordenes.Select(OrdenDto.Desde));
})
.WithName("ListarOrdenes").WithTags("Ordenes")
.WithSummary("Lista las órdenes de producción").Produces<IEnumerable<OrdenDto>>(200)
;
// INICIO PASO 2.1.17: recurso individual.
app.MapGet("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(x=>x.Id==id);
    // PASO 2.1.18: 200 para orden presente; 404 si no existe.
    IResult response=orden is null ? Results.NotFound() : Results.Ok(OrdenDto.Desde(orden));
    return response;
})
.WithName("ObtenerOrdenPorId").WithTags("Ordenes")
.WithSummary("Obtiene una orden por ID").Produces<OrdenDto>(200).Produces(404)
;


// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    // PASO 2.2.6: crear entidad y persistirla.
    var orden=new Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
```

```
    await db.SaveChangesAsync();
    // PASO 2.2.7: devolver 201 con Location.
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")
// PASO 2.3.15: filtro de POST.
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Crea una orden").Produces<OrdenDto>(201).ProducesValidationProblem(400).ProducesProblem(422)
;


// INICIO PASO 2.2.11: sustitución completa de los campos editables.
app.MapPut("/api/v1/ordenes/{id:guid}",async (Guid id,CrearOrdenRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cliente=request.Cliente;orden.Material=request.Material;
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("ReemplazarOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Sustituye campos
editables").Produces(204).Produces(404).ProducesProblem(409).ProducesValidationProblem(400).ProducesProblem(4
22)
;


// INICIO PASO 2.2.14: cambiar SOLO la cantidad.
app.MapPatch("/api/v1/ordenes/{id:guid}/cantidad",
    async (Guid id,ActualizarCantidadRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("CambiarCantidad").WithTags("Ordenes")
// PASO 2.3.17: filtro de PATCH.
.AddEndpointFilter<ValidationFilter<ActualizarCantidadRequest>>()
.WithSummary("Modifica
cantidad").Produces<OrdenDto>(200).Produces(404).ProducesValidationProblem(400).ProducesProblem(409).Produces
Problem(422)
;


// INICIO PASO 2.2.16: eliminar recurso persistido.
app.MapDelete("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede eliminar una orden finalizada.",statusCode:409);
    db.Ordenes.Remove(orden);
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("EliminarOrden").WithTags("Ordenes")
.WithSummary("Elimina una orden pendiente").Produces(204).Produces(404).ProducesProblem(409)
;
// INICIO PASO 2.2.18: transición explícita del dominio.
app.MapPost("/api/v1/ordenes/{id:guid}/finalizar",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
```

```
        title:"La orden ya estaba finalizada.",statusCode:409);
    orden.Estado="Finalizada";
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("FinalizarOrden").WithTags("Ordenes")
.WithSummary("Finaliza una orden").Produces<OrdenDto>(200).Produces(404).ProducesProblem(409)
;

// PASO 2.4.8: habilitar Swagger solo en Development.
app.UseDocumentacion(); // PASO 2.4.8
// El contexto hexagonal usa su propia base para que EnsureCreated no dependa de
// las tablas heredadas del curso M02.
if (app.Environment.IsDevelopment() && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<HexagonalDbContext>()
        .Database.EnsureCreatedAsync();
}
app.MapHexagonalOrdenes();
app.MapPatronesOrdenes();
// Base didáctica propia del contexto Producción, independiente de los modelos anteriores.
if (app.Environment.IsDevelopment() && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<ProduccionDbContext>().Database.EnsureCreatedAsync();
}
app.MapProduccion();
app.Run();
public partial class Program {} // permite WebApplicationFactory en tests
```

Registra el contexto de Producción antes de construir el contenedor. Inicializa la base didáctica propia solo dentro de la condición Development y sin sustituir migraciones productivas. Añade las rutas del nuevo contexto a las rutas ya existentes.

Integración funcional. Añade la base LocalDB AceriaProduccion sin modificar las conexiones heredadas; la separación es una decisión de esta práctica.

Archivo: `src/AceriaApp.Api/appsettings.json`. **Operación:** Reemplazar el contenido completo.

```
{
  "ConnectionStrings": {
    "AceriaApp": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaAppM02;Trusted_Connection=True;TrustServerCertificate=True",
    "AceriaHexagonal": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaAppM03Hexagonal;Trusted_Connection=True;TrustServerCertificate=True",
    "AceriaProduccion": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaProduccion;Trusted_Connection=True;TrustServerCertificate=True"
  }
}
```

Configura la conexión SQL Server propia de Producción. Las conexiones de los flujos heredados se conservan; ajusta servidor y autenticación a tu entorno sin incluir secretos en Git.

Integración funcional. Configura las referencias y versiones de este proyecto manteniendo los proyectos y catálogos heredados.

Archivo: `src/AceriaApp.Produccion.Aplicacion/AceriaApp.Produccion.Aplicacion.csproj`. **Operación:** Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings></PropertyGroup>
  <ItemGroup><ProjectReference Include="../AceriaApp.Produccion.Dominio/
AceriaApp.Produccion.Dominio.csproj" /></ItemGroup>
</Project>
```

Declara una dependencia de proyecto que el compilador necesita; no registra servicios en el contenedor.

Integración funcional. Proyecta la raíz y sus líneas a un contrato de salida sin exponer entidades del dominio.

Archivo: `src/AceriaApp.Produccion.Aplicacion/OrdenProduccionDto.cs`. **Operación:** Crear.

```
using AceriaApp.Produccion.Dominio;
namespace AceriaApp.Produccion.Aplicacion;
public sealed record LineaProduccionDto(Guid Id, string Material, int Cantidad);
public sealed record OrdenProduccionDto(Guid Id, string Cliente, string Estado,
    int CantidadTotal, IReadOnlyList<LineaProduccionDto> Lineas)
{
    public static OrdenProduccionDto Desde(OrdenProduccion orden) => new(
        orden.Id, orden.Cliente, orden.Estado.ToString(), orden.CantidadTotal,
        orden.Lineas.Select(l => new LineaProduccionDto(l.Id, l.Material.Valor,
l.Cantidad.Valor)).ToList().AsReadOnly());
}
```

Copia identidad y valores escalares de la línea; el cliente no recibe la entidad hija del agregado. Describe la respuesta de la orden con el total y sus líneas; no ofrece operaciones que modifiquen el dominio. Concentra la traducción de una entidad ya cargada a su contrato de salida; no realiza consultas ni confirmaciones.

Construye DTO nuevos para cada línea y materializa una colección de solo lectura, separada de la colección interna del agregado.

Integración funcional. Coordina recuperación, operación del agregado y commit; convierte solo reglas de negocio esperadas en resultados tipados.

Archivo: `src/AceriaApp.Produccion.Aplicacion/OrdenesProduccionCasosUso.cs`. **Operación:** Crear.

```
using AceriaApp.Produccion.Dominio;
namespace AceriaApp.Produccion.Aplicacion;

// La aplicación coordina; las reglas de capacidad y de estado viven en el agregado.
public sealed class OrdenesProduccionCasosUso(
    IOrdenProduccionRepositorio repositorio, IProduccionUnitOfWork unidad)
{
    public async Task<ResultadoProduccion<Guid>> CrearAsync(string cliente, string material, int cantidad,
CancellationToken ct = default)
    {
        try
        {
            var orden = new OrdenProduccion(cliente, material, cantidad);
            await repositorio.AgregarAsync(orden, ct);
            await unidad.CommitAsync(ct);
            return ResultadoProduccion<Guid>.Exito(orden.Id);
        }
        catch (ReglaProduccionException ex) { return ResultadoProduccion<Guid>.Fallo(Traducir(ex)); }
    }
    public async Task<ResultadoProduccion<OrdenProduccionDto>> ObtenerAsync(Guid id, CancellationToken ct =
default)
    {
        var orden = await repositorio.ObtenerAsync(id, ct);
        return orden is null ? Ausente() :
ResultadoProduccion<OrdenProduccionDto>.Exito(OrdenProduccionDto.Desde(orden));
    }
    public Task<ResultadoProduccion<OrdenProduccionDto>> PlanificarAsync(Guid id, CancellationToken ct =
default) =>
        ModificarAsync(id, o => o.Planificar(), ct);
    public Task<ResultadoProduccion<OrdenProduccionDto>> FinalizarAsync(Guid id, CancellationToken ct =
default) =>
        ModificarAsync(id, o => o.Finalizar(), ct);

    private async Task<ResultadoProduccion<OrdenProduccionDto>> ModificarAsync(Guid id,
Action<OrdenProduccion> cambio, CancellationToken ct)
    {
        var orden = await repositorio.ObtenerAsync(id, ct);
        if (orden is null) return Ausente();
        try
        {
```

```
            cambio(orden);
            await unidad.CommitAsync(ct);
            return ResultadoProduccion<OrdenProduccionDto>.Exito(OrdenProduccionDto.Desde(orden));
        }
        catch (ReglaProduccionException ex) { return
ResultadoProduccion<OrdenProduccionDto>.Fallo(Traducir(ex)); }
    }
    private static ResultadoProduccion<OrdenProduccionDto> Ausente() =>
        ResultadoProduccion<OrdenProduccionDto>.Fallo(new("orden.ausente", "La orden de producción no
existe.", TipoErrorProduccion.NoEncontrado));
    private static ErrorProduccion Traducir(ReglaProduccionException ex) => new(ex.Codigo, ex.Message,
        ex.Codigo == "estado.invalido" ? TipoErrorProduccion.Conflicto : TipoErrorProduccion.Validacion);
}
```

Delega las invariantes de creación en el dominio. Registra el agregado sin confirmar aún la escritura. Confirma los cambios de la unidad de trabajo del contexto Producción.

Invoca el comportamiento del agregado, sin asignar su estado desde fuera. Distingue una orden que no existe de una petición con datos inválidos. Convierte únicamente errores esperados de negocio; los fallos técnicos y la cancelación se propagan.

Integración funcional. Separa la recuperación del agregado del commit explícito; el contrato no depende de EF Core.

Archivo: `src/AceriaApp.Produccion.Aplicacion/PuertosProduccion.cs`. **Operación:** Crear.

```
using AceriaApp.Produccion.Dominio;
namespace AceriaApp.Produccion.Aplicacion;
public interface IOrdenProduccionRepositorio
{
    Task<OrdenProduccion?> ObtenerAsync(Guid id, CancellationToken ct = default);
    Task AgregarAsync(OrdenProduccion orden, CancellationToken ct = default);
}
public interface IProduccionUnitOfWork
{
    Task CommitAsync(CancellationToken ct = default);
}
```

`public interface IOrdenProduccionRepositorio` declara lo que necesita la aplicación para recuperar y preparar un agregado; el contrato no contiene DbContext ni SQL. Recupera una orden completa o null si no existe; el token se transmite al adaptador para respetar cancelación. Prepara el alta del agregado sin confirmarla. Mantener esta separación permite decidir la frontera de la operación en el caso de uso.

Confirma los cambios preparados en la unidad de trabajo actual. La implementación debe compartir el contexto del repositorio.

Integración funcional. Representa éxito o error tipado y prohíbe leer la rama incorrecta del resultado.

Archivo: `src/AceriaApp.Produccion.Aplicacion/ResultadoProduccion.cs`. **Operación:** Crear.

```
namespace AceriaApp.Produccion.Aplicacion;
public enum TipoErrorProduccion { Validacion, NoEncontrado, Conflicto }
public sealed record ErrorProduccion(string Codigo, string Mensaje, TipoErrorProduccion Tipo);
public sealed class ResultadoProduccion<T>
{
    private readonly T? valor;
    private readonly ErrorProduccion? error;
    public bool EsExito { get; }
    private ResultadoProduccion(T valor) { this.valor = valor; EsExito = true; }
    private ResultadoProduccion(ErrorProduccion error) { this.error = error; }
    public T Valor => EsExito ? valor! : throw new InvalidOperationException("Un fallo no contiene valor.");
    public ErrorProduccion Error => !EsExito ? error! : throw new InvalidOperationException("Un éxito no
contiene error.");
    public static ResultadoProduccion<T> Exito(T valor)
    { ArgumentNullException.ThrowIfNull(valor); return new(valor); }
    public static ResultadoProduccion<T> Fallo(ErrorProduccion error)
```

```
    { ArgumentNullException.ThrowIfNull(error); return new(error); }
}
```

Expresa categorías de aplicación que el adaptador HTTP traducirá a 400, 404 y 409. Impide usar un valor inexistente cuando el resultado representa un fallo. Protege también la rama de error para que el contrato no permita estados ambiguos.

Integración funcional. Configura las referencias y versiones de este proyecto manteniendo los proyectos y catálogos heredados.

Archivo: `src/AceriaApp.Produccion.Infraestructura/AceriaApp.Produccion.Infraestructura.csproj`. **Operación:** Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings></PropertyGroup>
  <ItemGroup><ProjectReference Include="../AceriaApp.Produccion.Aplicacion/
AceriaApp.Produccion.Aplicacion.csproj" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.21" />
    <FrameworkReference Include="Microsoft.AspNetCore.App" /></ItemGroup>
</Project>
```

Declara una dependencia de proyecto que el compilador necesita; no registra servicios en el contenedor. Fija la versión del paquete usado por esta capa o por sus pruebas para hacer reproducible la compilación.

Integración funcional. El repositorio recupera la raíz con sus líneas y registra altas; únicamente la unidad de trabajo confirma cambios.

Archivo: `src/AceriaApp.Produccion.Infraestructura/PersistenciaProduccion.cs`. **Operación:** Crear.

```
using AceriaApp.Produccion.Aplicacion;
using AceriaApp.Produccion.Dominio;
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Produccion.Infraestructura;
public sealed class OrdenProduccionRepositorio(ProduccionDbContext db) : IOrdenProduccionRepositorio
{
    public Task<OrdenProduccion?> ObtenerAsync(Guid id, CancellationToken ct = default) =>
        db.Ordenes.Include(o => o.Lineas).SingleOrDefaultAsync(o => o.Id == id, ct);
    public async Task AgregarAsync(OrdenProduccion orden, CancellationToken ct = default) =>
        await db.Ordenes.AddAsync(orden, ct);
}
public sealed class ProduccionUnitOfWork(ProduccionDbContext db) : IProduccionUnitOfWork
{
    public async Task CommitAsync(CancellationToken ct = default) => await db.SaveChangesAsync(ct);
}
```

Recupera el agregado completo para que las reglas dispongan de todas sus líneas. Agrega el grafo al seguimiento de EF sin ejecutar SaveChanges. Centraliza la confirmación explícita; un SaveChanges guarda el agregado en una transacción relacional.

Integración funcional. Mapea identidad, colección privada y value objects con EF Core en un modelo relacional propio del contexto Producción.

Archivo: `src/AceriaApp.Produccion.Infraestructura/ProduccionDbContext.cs`. **Operación:** Crear.

```
using AceriaApp.Produccion.Dominio;
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Produccion.Infraestructura;
public sealed class ProduccionDbContext(DbContextOptions<ProduccionDbContext> opciones) : DbContext(opciones)
{
    public DbSet<OrdenProduccion> Ordenes => Set<OrdenProduccion>();
    protected override void OnModelCreating(ModelBuilder model)
    {
        var orden = model.Entity<OrdenProduccion>();
        orden.ToTable("OrdenesProduccion");
```

```
        orden.HasKey(o => o.Id);
        orden.Property(o => o.Id).ValueGeneratedNever();
        orden.Property(o => o.Cliente).HasMaxLength(100).IsRequired();
        orden.Property(o => o.Estado).HasConversion<string>().HasMaxLength(20);
        orden.Ignore(o => o.CantidadTotal);
        orden.Ignore(o => o.Cantidad);
        orden.Ignore(o => o.Material);
        orden.HasMany(o =>
o.Lineas).WithOne().HasForeignKey("OrdenId").IsRequired().OnDelete(DeleteBehavior.Cascade);
        orden.Navigation(o => o.Lineas).HasField("_lineas").UsePropertyAccessMode(PropertyAccessMode.Field);
        var linea = model.Entity<LineaDeOrden>();
        linea.ToTable("LineasProduccion");
        linea.HasKey(l => l.Id);
        linea.Property(l => l.Id).ValueGeneratedNever();
        linea.Property(l => l.Material).HasConversion(v => v.Valor, v => new
NombreMaterial(v)).HasMaxLength(100).IsRequired();
        linea.Property(l => l.Cantidad).HasConversion(v => v.Valor, v => new CantidadProduccion(v));
    }
}
```

Conserva la identidad creada por el dominio en lugar de pedir otra a la base. El total se calcula desde las líneas y no se persiste como dato duplicado. Permite a EF materializar la colección privada sin exponer un setter público.

Almacena el valor escalar y reconstruye el value object validado durante la lectura.

Integración funcional. Compone los adaptadores scoped de Producción sobre SQL Server; desde 5.4 registra el ensamblado propio de MediatR y el puerto de lecturas.

Archivo: `src/AceriaApp.Produccion.Infraestructura/RegistroProduccion.cs`. **Operación:** Crear.

```
using AceriaApp.Produccion.Aplicacion;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
namespace AceriaApp.Produccion.Infraestructura;
public static class RegistroProduccion
{
    public static IServiceCollection AddProduccion(this IServiceCollection services, IConfiguration
configuration)
    {
        services.AddDbContext<ProduccionDbContext>(o =>
o.UseSqlServer(configuration.GetConnectionString("AceriaProduccion")));
        services.AddScoped<IOrdenProduccionRepositorio, OrdenProduccionRepositorio>();
        services.AddScoped<IProduccionUnitOfWork, ProduccionUnitOfWork>();
        services.AddScoped<OrdenesProduccionCasosUso>();
        return services;
    }
}
```

Configura SQL Server; las pruebas usan el mismo proveedor con una conexión a una base aislada. Usa una instancia por petición que comparte el DbContext con la unidad de trabajo.

Integración funcional. Configura las referencias y versiones de este proyecto manteniendo los proyectos y catálogos heredados.

Archivo: `tests/AceriaApp.CasosDeUso.Tests/AceriaApp.CasosDeUso.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsPackable>false</IsPackable><IsTestProject>true</IsTestProject></PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <ProjectReference Include="../../src/AceriaApp.Produccion.Dominio/AceriaApp.Produccion.Dominio.csproj" />
```

```
    <PackageReference Include="Testcontainers.MsSql" Version="3.10.0" />
    <PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="8.0.21" />
    <Compile Include="../Compartido/ProduccionApiFactory.cs" Link="ProduccionApiFactory.cs" />
    <Compile Include="../Compartido/SqlServerPruebas.cs" Link="SqlServerPruebas.cs" />
  </ItemGroup>
</Project>
```

Compila para .NET 8, igual que los proyectos heredados. Fija el cliente de contenedores para SQL Server; el proveedor EF Core SQL Server llega mediante la referencia a la API. Permite ejecutar peticiones HTTP contra el host real de la API.

Compila el helper heredado que crea y elimina únicamente la base aislada de cada prueba, sin añadir otro proyecto a la solución.

Integración funcional. Añade 7 pruebas funcionales del punto 5.3, además del catálogo original; verifica comportamiento real y regresiones heredadas.

Archivo: `tests/AceriaApp.CasosDeUso.Tests/CasosDeUsoFuncionalesTests.cs`. **Operación:** Crear.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Produccion.Dominio;
using AceriaApp.Produccion.Aplicacion;
using AceriaApp.Produccion.Infraestructura;
using AceriaApp.PruebasProduccion;
using Microsoft.EntityFrameworkCore;
using Xunit;
namespace AceriaApp.CasosDeUso.Tests;
public sealed class CasosDeUsoFuncionalesTests
{
    private static OrdenesProduccionCasosUso Casos(ProduccionDbContext db) =>
        new(new OrdenProduccionRepositorio(db), new ProduccionUnitOfWork(db));

    [Fact]
    public async Task Repositorio_no_guarda_hasta_el_commit_y_rehidrata_el_agregado()
    {
        using var factory = new ProduccionApiFactory();
        await using var escritura = factory.NuevoContexto();
        var orden = new OrdenProduccion("Axia", "Acero", 20);
        await new OrdenProduccionRepositorio(escritura).AgregarAsync(orden);
        await using (var antes = factory.NuevoContexto())
            Assert.Equal(0, await antes.Ordenes.CountAsync());
        await new ProduccionUnitOfWork(escritura).CommitAsync();
        await using var despues = factory.NuevoContexto();
        var leida = await new OrdenProduccionRepositorio(despues).ObtenerAsync(orden.Id);
        Assert.Equal(orden, leida);
        Assert.NotSame(orden, leida);
        Assert.Equal(orden.Lineas[0].Id, Assert.Single(leida!.Lineas).Id);
        Assert.Equal(new NombreMaterial("Acero"), leida.Lineas[0].Material);
        Assert.Equal(20, leida.CantidadTotal);
    }
    [Fact]
    public async Task Casos_de_uso_confirman_cada_transicion_en_otra_unidad_de_trabajo()
    {
        using var factory = new ProduccionApiFactory();
        Guid id;
        await using (var db = factory.NuevoContexto())
            id = (await Casos(db).CrearAsync(" Axia ", " Acero ", 25)).Valor;
        await using (var db = factory.NuevoContexto())
            Assert.Equal("Planificada", (await Casos(db).PlanificarAsync(id)).Valor.Estado);
        await using (var db = factory.NuevoContexto())
            Assert.Equal("Finalizada", (await Casos(db).FinalizarAsync(id)).Valor.Estado);
        await using var lectura = factory.NuevoContexto();
        var dto = (await Casos(lectura).ObtenerAsync(id)).Valor;
        Assert.Equal("Axia", dto.Cliente);
        Assert.Equal("Finalizada", dto.Estado);
        Assert.Equal(25, dto.CantidadTotal);
    }
    [Fact]
    public async Task Errores_esperados_son_resultados_tipados_sin_escrituras_parciales()
    {
        using var factory = new ProduccionApiFactory();
```

```
        await using var db = factory.NuevoContexto();
        var caso = Casos(db);
        var invalida = await caso.CrearAsync("Axia", "Acero", 501);
        Assert.False(invalida.EsExito);
        Assert.Equal(TipoErrorProduccion.Validacion, invalida.Error.Tipo);
        Assert.Equal("cantidad.invalida", invalida.Error.Codigo);
        Assert.Equal(TipoErrorProduccion.NoEncontrado, (await caso.ObtenerAsync(Guid.NewGuid())).Error.Tipo);
        Assert.Equal(TipoErrorProduccion.NoEncontrado, (await
caso.PlanificarAsync(Guid.NewGuid())).Error.Tipo);
        Assert.Empty(db.ChangeTracker.Entries());
        Assert.Equal(0, await db.Ordenes.CountAsync());
    }
    [Fact]
    public void Resultado_impide_observar_la_rama_incorrecta()
    {
        var ok = ResultadoProduccion<int>.Exito(5);
        var fallo = ResultadoProduccion<int>.Fallo(new("x", "Inválida", TipoErrorProduccion.Validacion));
        Assert.Equal(5, ok.Valor);
        Assert.Throws<InvalidOperationException>(() => ok.Error);
        Assert.Throws<InvalidOperationException>(() => fallo.Valor);
        Assert.Throws<ArgumentNullException>(() => ResultadoProduccion<string>.Exito(null!));
        Assert.Throws<ArgumentNullException>(() => ResultadoProduccion<int>.Fallo(null!));
    }
    [Fact]
    public async Task Http_crea_201_y_location_apunta_al_recurso_persistido()
    {
        using var factory = new ProduccionApiFactory(); using var client = factory.CreateClient();
        var respuesta = await client.PostAsJsonAsync("/api/v1/produccion/ordenes", new { Cliente = " Axia ",
Material = " Acero ", Cantidad = 20 });
        Assert.Equal(HttpStatusCode.Created, respuesta.StatusCode);
        Assert.NotNull(respuesta.Headers.Location);
        var dto = await client.GetFromJsonAsync<OrdenProduccionDto>(respuesta.Headers.Location);
        Assert.Equal("Axia", dto!.Cliente);
        Assert.Equal("Acero", Assert.Single(dto.Lineas).Material);
        await using var lectura = factory.NuevoContexto();
        Assert.Equal(dto.Id, (await lectura.Ordenes.SingleAsync()).Id);
    }
    [Fact]
    public async Task Http_rechaza_datos_invalidos_con_400_antes_de_persistir()
    {
        using var factory = new ProduccionApiFactory(); using var client = factory.CreateClient();
        var casos = new[] { (" ", "Acero", 20), ("Axia", " ", 20), ("Axia", "Acero", 0),
            ("Axia", "Acero", 501), (new string('C', 101), "Acero", 1), ("Axia", new string('M', 101), 1),
("Axia", (string)null!, 1) };
        foreach (var (cliente, material, cantidad) in casos)
        {
            var respuesta = await client.PostAsJsonAsync("/api/v1/produccion/ordenes", new { Cliente =
cliente, Material = material, Cantidad = cantidad });
            Assert.Equal(HttpStatusCode.BadRequest, respuesta.StatusCode);
        }
        await using var lectura = factory.NuevoContexto();
        Assert.Equal(0, await lectura.Ordenes.CountAsync());
    }
    [Fact]
    public async Task Http_distingue_404_y_409_sin_alterar_el_estado()
    {
        using var factory = new ProduccionApiFactory(); using var client = factory.CreateClient();
        var ausente = $"/api/v1/produccion/ordenes/{Guid.NewGuid()}";
        Assert.Equal(HttpStatusCode.NotFound, (await client.GetAsync(ausente)).StatusCode);
        Assert.Equal(HttpStatusCode.NotFound, (await client.PostAsync(ausente + "/finalizar",
null)).StatusCode);
        var creada = await client.PostAsJsonAsync("/api/v1/produccion/ordenes", new { Cliente = "Axia",
Material = "Acero", Cantidad = 20 });
        var conflicto = await client.PostAsync(creada.Headers.Location + "/finalizar", null);
        Assert.Equal(HttpStatusCode.Conflict, conflicto.StatusCode);
        Assert.Contains("estado.invalido", await conflicto.Content.ReadAsStringAsync());
        await using var lectura = factory.NuevoContexto();
        Assert.Equal(EstadoOrdenProduccion.Borrador, (await lectura.Ordenes.SingleAsync()).Estado);
    }
}
```

Comprueba el comportamiento observable: repositorio no guarda hasta el commit y rehidrata el agregado. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos. Comprueba el comportamiento observable: casos de uso confirman cada transicion en otra

unidad de trabajo. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos. Comprueba el comportamiento observable: errores esperados son resultados tipados sin escrituras parciales. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos.

Comprueba el comportamiento observable: resultado impide observar la rama incorrecta. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos. Comprueba el comportamiento observable: http crea 201 y location apunta al recurso persistido. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos. Comprueba el comportamiento observable: http rechaza datos invalidos con 400 antes de persistir. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos.

Comprueba el comportamiento observable: http distingue 404 y 409 sin alterar el estado. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos.

Integración funcional. Configura ProduccionDbContext sobre una base SQL Server aislada y permite verificar los datos desde contextos con conexiones independientes.

Archivo: `tests/Compartido/ProduccionApiFactory.cs`. **Operación:** Crear.

```
using AceriaApp.Produccion.Infraestructura;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.AspNetCore.TestHost;
using AceriaApp.PruebasSqlServer;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.DependencyInjection.Extensions;
namespace AceriaApp.PruebasProduccion;

// Cada factory posee una base SQL Server aislada y la elimina al terminar.
public sealed class ProduccionApiFactory : WebApplicationFactory<Program>
{
    private readonly BaseSqlServer store = BaseSqlServer.Crear();
    public ProduccionApiFactory()
    {
        try
        {
            using var db = NuevoContexto();
            if (!db.Database.IsSqlServer()) throw new InvalidOperationException("La prueba exige SQL
Server.");
            db.Database.EnsureCreated();
        }
        catch { store.Dispose(); throw; }
    }
    public ProduccionDbContext NuevoContexto() => new(
        new DbContextOptionsBuilder<ProduccionDbContext>().UseSqlServer(store.ConnectionString).Options);
    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.UseEnvironment("Development");
        builder.UseSetting("SkipDatabaseInitialization", "true");
        builder.ConfigureTestServices(services =>
        {
            services.RemoveAll<DbContextOptions<ProduccionDbContext>>();
            services.AddDbContext<ProduccionDbContext>(o => o.UseSqlServer(store.ConnectionString));
        });
    }
    protected override void Dispose(bool disposing)
    { base.Dispose(disposing); if (disposing) store.Dispose(); }
}
```

Crea una base SQL Server aislada para el host y las lecturas de esta prueba. Rechaza una configuración que no use el proveedor exigido. Crea las tablas de Producción antes de arrancar el host.

Cada contexto independiente abre su conexión a la misma base de la prueba. Evita inicializar las bases configuradas para la ejecución habitual de la API. El host usa SQL Server con la conexión aislada de esta factory.

Cierra el host antes de eliminar únicamente la base de esta prueba.

Verificación concreta. La solución contiene 20 proyectos, los archivos de integración coinciden con los bloques y la compilación completa termina con cero errores.

Diagnóstico si falla.

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.
- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

PASO 5.3.17 — Ejecutar todos los tests del punto 5.3

Las pruebas de catálogo verifican los datos declarados; las pruebas funcionales ejecutan las reglas y los adaptadores reales del contexto. Ambas evidencias son necesarias.

1. Ejecuta `dotnet test tests/AceriaApp.CasosDeUso.Tests/AceriaApp.CasosDeUso.Tests.csproj -c Release` desde la raíz de la etapa.
2. Comprueba 19 pruebas aprobadas: 12 de catálogo y 7 funcionales.
3. Exige cero fallos y cero omisiones; revisa los nombres de las pruebas para identificar el contrato comprobado.
4. Si falla una prueba, lee su aserción y corrige el archivo del paso correspondiente antes de continuar.

Verificación concreta. El proyecto ejecuta 19 tests: `Failed: 0, Passed: 19, Skipped: 0, Total: 19`.

Diagnóstico si falla.

- Ejecutar `dotnet test` sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

PASO 5.3.18 — Ejecutar todos los tests de la solución

Ejecutar todos los tests de la solución verifica que los tests de los puntos anteriores y del punto 5.3 pasan conjuntamente.

1. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build` después de la compilación Release.
2. Comprueba que se ejecutan los 13 proyectos de tests, incluidos los heredados de M04.
3. Suma los resultados por proyecto: deben ser 172 pruebas aprobadas, cero fallos y cero omisiones.
4. Conserva la salida como evidencia; si cambias código vuelve a compilar antes de utilizar `--no-build`.

Verificación concreta. Los 13 proyectos ejecutan conjuntamente 172 tests aprobados, cero fallos y cero omisiones.

Diagnóstico si falla.

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Omitir proyectos heredados de la solución: se pierde la comprobación de regresión.
- Eliminar o modificar pruebas anteriores sin una instrucción explícita: se pierde la cobertura heredada.

PASO 5.3.19 — Actualizar el README de la copia 5.3

El README del punto 5.3 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 5.2.

1. Abre el archivo `README.md` copiado de `5.2-entidades-value-objects` dentro de `5.3-casos-de-uso`.
2. Reemplaza su contenido completo por el bloque de este paso; el README de la carpeta anterior permanece intacto.
3. Comprueba el objetivo, el estado, la ejecución, los añadidos, la documentación y la verificación ejecutable.
4. Guarda el archivo y abre su vista previa para comprobar títulos, enlaces, rutas y cifras.

Archivo: `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 5.3 — Casos de uso

## Objetivo
Comprender los casos de uso como expresión de las operaciones del dominio y añadir al proyecto las clases de validación de casos de uso.

Conectar los casos de uso de Producción con sus puertos, persistencia y API.

## Estado del proyecto
- Solución `AceriaApp.sln` con 20 proyectos.
- Documento `docs/casos-uso.md` con la definición y las reglas.
- Clases `CasoDeUso`, `ValidadorCasosDeUso` y `CatalogoCasosDeUso` en `src/AceriaApp.Api/CasosDeUso/`.
- 12 tests de catálogo y 7 funcionales nuevos en `tests/AceriaApp.CasosDeUso.Tests`.
- Los tests acumulados suman 172 pruebas.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio con soporte de .NET 8.
2. Configurar SQL Server para los adaptadores heredados; las pruebas proporcionan su persistencia de prueba.
3. Configurar las conexiones `AceriaApp`, `AceriaHexagonal` y `AceriaProduccion` en el entorno local si se ejecuta la API con SQL Server.
4. Ejecutar `dotnet run --project src/AceriaApp.Api/AceriaApp.Api.csproj --urls http://localhost:5000` desde la raíz de la etapa.
5. Navegar a `http://localhost:5000/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

El perfil de lanzamiento heredado activa `Development`. En ese entorno, si `SkipDatabaseInitialization` es falso, la API inicializa la base de Producción mediante `EnsureCreatedAsync`; no requiere datos iniciales. Configura `ConnectionStrings:AceriaProduccion` para tu SQL Server antes del arranque.

Para probar el nuevo flujo, envía `POST /api/v1/produccion/ordenes` con `{"cliente":"Axia","material":"Acero","cantidad":200}`, sigue su cabecera `Location` y envía los comandos `POST /{id}/planificar` y `POST /{id}/finalizar` bajo la ruta de órdenes.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/casos-uso.md`.
- Carpeta `CasosDeUso` en el proyecto API con tres clases nuevas.
- Proyecto `tests/AceriaApp.CasosDeUso.Tests` con 12 tests de catálogo y 7 funcionales nuevos.
- Se conserva intacta la carpeta del punto 5.2.

- Conectar los casos de uso de Producción con sus puertos, persistencia y API. Los bloques completos están en el paso 16 del manual.

## Documentación
- [ADR-010](docs/ADR-010-capas-aplicacion.md) a [ADR-020](docs/ADR-020-entidades-value-objects.md) (heredados).
- [Casos de uso](docs/casos-uso.md).

## Verificación ejecutable

El catálogo describe ejemplos de diseño y sus indicadores; no inspecciona automáticamente las clases reales. Los archivos funcionales del paso 16 y sus tests comprueban el comportamiento ejecutable. Ejecuta `dotnet test AceriaApp.sln -c Release` y exige cero fallos y cero omisiones.
```

Verificación concreta. El archivo `README.md` existe; comprueba que sus títulos, contenido y enlaces coinciden con todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Modificar el README del punto 5.2: rompe la conservación de la etapa anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 5.2: rompe la separación.

PASO 5.3.20 — Crear el ADR-021 del punto 5.3

El ADR-021 documenta la decisión de adoptar el caso de uso como unidad de operación. Es un artefacto nuevo del punto 5.3 que no modifica los ADR existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `ADR-021-casos-uso.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que los ADR-010 a ADR-020 no se han modificado.

Archivo: `docs/ADR-021-casos-uso.md`. **Operación:** Crear.

```
# ADR-021: Casos de uso

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado DDD y ha definido entidades y value objects. Es necesario definir cómo se
expresan las operaciones del dominio y cómo se coordinan los elementos del modelo.

## Decisión
Se implementan casos de uso de Producción para crear, obtener, planificar y finalizar una orden.
`AceriaApp.Produccion.Aplicacion` depende del dominio y declara `IOrdenProduccionRepositorio`,
`IProduccionUnitOfWork` y `ResultadoProduccion<T>`. `AceriaApp.Produccion.Infraestructura` implementa la
persistencia mediante `ProduccionDbContext`. La API traduce el resultado a HTTP y compone las dependencias.
Los casos de uso orquestan la carga del agregado, sus operaciones y la confirmación; el agregado conserva las
invariantes. Las pruebas de lógica emplean dobles de puertos y las de integración utilizan SQL Server para
comprobar el comportamiento relacional. MediatR se añade a este contexto en 5.4; los manejadores anteriores
de M04 permanecen independientes.

## Alternativas consideradas
- Servicios de aplicación sin casos de uso explícitos: descartado por falta de intención.
- Representar todos los errores esperados mediante excepciones: se prefiere un resultado explícito para
distinguirlos de fallos técnicos.
- Invocar EF Core directamente desde aplicación: se prefiere el puerto de unidad de trabajo para desacoplar
la confirmación. La atomicidad la aporta el adaptador transaccional, no la interfaz por sí sola.

## Consecuencias
- Positivas: operaciones explícitas, coordinación clara, errores explícitos.
- Negativas: los contratos y adaptadores añaden archivos y decisiones de composición.
- Mitigación: la validación automática y la convención de nombres.

## Alcance del catálogo

`CatalogoCasosDeUso` contiene ejemplos CrearOrden, CancelarOrden, ObtenerOrden y ListarOrdenes. Sus
indicadores son datos pedagógicos; no constituyen un inventario de métodos implementados en Producción. En
este punto se ejecutan crear, obtener, planificar y finalizar una orden. El nuevo contexto tiene su propia
persistencia y no comparte entidades con el flujo hexagonal heredado. Los tests funcionales comprueban el
orden de las operaciones y la persistencia desde otro contexto de prueba.
```

Verificación concreta. El archivo `docs/ADR-021-casos-uso.md` existe; comprueba que sus títulos, contenido y enlaces coinciden con todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Modificar los ADR anteriores: rompe la conservación de la etapa anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-021`: rompe la convención.

PASO 5.3.21 — Comparar los documentos heredados con 5.2-entidades-value-objects

La comparación comprueba los documentos heredados byte a byte. Las nuevas clases y los cambios explícitos de integración pertenecen a la copia actual; la carpeta del punto anterior permanece intacta.

1. En la terminal, escribir `cd C:\cursos\AceriaApp` y pulsar Enter.
2. Escribir `for %f in (M05_DDD_CQRS\PROYECTO\5.2-entidades-value-objects\docs\*.md) do fc /L "%f" "M05_DDD_CQRS\PROYECTO\5.3-casos-de-uso\docs\%~nxf"` y pulsar Enter.
3. Verificar que la salida muestra `FC: no differences encountered` para cada archivo.
4. Verificar que el número de comparaciones coincide con el número de documentos del punto 5.2. Si algún archivo muestra diferencias, restaurarlo desde el punto 5.2.

Verificación concreta. Los 22 documentos heredados coinciden con la etapa anterior, sin omisiones ni modificaciones.

Diagnóstico si falla.

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.
- Omitir `/L` cuando se desea una comparación de texto: indica el modo explícitamente y revisa cada resultado.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

PASO 5.3.22 — Verificar la estructura del punto 5.3

La verificación estructural confirma que el punto 5.3 añade archivos nuevos conservando los documentos heredados.

1. Escribir `tree /F M05_DDD_CQRS\PROYECTO\5.3-casos-de-uso` y pulsar Enter.
2. Verificar que la salida muestra `AceriaApp.sln`, `README.md`, `src\AceriaApp.Api\CasosDeUso` con las tres clases nuevas, `tests\AceriaApp.CasosDeUso.Tests` con los dos archivos de tests de catálogo y las pruebas funcionales y `docs` con el documento de casos de uso y el ADR-021. Comprueba también los proyectos y archivos del contexto Producción indicados en el paso 16.
3. Verificar que las carpetas existentes del punto 5.2 permanecen intactas.
4. Verificar que no existe ninguna carpeta `labs` ni ningún archivo `metaprompt.md` ni `verificacion.md`.

Verificación concreta. la estructura contiene los archivos nuevos y conserva los del punto 5.2.

Diagnóstico si falla.

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 5.2: viola la regla de no modificar.
- Omitir la carpeta `CasosDeUso`: los tests no encuentran las clases.

PASO 5.3.23 — Verificar el `.gitignore` del punto 5.3

El `.gitignore` se hereda del punto 5.2 sin modificaciones. La verificación confirma que no se ha tocado.

1. Verificar que el archivo `.gitignore` existe en la carpeta del punto 5.3 (copiado del punto 5.2).
2. Abrir el archivo `.gitignore` con Visual Studio con soporte de .NET 8.
3. Verificar que contiene las líneas `**/bin/`, `**/obj/`, `.vs/`, `*.user`, `TestResults/` y `*.trx`.
4. Verificar que no se ha modificado respecto al punto 5.2. Pulsar `Ctrl+S` si no hay cambios y cerrar el archivo.

Verificación concreta. el `.gitignore` existe y contiene las seis líneas sin modificaciones.

Diagnóstico si falla.

- Modificar el `.gitignore` copiado del punto 5.2: rompe la conservación de la etapa anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el `.gitignore` con otro nombre: el archivo no se reconoce.

PASO 5.3.24 — Hacer el commit del punto 5.3 y verificar el estado final

El commit registra la etapa completa. Si existe uno por punto desde 1.1, el recorrido suma 21 commits de puntos; otros commits legítimos pueden aumentar el historial.

1. Ejecuta `git diff --stat` y revisa los archivos y entregables de esta etapa.
2. Ejecuta `git add .` después de comprobar que no se incluyen binarios, resultados temporales ni credenciales.
3. Ejecuta `git commit -m "Punto 5.3: Casos de uso"` con `user.name` y `user.email` configurados.
4. Comprueba `git log --oneline` y `git status`; deben registrar este punto y no dejar cambios pendientes de esta actividad.

Verificación concreta. El historial incluye el commit de 5.3 y `git status` no muestra cambios pendientes de esta actividad; no se impone una longitud rígida al historial.

Diagnóstico si falla.

- No tener configurado `user.name` y `user.email`: `git commit` falla.
- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

Reto de ampliación (optativo)

Trabaja en una copia independiente de la etapa obligatoria, por ejemplo `5.3-casos-de-uso-reto`. Conserva intacta la carpeta obligatoria y sus documentos. Los tres fragmentos añaden un contrato, un cálculo y una prueba al proyecto del punto.

Reto: añadir un método `Resumen` a `ValidadorCasosDeUso` que devuelva el número total de casos de uso, el número de escrituras, el número de lecturas y el número de casos que coordinan la unidad de trabajo, y un test que verifique el cálculo con el catálogo por defecto.

Paso 1: En `ValidadorCasosDeUso.cs`, añadir el siguiente `record` al final del archivo:

Archivo: `src/AceriaApp.Api/CasosDeUso/ValidadorCasosDeUso.cs`. **Operación:** Modificar.

Punto de inserción: después de este texto exacto, que debe aparecer una sola vez:

```
public record ResultadoCasosDeUso(bool EsValido, IReadOnlyList<string> Errores);
```

```
public record ResumenCasosDeUso(
    int Total,
    int Escrituras,
    int Lecturas,
    int CoordinanUnidadDeTrabajo);
```

Paso 2: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public record ResumenCasosDeUso(` declara un contrato inmutable. Número total de casos de uso. Número de casos de escritura. Número de casos de lectura. Número de casos que coordinan la unidad.

Paso 3: En `ValidadorCasosDeUso.cs`, añadir el siguiente método estático dentro de la clase:

Archivo: `src/AceriaApp.Api/CasosDeUso/ValidadorCasosDeUso.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}

public record ResultadoCasosDeUso(bool EsValido, IReadOnlyList<string> Errores);
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase indicado por el texto de inserción.

```
public static ResumenCasosDeUso Resumen(IEnumerable<CasoDeUso> casos)
{
    var lista = casos.ToList();
    var escrituras = lista.Count(c => c.Tipo == "Escritura");
    var lecturas = lista.Count(c => c.Tipo == "Lectura");
    var coordinan = lista.Count(c => c.CoordinaUnidadDeTrabajo);
    return new ResumenCasosDeUso(lista.Count, escrituras, lecturas, coordinan);
}
```

Paso 4: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public static ResumenCasosDeUso Resumen(IEnumerable<CasoDeUso> casos)` declara el método. Materializa la lista. Cuenta las escrituras. Cuenta las lecturas. Cuenta los que coordinan. Devuelve el resumen.

Paso 5: Guardar el archivo con `Ctrl+S`.

Paso 6: En `ValidadorCasosDeUsoTests.cs`, añadir el siguiente test:

Archivo: `tests/AceriaApp.CasosDeUso.Tests/ValidadorCasosDeUsoTests.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase indicado por el texto de inserción.

```
[Fact]
public void Resumen_ConCatalogoPorDefecto_CalculaTotales()
{
    var resumen = ValidadorCasosDeUso.Resumen(CatalogoCasosDeUso.CasosPorDefecto());

    resumen.Total.Should().Be(4);
    resumen.Escrituras.Should().Be(2);
    resumen.Lecturas.Should().Be(2);
    resumen.CoordinanUnidadDeTrabajo.Should().Be(2);
}
```

Paso 7: Explicar línea por línea:

Marca el método como test de xUnit. Declara el test. Invoca el método. Verifica el total. Verifica las escrituras. Verifica las lecturas. Verifica los que coordinan.

Paso 8: Ejecuta `dotnet test tests/AceriaApp.CasosDeUso.Tests/AceriaApp.CasosDeUso.Tests.csproj -c Release` en la copia optativa. Deben pasar 20 tests del proyecto, incluido el nuevo resumen. Ejecuta después `dotnet test AceriaApp.sln -c Release` y exige 173 pruebas aprobadas.

Paso 9: Comprueba que solo cambian `ValidadorCasosDeUso.cs` y `ValidadorCasosDeUsoTests.cs` en la copia optativa; la etapa obligatoria `5.3-casos-de-uso` y el punto anterior permanecen idénticos.

Paso 10: Hacer commit con `git commit -m "Reto 5.3: método Resumen en ValidadorCasosDeUso"`.

Verificación: `Passed: 20` en el proyecto del punto y 173 pruebas aprobadas en la solución de la copia, cero fallos y cero omisiones.

Resultado verificable

Al finalizar este punto, el alumno debe tener:

- La carpeta `M05_DDD_CQRS/PROYECTO/5.3-casos-de-uso/` con la solución `AceriaApp.sln` que contiene 20 proyectos.
- El documento `docs/casos-uso.md` con la definición y las reglas.
- El documento `docs/ADR-021-casos-uso.md` con la decisión sobre los casos de uso.
- Las clases `CasoDeUso`, `ValidadorCasosDeUso` y `CatalogoCasosDeUso` en `src/AceriaApp.Api/CasosDeUso/`.
- El proyecto `tests/AceriaApp.CasosDeUso.Tests` añade 12 tests de catálogo y 7 funcionales; el reto añade uno más.
- `dotnet test AceriaApp.sln -c Release` ejecuta 172 pruebas en la etapa obligatoria y 173 en una copia con su reto; ninguna falla ni se omite.
- Los ADR-010 a ADR-020 del punto 5.2 sin modificar.
- Un commit revisado para 5.3, con el historial previo conservado.

Continuidad del aprendizaje

El punto 5.3 ha definido los casos de uso como expresión de las operaciones del dominio. El proyecto AceriaApp dispone ahora de un validador automático de casos de uso que comprueba la presencia de manejador, el uso de Result, la coordinación de la unidad de trabajo y la ausencia de reglas de negocio, y 12 tests de catálogo más 7 funcionales nuevos. Esta base permite avanzar hacia el punto 5.4, donde se estudiará la separación entre commands y queries como base de CQRS.

Punto 5.4 — Separación entre commands y queries

Distinguir comandos y consultas sin duplicar el dominio

CQRS consiste en separar la intención de modificar estado de la necesidad de obtener información. No exige automáticamente dos bases de datos ni un bus de eventos: en una primera aplicación puede materializarse mediante handlers y contratos distintos sobre una persistencia compartida. La etapa 5.4 comprobará esa diferencia en Producción, manteniendo las invariantes del agregado y evitando que una query modifique datos.

La práctica introduce descripciones de mensajes y validaciones para analizar la separación, pero las pruebas deben seguir también un comando real desde HTTP hasta la escritura y una consulta real desde el almacenamiento hasta el DTO. Una proyección de lectura no es una entidad de dominio disfrazada; su forma responde a lo que necesita el consumidor. Distinguiremos esas garantías de las que podrían exigir más adelante mensajería, consistencia eventual o un modelo de lectura distribuido.

Carpeta de trabajo: `M05_DDD_CQRS/PROYECTO/5.4-commands-queries`.

Objetivo: completar los veinticuatro pasos, conservar los documentos heredados y comprobar el modelo con código y tests ejecutables. La etapa suma 190 pruebas en 21 proyectos.

Ejercicio guiado. Añadir al punto 5.3 un documento de separación entre comandos y consultas en `docs/`, una clase de validación de la separación en el proyecto API y un test unitario que verifique la validación. El punto 5.4 parte del contenido del punto 5.3 conservando la carpeta anterior y los documentos heredados; los cambios de integración se indican explícitamente en esta copia. La implementación del paso 16 añade comportamiento ejecutable del contexto Producción al ejercicio de catálogo.

PASO 5.4.1 — Crear la carpeta del punto 5.4 a partir del punto 5.3

La copia conserva una etapa autónoma. 5.3-casos-de-uso aporta 172 tests aprobados y 20 proyectos antes de incorporar este punto.

1. Abre la terminal integrada y localiza `M05_DDD_CQRS/PROYECTO/5.3-casos-de-uso`; esa es la etapa completa que vas a continuar.
2. Crea `M05_DDD_CQRS/PROYECTO/5.4-commands-queries` y copia en ella el contenido de `5.3-casos-de-uso`; en Windows puedes usar `xcopy` con `/E /I /H /Y`.
3. Excluye `bin`, `obj` y `TestResults`; abre únicamente la copia de `5.4-commands-queries` para conservar intacto el punto anterior.
4. Ejecuta `dir` dentro de `5.4-commands-queries` y verifica `AceriaApp.sln`, `README.md`, `src`, `tests`, `docs` y `global.json`.

Verificación concreta. La copia contiene la solución y los 20 proyectos anteriores; todavía no contiene los archivos de 5.4.

Diagnóstico si falla.

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.

PASO 5.4.2 — Abrir la solución copiada en Visual Studio con soporte de .NET 8

Abrir la solución copiada permite trabajar en el punto 5.4 sobre la base del punto 5.3. La compilación verifica que la copia es correcta.

1. Abre `AceriaApp.sln` dentro de `5.4-commands-queries` con Visual Studio; en Windows puedes ejecutar `start AceriaApp.sln`.
2. Ejecuta `dotnet sln AceriaApp.sln list` y comprueba los 20 proyectos heredados; todas sus rutas deben existir en esta etapa.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release` para comprobar el punto de partida.
4. Si hay DLL bloqueadas por una API en ejecución, detén esa aplicación y repite la compilación. Continúa cuando no haya errores.

Verificación concreta. La solución muestra 20 proyectos y `dotnet build AceriaApp.sln -c Release` termina con `Build succeeded.`.

Diagnóstico si falla.

- Abrir la solución del punto 5.3 en lugar de la del punto 5.4: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener la API del punto 5.3 en ejecución: el puerto queda ocupado y la compilación falla.

PASO 5.4.3 — Verificar que la carpeta `docs` contiene los documentos del punto 5.3

La nueva etapa preserva 24 documentos y añade su ficha temática y su ADR. Los cambios de integración se aplican únicamente en la copia actual.

1. Expande `docs` dentro de la copia de `5.4-commands-queries`.
2. Comprueba los 24 documentos heredados: ADR-010 a ADR-021 y fichas de los puntos anteriores.
3. Compara sus nombres y contenido con `docs` de `5.3-casos-de-uso`; no modifiques ni elimines documentos anteriores.
4. Confirma que todavía no existen `docs/commands-queries.md` ni `docs/ADR-022-commands-queries.md`.

Verificación concreta. Existen 24 documentos heredados idénticos y no existen todavía los dos de este punto.

Diagnóstico si falla.

- Modificar alguno de los ADR o documentos existentes: rompe la conservación de la etapa anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 5.3.
- Crear la carpeta `docs` si ya existe: `mkdir` falla.

PASO 5.4.4 — Crear el documento de separación entre comandos y consultas en `docs`

El documento de separación entre comandos y consultas es el artefacto de `docs` específico del punto 5.4. No modifica los 24 documentos existentes del punto 5.3.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `commands-queries.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que el archivo aparece en la carpeta `docs` sin subrayados rojos.

Archivo: `docs/commands-queries.md`. **Operación:** Crear.

```
# Separación entre Commands y Queries — Punto 5.4

## CQRS

Patrón arquitectónico que separa las operaciones de escritura de las operaciones de lectura.

## Commands

- Expresan una intención de cambio.
- Solicitan cambios; la entrega exactamente una vez no está garantizada por CQRS.
- Su idempotencia depende del contrato y de su implementación; deben decidir cómo tratar duplicados.

## Queries

- Expresan una solicitud de información.
- Son idempotentes.
- No modifican el estado.

## Separación de modelos

- Modelo de escritura: entidades, value objects, invariantes.
- Modelo de lectura: proyecciones, DTOs, vistas.

## Beneficios

- Separación de responsabilidades.
- Optimización por modelo.
- Escalabilidad independiente.
```

```
- Evolución independiente.

## Costes

- Complejidad adicional.
- Consistencia eventual cuando las proyecciones se actualizan de forma asíncrona; no es obligatoria en CQRS.
- Duplicación de conceptos.
- Sincronización entre modelos.

## CQRS con un único modelo de datos

La separación distingue contratos de escritura y proyecciones de lectura. Pueden usar una misma base de
datos; no se requiere duplicar el almacenamiento.

## CQRS con dos modelos de datos

El modelo de escritura y el de lectura usan bases de datos distintas. La propagación se realiza con eventos.

## Cuándo aplicar CQRS

- Lecturas complejas: sí.
- Alto volumen de lectura: sí.
- Requisitos distintos: sí.
- CRUD simple: no.
- Consistencia fuerte: compatible con una base de datos compartida y proyección síncrona.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Commands ejecutables | Crear, planificar y finalizar órdenes de Producción |
| Queries ejecutables | Obtener y listar proyecciones de órdenes de Producción |
| Manejadores | Separados por tipo |
| Base de datos de Producción | Única para escritura y lectura, sin sincronización asíncrona |
| Validación | Test de coherencia |

## Correspondencia ejecutable

Los nombres de `CatalogoCqrs` son ejemplos de análisis y no registran manejadores. `DevuelveResult` es una
convención del laboratorio, no un requisito universal de CQRS: una consulta puede devolver directamente un
DTO y un comando puede expresar el resultado con otro contrato. El flujo de Producción dispone de sus propios
mensajes de crear, planificar, finalizar, obtener y listar. Los comandos trabajan sobre el agregado mediante
los casos de uso; las consultas construyen una proyección DTO sin seguimiento de EF Core. La unidad de
trabajo no se usa para confirmar una consulta. Los registros MediatR del contexto conservan el flujo heredado
de M04.
```

Verificación concreta. El archivo `docs/commands-queries.md` existe; comprueba que sus títulos, contenido y enlaces coinciden con todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: rompe la conservación de la etapa anterior.
- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

PASO 5.4.5 — Crear la carpeta `Cqrs` en el proyecto API

La carpeta `Cqrs` albergará las clases de validación de la separación. Se crea como carpeta nueva sin tocar las carpetas existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.Api`.
2. Seleccionar `Add` → `New Folder` y nombrar la carpeta `Cqrs`.
3. Verificar que la carpeta `Cqrs` aparece dentro de `AceriaApp.Api` sin afectar a las carpetas existentes.
4. Verificar que no se ha modificado ningún archivo existente del punto 5.3.

Verificación concreta. la carpeta `Cqrs` existe dentro de `AceriaApp.Api` y las carpetas existentes permanecen intactas.

Diagnóstico si falla.

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: rompe la conservación de la etapa anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

PASO 5.4.6 — Crear la clase `MensajeCqrs`

`MensajeCqrs` identifica si una operación pretende consultar o cambiar estado. Es una descripción docente; los handlers y DTO de lectura reales deberán respetar esa separación cuando se ejecuten desde la API.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `Cqrs`.
2. Seleccionar `Add` → `Class` y nombrar la clase `MensajeCqrs.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/Cqrs/MensajeCqrs.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Cqrs;

public class MensajeCqrs
{
    public string Nombre { get; init; } = string.Empty;
    public string Tipo { get; init; } = string.Empty;
    public bool TieneManejador { get; init; }
    public bool UsaSoloLectura { get; init; }
    public bool UsaEscritura { get; init; }
    public bool DevuelveResult { get; init; }
}
```

Verificación concreta. la clase `MensajeCqrs` compila sin errores y expone seis propiedades.

Diagnóstico si falla.

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar las propiedades de tipo `string` con `string.Empty`: el compilador emite advertencias.
- Usar `set` en lugar de `init`: permite modificar la instancia después de crearla.

PASO 5.4.7 — Crear la clase `ValidadorCqrs`

`ValidadorCqrs` comprueba criterios declarados de mensajes. No puede acreditar que una query no escriba: esa afirmación requiere ejecutar la consulta y observar la base antes y después.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `Cqrs`.
2. Seleccionar `Add` → `Class` y nombrar la clase `ValidadorCqrs.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Convención del laboratorio. El indicador `DevuelveResult` se exige en estos ejemplos para practicar el contrato de resultado. CQRS no impone ese tipo de retorno; una consulta puede devolver un DTO directamente. El indicador no inspecciona el tipo devuelto por los manejadores reales.

Alcance del validador. Comprueba los indicadores de los ejemplos del catálogo. No analiza por reflexión los tipos de producción ni garantiza por sí solo la arquitectura. Las reglas del modelo ejecutable y sus límites se comprueban en las pruebas funcionales del paso 16.

Archivo: `src/AceriaApp.Api/Cqrs/ValidadorCqrs.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Cqrs;

public class ValidadorCqrs
{
    private static readonly string[] TiposPermitidos = { "Command", "Query" };

    public ResultadoCqrs Validar(IEnumerable<MensajeCqrs> mensajes)
    {
        var errores = new List<string>();
        var lista = mensajes.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay mensajes declarados.");
        }

        foreach (var mensaje in lista)
        {
            if (string.IsNullOrWhiteSpace(mensaje.Nombre))
            {
                errores.Add("Un mensaje no declara el nombre.");
            }

            if (!TiposPermitidos.Contains(mensaje.Tipo))
            {
                errores.Add($"Tipo no permitido en '{mensaje.Nombre}': '{mensaje.Tipo}'.");
            }

            if (!mensaje.TieneManejador)
            {
                errores.Add($"El mensaje '{mensaje.Nombre}' no tiene manejador.");
            }

            if (!mensaje.DevuelveResult)
            {
                errores.Add($"El mensaje '{mensaje.Nombre}' no devuelve Result.");
            }

            if (mensaje.Tipo == "Command" && !mensaje.UsaEscritura)
            {
                errores.Add($"El command '{mensaje.Nombre}' no usa escritura.");
            }

            if (mensaje.Tipo == "Query" && !mensaje.UsaSoloLectura)
            {
                errores.Add($"La query '{mensaje.Nombre}' no es de solo lectura.");
            }

            if (mensaje.Tipo == "Query" && mensaje.UsaEscritura)
            {
                errores.Add($"La query '{mensaje.Nombre}' usa escritura.");
            }
        }

        return new ResultadoCqrs(errores.Count == 0, errores);
    }
}

public record ResultadoCqrs(bool EsValido, IReadOnlyList<string> Errores);
```

Verificación concreta. la clase `ValidadorCqrs` compila sin errores y expone un método y un `record`.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<string>` e `IReadOnlyList<string>` no se resuelven.
- Olvidar el `using System.Linq;`: `Contains` y `ToList` no se resuelven.
- No comprobar el caso de lista vacía: el validador devuelve válido sin mensajes.

PASO 5.4.8 — Añadir los `using` necesarios en `ValidadorCqrs`

Los tipos `List<string>` e `IReadOnlyList<string>` pertenecen a `System.Collections.Generic`. Los métodos `Contains` y `ToList` pertenecen a `System.Linq`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. En el panel Solution Explorer, hacer doble clic en `ValidadorCqrs.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; el texto de inserción debe aparecer exactamente una vez.
3. Verificar que no hay subrayados rojos en `List<string>`, `Contains` ni `ToList`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/Cqrs/ValidadorCqrs.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.Cqrs;
```

```
using System.Collections.Generic;
using System.Linq;
```

Verificación concreta. `ValidadorCqrs.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir solo `System.Collections.Generic` y olvidar `System.Linq`: `Contains` y `ToList` no se resuelven.
- Mezclar directivas using con declaraciones de tipos: coloca las directivas antes del namespace para mantener el orden del archivo.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 5.4.9 — Crear la clase `CatalogoCqrs`

`CatalogoCqrs` organiza comandos y consultas como lenguaje de análisis. El código de aplicación continúa necesitando contratos y handlers verificables que protejan el estado de Producción.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `Cqrs`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoCqrs.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/Cqrs/CatalogoCqrs.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Cqrs;

public class CatalogoCqrs
{
    public static List<MensajeCqrs> MensajesPorDefecto()
    {
        return new List<MensajeCqrs>
        {
            new() { Nombre = "CrearOrdenCommand", Tipo = "Command", TieneManejador = true, UsaEscritura =
true, UsaSoloLectura = false, DevuelveResult = true },
            new() { Nombre = "CancelarOrdenCommand", Tipo = "Command", TieneManejador = true, UsaEscritura =
true, UsaSoloLectura = false, DevuelveResult = true },
            new() { Nombre = "ObtenerOrdenQuery", Tipo = "Query", TieneManejador = true, UsaEscritura =
false, UsaSoloLectura = true, DevuelveResult = true },
            new() { Nombre = "ListarOrdenesQuery", Tipo = "Query", TieneManejador = true, UsaEscritura =
false, UsaSoloLectura = true, DevuelveResult = true }
        };
    }
}
```

Verificación concreta. la clase `CatalogoCqrs` compila sin errores y expone un método estático.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<T>` no se resuelve.
- No inicializar las propiedades de los objetos: el compilador emite advertencias.
- Reutilizar una lista estática mutable: cada llamada del catálogo debe construir una lista independiente.

PASO 5.4.10 — Añadir el `using` necesario en `CatalogoCqrs`

El tipo `List<T>` pertenece a `System.Collections.Generic`. En proyectos con `ImplicitUsings` desactivado, hay que añadir el using explícito.

1. En el panel Solution Explorer, hacer doble clic en `CatalogoCqrs.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; el texto de inserción debe aparecer exactamente una vez.
3. Verificar que no hay subrayados rojos en `List<T>`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/Cqrs/CatalogoCqrs.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.Cqrs;
```

```
using System.Collections.Generic;
```

Verificación concreta. `CatalogoCqrs.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir `using System.Linq;` en lugar de `System.Collections.Generic`: no resuelve `List<T>`.
- Colocar el using después de una clase: las directivas deben preceder las declaraciones de tipos.
- Olvidar el using y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 5.4.11 — Crear el proyecto de tests del punto 5.4

El punto 5.4 añade un proyecto de tests específico para las clases de CQRS. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

1. Crea `tests/AceriaApp.Cqrs.Tests` y su archivo `AceriaApp.Cqrs.Tests.csproj` con el XML completo. Si utilizas la plantilla xUnit de Visual Studio, ajusta el archivo generado.
2. Elimina `UnitTest1.cs` si lo creó la plantilla; las pruebas completas se añaden en los pasos 14 y 15.
3. Ejecuta `dotnet sln AceriaApp.sln add tests/AceriaApp.Cqrs.Tests/AceriaApp.Cqrs.Tests.csproj` incluso si la solución ya existe.
4. Ejecuta `dotnet sln AceriaApp.sln list` y `dotnet restore tests/AceriaApp.Cqrs.Tests/AceriaApp.Cqrs.Tests.csproj`.

Archivo: `tests/AceriaApp.Cqrs.Tests/AceriaApp.Cqrs.Tests.csproj`. **Operación:** Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
```

```
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. La solución muestra 21 proyectos, incluido `AceriaApp.Cqrs.Tests`; el csproj fija .NET 8, SDK de tests 17.12.0, xUnit 2.9.2 y runner 2.8.2.

Diagnóstico si falla.

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 5.3: rompe la conservación de la etapa anterior.
- Usar `dotnet new mstest`: el curso usa xUnit.

PASO 5.4.12 — Añadir la referencia del proyecto de tests a la API

La referencia permite que el proyecto de tests acceda a las clases de `Cqrs` del proyecto API.

1. Ejecuta `dotnet add tests/AceriaApp.Cqrs.Tests/AceriaApp.Cqrs.Tests.csproj reference src/AceriaApp.Api/AceriaApp.Api.csproj` o deja el archivo como el XML completo.
2. Comprueba que la referencia va del proyecto de tests a la API; la API no debe depender de los tests.
3. Abre `tests/AceriaApp.Cqrs.Tests/AceriaApp.Cqrs.Tests.csproj` y confirma la ruta `../../src/AceriaApp.Api/AceriaApp.Api.csproj`.
4. Ejecuta `dotnet build tests/AceriaApp.Cqrs.Tests/AceriaApp.Cqrs.Tests.csproj -c Release` y verifica que resuelve la API.

Archivo: `tests/AceriaApp.Cqrs.Tests/AceriaApp.Cqrs.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. el archivo `AceriaApp.Cqrs.Tests.csproj` contiene un `ProjectReference` al proyecto API.

Diagnóstico si falla.

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

PASO 5.4.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Ejecuta `dotnet add tests/AceriaApp.Cqrs.Tests/AceriaApp.Cqrs.Tests.csproj package FluentAssertions --version 6.12.2` o escribe el XML completo.
2. Ejecuta `dotnet list tests/AceriaApp.Cqrs.Tests/AceriaApp.Cqrs.Tests.csproj package` y localiza `FluentAssertions 6.12.2`.
3. Conserva la referencia a la API y las versiones fijadas de xUnit.
4. Ejecuta `dotnet build tests/AceriaApp.Cqrs.Tests/AceriaApp.Cqrs.Tests.csproj -c Release`; las aserciones `Should()` estarán disponibles.

Archivo: `tests/AceriaApp.Cqrs.Tests/AceriaApp.Cqrs.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. El archivo y `dotnet list package` muestran FluentAssertions 6.12.2; la compilación termina correctamente.

Diagnóstico si falla.

- Instalar una versión preliminar: el curso usa la versión fijada en el archivo de proyecto.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

PASO 5.4.14 — Crear el archivo `ValidadorCqrsTests.cs`

Los tests verifican que el validador acepta el catálogo por defecto y rechaza los mensajes que violan las reglas de CQRS.

1. En el panel Solution Explorer, expandir `tests` → `AceriaApp.Cqrs.Tests`.
2. Crear el archivo `ValidadorCqrsTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.Cqrs.Tests/ValidadorCqrsTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.Cqrs;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Cqrs.Tests;

public class ValidadorCqrsTests
{
```

```
    private readonly ValidadorCqrs _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoCqrs.MensajesPorDefecto());

        resultado.EsValido.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinMensajes_DevuelveError()
    {
        var resultado = _validador.Validar(new List<MensajeCqrs>());

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay mensajes"));
    }

    [Fact]
    public void Validar_TipoNoPermitido_DevuelveError()
    {
        var mensajes = new List<MensajeCqrs>
        {
            new() { Nombre = "X", Tipo = "TipoInventado", TieneManejador = true, DevuelveResult = true }
        };

        var resultado = _validador.Validar(mensajes);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Tipo no permitido"));
    }

    [Fact]
    public void Validar_SinManejador_DevuelveError()
    {
        var mensajes = new List<MensajeCqrs>
        {
            new() { Nombre = "X", Tipo = "Command", TieneManejador = false, UsaEscritura = true,
DevuelveResult = true }
        };

        var resultado = _validador.Validar(mensajes);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene manejador"));
    }

    [Fact]
    public void Validar_CommandSinEscritura_DevuelveError()
    {
        var mensajes = new List<MensajeCqrs>
        {
            new() { Nombre = "X", Tipo = "Command", TieneManejador = true, UsaEscritura = false,
DevuelveResult = true }
        };

        var resultado = _validador.Validar(mensajes);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no usa escritura"));
    }

    [Fact]
    public void Validar_QuerySinSoloLectura_DevuelveError()
    {
        var mensajes = new List<MensajeCqrs>
        {
            new() { Nombre = "X", Tipo = "Query", TieneManejador = true, UsaSoloLectura = false,
DevuelveResult = true }
        };

        var resultado = _validador.Validar(mensajes);

        resultado.EsValido.Should().BeFalse();
```

```
        resultado.Errores.Should().Contain(e => e.Contains("no es de solo lectura"));
    }

    [Fact]
    public void Validar_QueryConEscritura_DevuelveError()
    {
        var mensajes = new List<MensajeCqrs>
        {
            new() { Nombre = "X", Tipo = "Query", TieneManejador = true, UsaSoloLectura = true, UsaEscritura
= true, DevuelveResult = true }
        };

        var resultado = _validador.Validar(mensajes);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("usa escritura"));
    }
}
```

Verificación concreta. el archivo `ValidadorCqrsTests.cs` compila y contiene siete métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.Cqrs;`: las clases no se resuelven.
- Usar `Assert.True` en lugar de `Should().BeTrue()`: el curso usa FluentAssertions.
- No cubrir el caso de query con escritura: la regla específica no se verifica.

PASO 5.4.15 — Crear el archivo `CatalogoCqrsTests.cs`

Los tests verifican que el catálogo por defecto contiene los mensajes esperados y la separación correcta entre commands y queries.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.Cqrs.Tests`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoCqrsTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.Cqrs.Tests/CatalogoCqrsTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.Cqrs;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Cqrs.Tests;

public class CatalogoCqrsTests
{
    [Fact]
    public void MensajesPorDefecto_ContieneCuatroMensajes()
    {
        var mensajes = CatalogoCqrs.MensajesPorDefecto();

        mensajes.Should().HaveCount(4);
    }

    [Fact]
    public void MensajesPorDefecto_ContieneDosCommandsYDosQueries()
    {
        var mensajes = CatalogoCqrs.MensajesPorDefecto();

        mensajes.Count(m => m.Tipo == "Command").Should().Be(2);
        mensajes.Count(m => m.Tipo == "Query").Should().Be(2);
    }

    [Fact]
    public void MensajesPorDefecto_CommandsUsanEscritura()
    {
        var mensajes = CatalogoCqrs.MensajesPorDefecto();
```

```
        mensajes.Where(m => m.Tipo == "Command").Should().OnlyContain(m => m.UsaEscritura);
    }

    [Fact]
    public void MensajesPorDefecto_QueriesSonDeSoloLectura()
    {
        var mensajes = CatalogoCqrs.MensajesPorDefecto();

        mensajes.Where(m => m.Tipo == "Query").Should().OnlyContain(m => m.UsaSoloLectura);
    }

    [Fact]
    public void MensajesPorDefecto_TodosDevuelvenResult()
    {
        var mensajes = CatalogoCqrs.MensajesPorDefecto();

        mensajes.Should().OnlyContain(m => m.DevuelveResult);
    }
}
```

Verificación concreta. el archivo `CatalogoCqrsTests.cs` compila y contiene cinco métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.Cqrs;`: las clases no se resuelven.
- Olvidar el `using System.Linq;`: `Count`, `Where` y `OnlyContain` no se resuelven.
- No verificar la separación: el catálogo podría mezclar commands y queries.

PASO 5.4.16 — Separar comandos y consultas con proyecciones de lectura

La comprobación funcional debe seguir un comando que modifica la orden y una query que devuelve una proyección sin exponer el agregado. Verifica en SQL Server el cambio del comando y la ausencia de efectos de la lectura, sin asumir que CQRS exige bases físicas separadas.

1. Lee cada archivo de integración de este paso y localiza su responsabilidad en las capas del contexto Producción.
2. Crea o modifica los archivos exactamente como se indica; sigue la lectura numerada para conectar contratos, reglas y pruebas.
3. Conserva todos los proyectos de la solución y comprueba las referencias indicadas en cada archivo de proyecto.
4. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`; `dotnet sln AceriaApp.sln list` debe mostrar 21 proyectos.

Integración funcional. Expone el contrato HTTP de Producción con 201 y Location al crear, y errores 400, 404 o 409 según su significado.

Archivo: `src/AceriaApp.Api/Produccion/ProduccionEndpoints.cs`. **Operación:** Reemplazar el contenido completo.

```
using MediatR;
using AceriaApp.Produccion.Aplicacion;
namespace AceriaApp.Api.Produccion;
public sealed record CrearOrdenProduccionRequest(string Cliente, string Material, int Cantidad);
public static class ProduccionEndpoints
{
    public static IEndpointRouteBuilder MapProduccion(this IEndpointRouteBuilder app)
    {
        var grupo = app.MapGroup("/api/v1/produccion/ordenes").WithTags("Produccion");
        grupo.MapPost("", async (CrearOrdenProduccionRequest request, ISender sender, CancellationToken ct)
=>
        {
            var resultado = await sender.Send(new CrearOrdenProduccionCommand(request.Cliente,
request.Material, request.Cantidad), ct);
            return resultado.EsExito
                ? Results.Created($"/api/v1/produccion/ordenes/{resultado.Valor}", new { id =
```

```
resultado.Valor })
                : ErrorHttp(resultado.Error);
        });
        grupo.MapGet("/{id:guid}", async (Guid id, ISender sender, CancellationToken ct) =>
            Responder(await sender.Send(new ObtenerOrdenProduccionQuery(id), ct)));
        grupo.MapPost("/{id:guid}/planificar", async (Guid id, ISender sender, CancellationToken ct) =>
            Responder(await sender.Send(new PlanificarOrdenProduccionCommand(id), ct)));
        grupo.MapPost("/{id:guid}/finalizar", async (Guid id, ISender sender, CancellationToken ct) =>
            Responder(await sender.Send(new FinalizarOrdenProduccionCommand(id), ct)));
        grupo.MapGet("", async (ISender sender, CancellationToken ct) =>
            Results.Ok(await sender.Send(new ListarOrdenesProduccionQuery(), ct)));
        return app;
    }
    private static IResult Responder<T>(ResultadoProduccion<T> resultado) =>
        resultado.EsExito ? Results.Ok(resultado.Valor) : ErrorHttp(resultado.Error);
    private static IResult ErrorHttp(ErrorProduccion error) => Results.Problem(
        title: error.Mensaje,
        statusCode: error.Tipo switch
        {
            TipoErrorProduccion.Validacion => 400,
            TipoErrorProduccion.NoEncontrado => 404,
            TipoErrorProduccion.Conflicto => 409,
            _ => throw new ArgumentOutOfRangeException(nameof(error))
        }, extensions: new Dictionary<string, object?> { ["codigo"] = error.Codigo });
}
```

Mantiene una ruta explícita del contexto nuevo, sin colisionar con las APIs de los módulos anteriores. Devuelve 201 y la URI que permite consultar el recurso recién creado. Traduce una transición incompatible con el estado actual a conflicto HTTP.

Integración funcional. Configura las referencias y versiones de este proyecto manteniendo los proyectos y catálogos heredados.

Archivo: `src/AceriaApp.Produccion.Aplicacion/AceriaApp.Produccion.Aplicacion.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings></PropertyGroup>
  <ItemGroup><ProjectReference Include="../AceriaApp.Produccion.Dominio/
AceriaApp.Produccion.Dominio.csproj" /><PackageReference Include="MediatR" Version="12.4.1" /></ItemGroup>
</Project>
```

Declara una dependencia de proyecto que el compilador necesita; no registra servicios en el contenedor.

Integración funcional. Define un puerto de consulta que devuelve DTOs y no expone operaciones de escritura ni unidad de trabajo.

Archivo: `src/AceriaApp.Produccion.Aplicacion/IOrdenProduccionLecturas.cs`. **Operación:** Crear.

```
namespace AceriaApp.Produccion.Aplicacion;
// Contrato de consulta: no permite modificar entidades ni confirmar cambios.
public interface IOrdenProduccionLecturas
{
    Task<OrdenProduccionDto?> ObtenerAsync(Guid id, CancellationToken ct = default);
    Task<IReadOnlyList<OrdenProduccionDto>> ListarAsync(CancellationToken ct = default);
}
```

Crea un puerto exclusivo de lectura; sus consumidores no pueden agregar entidades ni confirmar escrituras mediante él. Devuelve una proyección DTO o ausencia para el identificador solicitado. El contrato de lectura no expone la raíz del agregado. Devuelve una colección de respuestas ya proyectadas y propaga cancelación, sin necesitar una unidad de trabajo de escritura.

Integración funcional. Conecta cada command con los casos de uso y cada query con el puerto de lectura; el dominio conserva las invariantes.

Archivo: `src/AceriaApp.Produccion.Aplicacion/ManejadoresProduccion.cs`. **Operación:** Crear.

```
using MediatR;
namespace AceriaApp.Produccion.Aplicacion;
public sealed class CrearOrdenProduccionHandler(OrdenesProduccionCasosUso casos)
    : IRequestHandler<CrearOrdenProduccionCommand, ResultadoProduccion<Guid>>
{
    public Task<ResultadoProduccion<Guid>> Handle(CrearOrdenProduccionCommand request, CancellationToken ct)
=>
        casos.CrearAsync(request.Cliente, request.Material, request.Cantidad, ct);
}
public sealed class PlanificarOrdenProduccionHandler(OrdenesProduccionCasosUso casos)
    : IRequestHandler<PlanificarOrdenProduccionCommand, ResultadoProduccion<OrdenProduccionDto>>
{
    public Task<ResultadoProduccion<OrdenProduccionDto>> Handle(PlanificarOrdenProduccionCommand request,
CancellationToken ct) =>
        casos.PlanificarAsync(request.Id, ct);
}
public sealed class FinalizarOrdenProduccionHandler(OrdenesProduccionCasosUso casos)
    : IRequestHandler<FinalizarOrdenProduccionCommand, ResultadoProduccion<OrdenProduccionDto>>
{
    public Task<ResultadoProduccion<OrdenProduccionDto>> Handle(FinalizarOrdenProduccionCommand request,
CancellationToken ct) =>
        casos.FinalizarAsync(request.Id, ct);
}
public sealed class ObtenerOrdenProduccionHandler(IOrdenProduccionLecturas lecturas)
    : IRequestHandler<ObtenerOrdenProduccionQuery, ResultadoProduccion<OrdenProduccionDto>>
{
    public async Task<ResultadoProduccion<OrdenProduccionDto>> Handle(ObtenerOrdenProduccionQuery request,
CancellationToken ct)
    {
        var dto = await lecturas.ObtenerAsync(request.Id, ct);
        return dto is null
            ? ResultadoProduccion<OrdenProduccionDto>.Fallo(new("orden.ausente", "La orden de producción no
existe.", TipoErrorProduccion.NoEncontrado))
            : ResultadoProduccion<OrdenProduccionDto>.Exito(dto);
    }
}
public sealed class ListarOrdenesProduccionHandler(IOrdenProduccionLecturas lecturas)
    : IRequestHandler<ListarOrdenesProduccionQuery, IReadOnlyList<OrdenProduccionDto>>
{
    public Task<IReadOnlyList<OrdenProduccionDto>> Handle(ListarOrdenesProduccionQuery request,
CancellationToken ct) =>
        lecturas.ListarAsync(ct);
}
```

Delega la escritura en el caso de uso y transmite el mismo token de cancelación; el handler no duplica las invariantes del dominio. Pide al caso de uso cargar la raíz, ejecutar la transición y confirmar; no asigna el estado desde el manejador. Conserva la separación de intenciones y deja que el dominio rechace una transición inválida.

El manejador de consulta recibe exclusivamente el puerto de lectura, sin repositorio de escritura ni unidad de trabajo. `var dto = await lecturas.ObtenerAsync(request.Id, ct);` obtiene la proyección sin cargar una entidad con seguimiento para modificarla; la ausencia se resuelve a continuación. Representa la ausencia con el código estable orden.ausente y la categoría NoEncontrado para que la API la traduzca a 404.

La consulta de colección devuelve los DTO del puerto sin ejecutar CommitAsync; una colección vacía es una respuesta normal.

Integración funcional. Declara los commands de escritura y las queries de lectura como contratos MediatR con respuestas explícitas.

Archivo: `src/AceriaApp.Produccion.Aplicacion/MensajesProduccion.cs`. **Operación:** Crear.

```
using MediatR;
namespace AceriaApp.Produccion.Aplicacion;
public sealed record CrearOrdenProduccionCommand(string Cliente, string Material, int Cantidad)
    : IRequest<ResultadoProduccion<Guid>>;
```

```
public sealed record PlanificarOrdenProduccionCommand(Guid Id)
    : IRequest<ResultadoProduccion<OrdenProduccionDto>>;
public sealed record FinalizarOrdenProduccionCommand(Guid Id)
    : IRequest<ResultadoProduccion<OrdenProduccionDto>>;
public sealed record ObtenerOrdenProduccionQuery(Guid Id)
    : IRequest<ResultadoProduccion<OrdenProduccionDto>>;
public sealed record ListarOrdenesProduccionQuery : IRequest<IReadOnlyList<OrdenProduccionDto>>;
```

Transporta la intención de crear una orden y sus datos; la solicitud no ejecuta reglas por sí misma. Expresa una transición mediante identidad, sin permitir asignar directamente el estado de la entidad. Separa la finalización de la planificación para que cada intención tenga un contrato reconocible.

Solicita información sin campos de modificación. Su manejador consulta un puerto de lectura y traduce la ausencia. Solicita la colección y devuelve directamente `IReadOnlyList<OrdenProduccionDto>`. Esta variante demuestra que CQRS no exige envolver toda respuesta en Result.

Integración funcional. Proyecta columnas y líneas con AsNoTracking sobre la misma base relacional; no materializa agregados con seguimiento ni confirma cambios.

Archivo: `src/AceriaApp.Produccion.Infraestructura/OrdenProduccionLecturas.cs`. **Operación:** Crear.

```
using AceriaApp.Produccion.Aplicacion;
using AceriaApp.Produccion.Dominio;
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Produccion.Infraestructura;

// CQRS sobre la misma base: SQL selecciona columnas, la aplicación recibe DTOs.
public sealed class OrdenProduccionLecturas(ProduccionDbContext db) : IOrdenProduccionLecturas
{
    public async Task<OrdenProduccionDto?> ObtenerAsync(Guid id, CancellationToken ct = default) =>
        (await ProyectarAsync(db.Ordenes.Where(o => o.Id == id), ct)).SingleOrDefault();
    public Task<IReadOnlyList<OrdenProduccionDto>> ListarAsync(CancellationToken ct = default) =>
        ProyectarAsync(db.Ordenes.OrderBy(o => o.Cliente).ThenBy(o => o.Id), ct);

    private static async Task<IReadOnlyList<OrdenProduccionDto>> ProyectarAsync(
        IQueryable<OrdenProduccion> consulta, CancellationToken ct)
    {
        // Se proyectan valores mapeados; no se materializan raíces con tracking.
        var filas = await consulta.AsNoTracking().Select(o => new
        {
            o.Id, o.Cliente, o.Estado,
            Lineas = o.Lineas.Select(l => new { l.Id, l.Material, l.Cantidad }).ToList()
        }).ToListAsync(ct);
        return filas.Select(o => new OrdenProduccionDto(o.Id, o.Cliente, o.Estado.ToString(),
            o.Lineas.Sum(l => l.Cantidad.Valor),
            o.Lineas.Select(l => new LineaProduccionDto(l.Id, l.Material.Valor,
l.Cantidad.Valor)).ToList().AsReadOnly()))
            .ToList().AsReadOnly();
    }
}
```

Filtra el detalle por identidad en SQL antes de construir la respuesta. Proyecta los datos necesarios sin incorporar entidades al ChangeTracker. Incluye los valores escalares convertidos de las líneas sin pasar la raíz del agregado a la API.

Calcula el total del DTO con los valores consultados; la query no cambia el dominio ni ejecuta un commit.

Integración funcional. Compone los adaptadores scoped de Producción sobre SQL Server; desde 5.4 registra el ensamblado propio de MediatR y el puerto de lecturas.

Archivo: `src/AceriaApp.Produccion.Infraestructura/RegistroProduccion.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Produccion.Aplicacion;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
```

```
using Microsoft.Extensions.DependencyInjection;
namespace AceriaApp.Produccion.Infraestructura;
public static class RegistroProduccion
{
    public static IServiceCollection AddProduccion(this IServiceCollection services, IConfiguration
configuration)
    {
        services.AddDbContext<ProduccionDbContext>(o =>
o.UseSqlServer(configuration.GetConnectionString("AceriaProduccion")));
        services.AddScoped<IOrdenProduccionRepositorio, OrdenProduccionRepositorio>();
        services.AddScoped<IProduccionUnitOfWork, ProduccionUnitOfWork>();
        services.AddScoped<OrdenesProduccionCasosUso>();
        services.AddScoped<IOrdenProduccionLecturas, OrdenProduccionLecturas>();
        services.AddMediatR(cfg =>
cfg.RegisterServicesFromAssembly(typeof(CrearOrdenProduccionCommand).Assembly));
        return services;
    }
}
```

Configura SQL Server; las pruebas usan el mismo proveedor con una conexión a una base aislada. Usa una instancia por petición que comparte el DbContext con la unidad de trabajo. Descubre los handlers del ensamblado propio de Producción sin incorporarlos al escaneo de M4.

Integración funcional. Configura las referencias y versiones de este proyecto manteniendo los proyectos y catálogos heredados.

Archivo: `tests/AceriaApp.Cqrs.Tests/AceriaApp.Cqrs.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsPackable>false</IsPackable><IsTestProject>true</IsTestProject></PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <ProjectReference Include="../../src/AceriaApp.Produccion.Dominio/AceriaApp.Produccion.Dominio.csproj" />
    <PackageReference Include="Testcontainers.MsSql" Version="3.10.0" />
    <PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="8.0.21" />
    <Compile Include="../Compartido/ProduccionApiFactory.cs" Link="ProduccionApiFactory.cs" />
    <Compile Include="../Compartido/SqlServerPruebas.cs" Link="SqlServerPruebas.cs" />
  </ItemGroup>
</Project>
```

Compila para .NET 8, igual que los proyectos heredados. Fija el cliente de contenedores para SQL Server; el proveedor EF Core SQL Server llega mediante la referencia a la API. Permite ejecutar peticiones HTTP contra el host real de la API.

Compila el helper heredado que crea y elimina únicamente la base aislada de cada prueba, sin añadir otro proyecto a la solución.

Integración funcional. Añade 6 pruebas funcionales del punto 5.4, además del catálogo original; verifica comportamiento real y regresiones heredadas.

Archivo: `tests/AceriaApp.Cqrs.Tests/CqrsFuncionalesTests.cs`. **Operación:** Crear.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Produccion.Dominio;
using AceriaApp.Produccion.Aplicacion;
using AceriaApp.Produccion.Infraestructura;
using AceriaApp.PruebasProduccion;
using MediatR;
using AceriaApp.PruebasSqlServer;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Diagnostics;
```

```
using Microsoft.Extensions.DependencyInjection;
using Xunit;
namespace AceriaApp.Cqrs.Tests;
public sealed class CqrsFuncionalesTests
{
    [Fact]
    public async Task Mediador_resuelve_el_command_y_confirma_una_orden_real()
    {
        using var factory = new ProduccionApiFactory();
        using var scope = factory.Services.CreateScope();
        var sender = scope.ServiceProvider.GetRequiredService<ISender>();
        var resultado = await sender.Send(new CrearOrdenProduccionCommand(" Axia ", " Acero ", 40));
        Assert.True(resultado.EsExito);
        await using var db = factory.NuevoContexto();
        var orden = await db.Ordenes.Include(o => o.Lineas).SingleAsync();
        Assert.Equal(resultado.Valor, orden.Id);
        Assert.Equal("Axia", orden.Cliente);
        Assert.Equal(40, orden.CantidadTotal);
    }
    [Fact]
    public async Task Queries_proyectan_sin_tracking_ni_commit_y_los_DTO_no_mutan_el_dominio()
    {
        await using var store = BaseSqlServer.Crear();
        var opciones = new
DbContextOptionsBuilder<ProduccionDbContext>().UseSqlServer(store.ConnectionString).Options;
        Guid id;
        await using (var escritura = new ProduccionDbContext(opciones))
        {
            await escritura.Database.EnsureCreatedAsync();
            var orden = new OrdenProduccion("Axia", "Acero", 200);
            orden.AgregarLinea("Inoxidable", 100);
            id = orden.Id;
            escritura.Ordenes.Add(orden);
            await escritura.SaveChangesAsync();
        }
        var soloLectura = new
DbContextOptionsBuilder<ProduccionDbContext>().UseSqlServer(store.ConnectionString)
            .AddInterceptors(new ProhibirGuardado()).Options;
        await using var db = new ProduccionDbContext(soloLectura);
        Assert.Equal("Microsoft.EntityFrameworkCore.SqlServer", db.Database.ProviderName);
        var puerto = new OrdenProduccionLecturas(db);
        var detalle = await new ObtenerOrdenProduccionHandler(puerto).Handle(new(id), default);
        var lista = await new ListarOrdenesProduccionHandler(puerto).Handle(new(), default);
        Assert.Equal(300, detalle.Valor.CantidadTotal);
        Assert.Equal(2, detalle.Valor.Lineas.Count);
        Assert.Equal(id, Assert.Single(lista).Id);
        Assert.Empty(db.ChangeTracker.Entries());
        var copia = detalle.Valor with { Cliente = "Modificado fuera del dominio" };
        Assert.NotEqual(copia.Cliente, (await puerto.ObtenerAsync(id))!.Cliente);
        await using var comprobacion = new ProduccionDbContext(opciones);
        Assert.Equal(EstadoOrdenProduccion.Borrador, (await comprobacion.Ordenes.SingleAsync()).Estado);
    }
    [Fact]
    public async Task Query_inexistente_devuelve_error_tipado_y_la_lista_vacia_es_valida()
    {
        using var factory = new ProduccionApiFactory();
        using var scope = factory.Services.CreateScope();
        var sender = scope.ServiceProvider.GetRequiredService<ISender>();
        var resultado = await sender.Send(new ObtenerOrdenProduccionQuery(Guid.NewGuid()));
        Assert.False(resultado.EsExito);
        Assert.Equal(TipoErrorProduccion.NoEncontrado, resultado.Error.Tipo);
        Assert.Empty(await sender.Send(new ListarOrdenesProduccionQuery()));
    }
    [Fact]
    public async Task Command_invalido_no_elude_las_invariantes_del_agregado()
    {
        using var factory = new ProduccionApiFactory();
        using var scope = factory.Services.CreateScope();
        var sender = scope.ServiceProvider.GetRequiredService<ISender>();
        var resultado = await sender.Send(new CrearOrdenProduccionCommand("Axia", "Acero", 501));
        Assert.Equal(TipoErrorProduccion.Validacion, resultado.Error.Tipo);
        await using var db = factory.NuevoContexto();
        Assert.Equal(0, await db.Ordenes.CountAsync());
    }
    [Fact]
    public async Task Commands_de_estado_usados_por_el_mediador_respetan_la_secuencia()
```

```
    {
        using var factory = new ProduccionApiFactory();
        using var scope = factory.Services.CreateScope();
        var sender = scope.ServiceProvider.GetRequiredService<ISender>();
        var id = (await sender.Send(new CrearOrdenProduccionCommand("Axia", "Acero", 20))).Valor;
        Assert.Equal(TipoErrorProduccion.Conflicto, (await sender.Send(new
FinalizarOrdenProduccionCommand(id))).Error.Tipo);
        Assert.True((await sender.Send(new PlanificarOrdenProduccionCommand(id))).EsExito);
        Assert.True((await sender.Send(new FinalizarOrdenProduccionCommand(id))).EsExito);
        Assert.Equal(TipoErrorProduccion.Conflicto, (await sender.Send(new
FinalizarOrdenProduccionCommand(id))).Error.Tipo);
        await using var db = factory.NuevoContexto();
        Assert.Equal(EstadoOrdenProduccion.Finalizada, (await db.Ordenes.SingleAsync()).Estado);
    }
    [Fact]
    public async Task Http_lista_proyecciones_de_las_ordenes_creadas()
    {
        using var factory = new ProduccionApiFactory(); using var client = factory.CreateClient();
        await client.PostAsJsonAsync("/api/v1/produccion/ordenes", new { Cliente = "Axia", Material =
"Acero", Cantidad = 25 });
        var respuesta = await client.GetAsync("/api/v1/produccion/ordenes");
        Assert.Equal(HttpStatusCode.OK, respuesta.StatusCode);
        var lista = await respuesta.Content.ReadFromJsonAsync<OrdenProduccionDto[]>();
        Assert.Equal(25, Assert.Single(lista!).CantidadTotal);
        Assert.Equal("Acero", Assert.Single(lista![0].Lineas).Material);
    }
    private sealed class ProhibirGuardado : SaveChangesInterceptor
    {
        public override InterceptionResult<int> SavingChanges(DbContextEventData data,
InterceptionResult<int> result) =>
            throw new InvalidOperationException("Una query no debe guardar cambios.");
        public override ValueTask<InterceptionResult<int>> SavingChangesAsync(DbContextEventData data,
            InterceptionResult<int> result, CancellationToken ct = default) =>
            throw new InvalidOperationException("Una query no debe guardar cambios.");
    }
}
```

El command confirma una orden real; un contexto independiente recupera el agregado. Aísla los datos de las queries en una base SQL Server exclusiva. Las queries fallan si intentan llamar a SaveChanges.

Comprueba que la proyección se ejecuta con SQL Server. Exige lecturas sin seguimiento y verifica que alterar un DTO no muta el agregado. La ausencia se expresa mediante resultado tipado y la lista vacía es válida.

Comprueba planificación, finalización y rechazo de transiciones inválidas. La API devuelve la proyección con las líneas persistidas.

Verificación concreta. La solución contiene 21 proyectos, los archivos de integración coinciden con los bloques y la compilación completa termina con cero errores.

Diagnóstico si falla.

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.
- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

PASO 5.4.17 — Ejecutar todos los tests del punto 5.4

Las pruebas de catálogo verifican los datos declarados; las pruebas funcionales ejecutan las reglas y los adaptadores reales del contexto. Ambas evidencias son necesarias.

1. Ejecuta `dotnet test tests/AceriaApp.Cqrs.Tests/AceriaApp.Cqrs.Tests.csproj -c Release` desde la raíz de la etapa.
2. Comprueba 18 pruebas aprobadas: 12 de catálogo y 6 funcionales.

3. Exige cero fallos y cero omisiones; revisa los nombres de las pruebas para identificar el contrato comprobado.
4. Si falla una prueba, lee su aserción y corrige el archivo del paso correspondiente antes de continuar.

Verificación concreta. El proyecto ejecuta 18 tests: `Failed: 0, Passed: 18, Skipped: 0, Total: 18`.

Diagnóstico si falla.

- Ejecutar `dotnet test` sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

PASO 5.4.18 — Ejecutar todos los tests de la solución

Ejecutar todos los tests de la solución verifica que los tests de los puntos anteriores y del punto 5.4 pasan conjuntamente.

1. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build` después de la compilación Release.
2. Comprueba que se ejecutan los 14 proyectos de tests, incluidos los heredados de M04.
3. Suma los resultados por proyecto: deben ser 190 pruebas aprobadas, cero fallos y cero omisiones.
4. Conserva la salida como evidencia; si cambias código vuelve a compilar antes de utilizar `--no-build`.

Verificación concreta. Los 14 proyectos ejecutan conjuntamente 190 tests aprobados, cero fallos y cero omisiones.

Diagnóstico si falla.

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Omitir proyectos heredados de la solución: se pierde la comprobación de regresión.
- Eliminar o modificar pruebas anteriores sin una instrucción explícita: se pierde la cobertura heredada.

PASO 5.4.19 — Actualizar el README de la copia 5.4

El README del punto 5.4 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 5.3.

1. Abre el archivo `README.md` copiado de `5.3-casos-de-uso` dentro de `5.4-commands-queries`.
2. Reemplaza su contenido completo por el bloque de este paso; el README de la carpeta anterior permanece intacto.
3. Comprueba el objetivo, el estado, la ejecución, los añadidos, la documentación y la verificación ejecutable.
4. Guarda el archivo y abre su vista previa para comprobar títulos, enlaces, rutas y cifras.

Archivo: `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 5.4 — Separación entre Commands y Queries

## Objetivo
Comprender el patrón CQRS y añadir al proyecto las clases de validación de la separación entre comandos y
consultas.

Despachar comandos y consultas con MediatR y devolver proyecciones DTO sin seguimiento.

## Estado del proyecto
- Solución `AceriaApp.sln` con 21 proyectos.
- Documento `docs/commands-queries.md` con la definición y las reglas.
- Clases `MensajeCqrs`, `ValidadorCqrs` y `CatalogoCqrs` en `src/AceriaApp.Api/Cqrs/`.
- 12 tests de catálogo y 6 funcionales nuevos en `tests/AceriaApp.Cqrs.Tests`.
```

```
- Los tests acumulados suman 190 pruebas.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio con soporte de .NET 8.
2. Configurar SQL Server para los adaptadores heredados; las pruebas proporcionan su persistencia de prueba.
3. Configurar las conexiones `AceriaApp`, `AceriaHexagonal` y `AceriaProduccion` en el entorno local si se
ejecuta la API con SQL Server.
4. Ejecutar `dotnet run --project src/AceriaApp.Api/AceriaApp.Api.csproj --urls http://localhost:5000` desde
la raíz de la etapa.
5. Navegar a `http://localhost:5000/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

El perfil de lanzamiento heredado activa `Development`. En ese entorno, si `SkipDatabaseInitialization` es
falso, la API inicializa la base de Producción mediante `EnsureCreatedAsync`; no requiere datos iniciales.
Configura `ConnectionStrings:AceriaProduccion` para tu SQL Server antes del arranque.

Para probar el nuevo flujo, envía `POST /api/v1/produccion/ordenes` con
`{"cliente":"Axia","material":"Acero","cantidad":200}`, sigue su cabecera `Location` y envía los comandos
`POST /{id}/planificar` y `POST /{id}/finalizar` bajo la ruta de órdenes.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/commands-queries.md`.
- Carpeta `Cqrs` en el proyecto API con tres clases nuevas.
- Proyecto `tests/AceriaApp.Cqrs.Tests` con 12 tests de catálogo y 6 funcionales nuevos.
- Se conserva intacta la carpeta del punto 5.3.

- Despachar comandos y consultas con MediatR y devolver proyecciones DTO sin seguimiento. Los bloques
completos están en el paso 16 del manual.

## Documentación
- [ADR-010](docs/ADR-010-capas-aplicacion.md) a [ADR-021](docs/ADR-021-casos-uso.md) (heredados).
- [Separación entre Commands y Queries](docs/commands-queries.md).

## Verificación ejecutable

El catálogo describe ejemplos de diseño y sus indicadores; no inspecciona automáticamente las clases reales.
Los archivos funcionales del paso 16 y sus tests comprueban el comportamiento ejecutable. Ejecuta `dotnet
test AceriaApp.sln -c Release` y exige cero fallos y cero omisiones.
```

Verificación concreta. El archivo `README.md` existe; comprueba que sus títulos, contenido y enlaces coinciden con todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Modificar el README del punto 5.3: rompe la conservación de la etapa anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 5.3: rompe la separación.

PASO 5.4.20 — Crear el ADR-022 del punto 5.4

El ADR-022 documenta la decisión de adoptar la separación entre commands y queries. Es un artefacto nuevo del punto 5.4 que no modifica los ADR existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `ADR-022-commands-queries.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que los ADR-010 a ADR-021 no se han modificado.

Archivo: `docs/ADR-022-commands-queries.md`. **Operación:** Crear.

```
# ADR-022: Separación entre Commands y Queries

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado DDD y ha definido casos de uso. Es necesario separar las operaciones de
escritura de las de lectura.
```

```
## Decisión
Se incorporan mensajes y manejadores MediatR 12.4.1 en `AceriaApp.Produccion.Aplicacion`. Los comandos
delegan las escrituras en los casos de uso del contexto. Las consultas devuelven DTO mediante un puerto de
lectura y una proyección `AsNoTracking` de infraestructura, sin exponer el agregado ni confirmar cambios. La
API despacha mensajes con `ISender`. Lectura y escritura comparten la base de Producción; no existe
replicación asíncrona ni requisito de consistencia eventual. Las pruebas verifican el despacho, la
cancelación y la ausencia de escrituras o seguimiento en consultas. El catálogo mantiene sus cuatro mensajes
demostrativos y se valida de forma separada.

## Alternativas consideradas
- Modelo único sin separación: descartado por falta de claridad.
- CQRS con dos bases de datos: descartado por complejidad prematura.
- CQRS con proyecciones actualizadas por eventos: aplazado; el contexto usa lectura y escritura sobre la
misma base.

## Consecuencias
- Positivas: separación de responsabilidades, claridad, evolución independiente.
- Negativas: cada modelo requiere sus propios manejadores.
- Mitigación: la validación automática y la convención de nombres.

## Correspondencia ejecutable

Los nombres de `CatalogoCqrs` son ejemplos de análisis y no registran manejadores. `DevuelveResult` es una
convención del laboratorio, no un requisito universal de CQRS: una consulta puede devolver directamente un
DTO y un comando puede expresar el resultado con otro contrato. El flujo de Producción dispone de sus propios
mensajes de crear, planificar, finalizar, obtener y listar. Los comandos trabajan sobre el agregado mediante
los casos de uso; las consultas construyen una proyección DTO sin seguimiento de EF Core. La unidad de
trabajo no se usa para confirmar una consulta. Los registros MediatR del contexto conservan el flujo heredado
de M04.
```

Verificación concreta. El archivo `docs/ADR-022-commands-queries.md` existe; comprueba que sus títulos, contenido y enlaces coinciden con todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Modificar los ADR anteriores: rompe la conservación de la etapa anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-022`: rompe la convención.

PASO 5.4.21 — Comparar los documentos heredados con 5.3-casos-de-uso

La comparación comprueba los documentos heredados byte a byte. Las nuevas clases y los cambios explícitos de integración pertenecen a la copia actual; la carpeta del punto anterior permanece intacta.

1. En la terminal, escribir `cd C:\cursos\AceriaApp` y pulsar Enter.
2. Escribir `for %f in (M05_DDD_CQRS\PROYECTO\5.3-casos-de-uso\docs\*.md) do fc /L "%f" "M05_DDD_CQRS\PROYECTO\5.4-commands-queries\docs\%~nxf"` y pulsar Enter.
3. Verificar que la salida muestra `FC: no differences encountered` para cada archivo.
4. Verificar que el número de comparaciones coincide con el número de documentos del punto 5.3. Si algún archivo muestra diferencias, restaurarlo desde el punto 5.3.

Verificación concreta. Los 24 documentos heredados coinciden con la etapa anterior, sin omisiones ni modificaciones.

Diagnóstico si falla.

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.
- Omitir `/L` cuando se desea una comparación de texto: indica el modo explícitamente y revisa cada resultado.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

PASO 5.4.22 — Verificar la estructura del punto 5.4

La verificación estructural confirma que el punto 5.4 añade archivos nuevos conservando los documentos heredados.

1. Escribir `tree /F M05_DDD_CQRS\PROYECTO\5.4-commands-queries` y pulsar Enter.
2. Verificar que la salida muestra `AceriaApp.sln`, `README.md`, `src\AceriaApp.Api\Cqrs` con las tres clases nuevas, `tests\AceriaApp.Cqrs.Tests` con los dos archivos de tests de catálogo y las pruebas funcionales y `docs` con el documento de commands y queries y el ADR-022. Comprueba también los proyectos y archivos del contexto Producción indicados en el paso 16.
3. Verificar que las carpetas existentes del punto 5.3 permanecen intactas.
4. Verificar que no existe ninguna carpeta `labs` ni ningún archivo `metaprompt.md` ni `verificacion.md`.

Verificación concreta. la estructura contiene los archivos nuevos y conserva los del punto 5.3.

Diagnóstico si falla.

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 5.3: viola la regla de no modificar.
- Omitir la carpeta `Cqrs`: los tests no encuentran las clases.

PASO 5.4.23 — Verificar el `.gitignore` del punto 5.4

El `.gitignore` se hereda del punto 5.3 sin modificaciones. La verificación confirma que no se ha tocado.

1. Verificar que el archivo `.gitignore` existe en la carpeta del punto 5.4 (copiado del punto 5.3).
2. Abrir el archivo `.gitignore` con Visual Studio con soporte de .NET 8.
3. Verificar que contiene las líneas `**/bin/`, `**/obj/`, `.vs/`, `*.user`, `TestResults/` y `*.trx`.
4. Verificar que no se ha modificado respecto al punto 5.3. Pulsar `Ctrl+S` si no hay cambios y cerrar el archivo.

Verificación concreta. el `.gitignore` existe y contiene las seis líneas sin modificaciones.

Diagnóstico si falla.

- Modificar el `.gitignore` copiado del punto 5.3: rompe la conservación de la etapa anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el `.gitignore` con otro nombre: el archivo no se reconoce.

PASO 5.4.24 — Hacer el commit del punto 5.4 y verificar el estado final

El commit registra la etapa completa. Si existe uno por punto desde 1.1, el recorrido suma 22 commits de puntos; otros commits legítimos pueden aumentar el historial.

1. Ejecuta `git diff --stat` y revisa los archivos y entregables de esta etapa.
2. Ejecuta `git add .` después de comprobar que no se incluyen binarios, resultados temporales ni credenciales.
3. Ejecuta `git commit -m "Punto 5.4: Separación entre commands y queries"` con `user.name` y `user.email` configurados.
4. Comprueba `git log --oneline` y `git status`; deben registrar este punto y no dejar cambios pendientes de esta actividad.

Verificación concreta. El historial incluye el commit de 5.4 y `git status` no muestra cambios pendientes de esta actividad; no se impone una longitud rígida al historial.

Diagnóstico si falla.

- No tener configurado `user.name` y `user.email`: `git commit` falla.
- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

Reto de ampliación (optativo)

Trabaja en una copia independiente de la etapa obligatoria, por ejemplo `5.4-commands-queries-reto`. Conserva intacta la carpeta obligatoria y sus documentos. Los tres fragmentos añaden un contrato, un cálculo y una prueba al proyecto del punto.

Reto: añadir un método `Resumen` a `ValidadorCqrs` que devuelva el número total de mensajes, el número de commands, el número de queries y el número de mensajes que devuelven Result, y un test que verifique el cálculo con el catálogo por defecto.

Paso 1: En `ValidadorCqrs.cs`, añadir el siguiente `record` al final del archivo:

Archivo: `src/AceriaApp.Api/Cqrs/ValidadorCqrs.cs`. **Operación:** Modificar.

Punto de inserción: después de este texto exacto, que debe aparecer una sola vez:

```
public record ResultadoCqrs(bool EsValido, IReadOnlyList<string> Errores);
```

```
public record ResumenCqrs(
    int Total,
    int Commands,
    int Queries,
    int DevuelvenResult);
```

Paso 2: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public record ResumenCqrs(` declara un contrato inmutable. Número total de mensajes. Número de commands. Número de queries. Número de mensajes que devuelven Result.

Paso 3: En `ValidadorCqrs.cs`, añadir el siguiente método estático dentro de la clase:

Archivo: `src/AceriaApp.Api/Cqrs/ValidadorCqrs.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}

public record ResultadoCqrs(bool EsValido, IReadOnlyList<string> Errores);
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase indicado por el texto de inserción.

```
public static ResumenCqrs Resumen(IEnumerable<MensajeCqrs> mensajes)
{
    var lista = mensajes.ToList();
    var commands = lista.Count(m => m.Tipo == "Command");
    var queries = lista.Count(m => m.Tipo == "Query");
    var devuelven = lista.Count(m => m.DevuelveResult);
    return new ResumenCqrs(lista.Count, commands, queries, devuelven);
}
```

Paso 4: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public static ResumenCqrs Resumen(IEnumerable<MensajeCqrs> mensajes)` declara el método. Materializa la lista. Cuenta los commands. Cuenta las queries. Cuenta los que devuelven Result. Devuelve el resumen.

Paso 5: Guardar el archivo con `Ctrl+S`.

Paso 6: En `ValidadorCqrsTests.cs`, añadir el siguiente test:

Archivo: `tests/AceriaApp.Cqrs.Tests/ValidadorCqrsTests.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase indicado por el texto de inserción.

```
[Fact]
public void Resumen_ConCatalogoPorDefecto_CalculaTotales()
{
    var resumen = ValidadorCqrs.Resumen(CatalogoCqrs.MensajesPorDefecto());

    resumen.Total.Should().Be(4);
    resumen.Commands.Should().Be(2);
    resumen.Queries.Should().Be(2);
    resumen.DevuelvenResult.Should().Be(4);
}
```

Paso 7: Explicar línea por línea:

Marca el método como test de xUnit. Declara el test. Invoca el método. Verifica el total. Verifica los commands. Verifica las queries. Verifica los que devuelven Result.

Paso 8: Ejecuta `dotnet test tests/AceriaApp.Cqrs.Tests/AceriaApp.Cqrs.Tests.csproj -c Release` en la copia optativa. Deben pasar 19 tests del proyecto, incluido el nuevo resumen. Ejecuta después `dotnet test AceriaApp.sln -c Release` y exige 191 pruebas aprobadas.

Paso 9: Comprueba que solo cambian `ValidadorCqrs.cs` y `ValidadorCqrsTests.cs` en la copia optativa; la etapa obligatoria `5.4-commands-queries` y el punto anterior permanecen idénticos.

Paso 10: Hacer commit con `git commit -m "Reto 5.4: método Resumen en ValidadorCqrs"`.

Verificación: `Passed: 19` en el proyecto del punto y 191 pruebas aprobadas en la solución de la copia, cero fallos y cero omisiones.

Resultado verificable

Al finalizar este punto, el alumno debe tener:

- La carpeta `M05_DDD_CQRS/PROYECTO/5.4-commands-queries/` con la solución `AceriaApp.sln` que contiene 21 proyectos.
- El documento `docs/commands-queries.md` con la definición y las reglas.
- El documento `docs/ADR-022-commands-queries.md` con la decisión sobre la separación.
- Las clases `MensajeCqrs`, `ValidadorCqrs` y `CatalogoCqrs` en `src/AceriaApp.Api/Cqrs/`.
- El proyecto `tests/AceriaApp.Cqrs.Tests` añade 12 tests de catálogo y 6 funcionales; el reto añade uno más.
- `dotnet test AceriaApp.sln -c Release` ejecuta 190 pruebas en la etapa obligatoria y 191 en una copia con su reto; ninguna falla ni se omite.

- Los ADR-010 a ADR-021 del punto 5.3 sin modificar.
- Un commit revisado para 5.4, con el historial previo conservado.

Continuidad del aprendizaje

El punto 5.4 ha definido la separación entre comandos y consultas como base de CQRS. El proyecto AceriaApp dispone ahora de un validador automático de mensajes CQRS que comprueba la presencia de manejador, el uso de Result, el uso de escritura en los commands y el uso de solo lectura en las queries, y 12 tests de catálogo más 6 funcionales nuevos. Esta base permite avanzar hacia el punto 5.5, donde se aplicará DDD al dominio de la acería y se cerrará el módulo 5 con el resumen del estado del proyecto.

Punto 5.5 — Aplicación al dominio de la acería

Completar el recorrido de Producción y comprobar sus límites

El último punto integra las piezas anteriores alrededor del ciclo completo de una orden: crearla, agregar sus datos válidos, planificarla, consultarla y finalizarla. Las decisiones del dominio permanecen dentro del agregado; los casos de uso coordinan las operaciones y el adaptador SQL conserva el estado, mientras ASP.NET Core representa resultados y fallos con códigos HTTP adecuados.

En 5.5 el alumno debe ser capaz de reconstruir el recorrido desde una petición hasta el estado persistido y su lectura posterior. El catálogo de elementos de la acería se mantiene como instrumento de análisis, no como sustituto del modelo ejecutable. Las pruebas de cada capa y las de integración deben respaldarse entre sí y dejar claro qué ocurre si una orden no existe, si se supera la capacidad o si se intenta una transición prohibida. Así cerramos DDD y CQRS sin atribuir al curso capacidades distribuidas que todavía no se han implementado.

Carpeta de trabajo: `M05_DDD_CQRS/PROYECTO/5.5-dominio-aceria`.

Objetivo: completar los veinticuatro pasos, conservar los documentos heredados y comprobar el modelo con código y tests ejecutables. La etapa suma 208 pruebas en 22 proyectos.

Ejercicio guiado. Añadir al punto 5.4 un documento de aplicación de DDD al dominio de la acería en `docs/`, una clase de validación del modelo del dominio de la acería en el proyecto API y un test unitario que verifique la validación. Cerrar el módulo 5 con el resumen del estado del proyecto. El punto 5.5 parte del contenido del punto 5.4 conservando la carpeta anterior y los documentos heredados; los cambios de integración se indican explícitamente en esta copia. La implementación del paso 16 añade comportamiento ejecutable del contexto Producción al ejercicio de catálogo.

PASO 5.5.1 — Crear la carpeta del punto 5.5 a partir del punto 5.4

La copia conserva una etapa autónoma. 5.4-commands-queries aporta 190 tests aprobados y 21 proyectos antes de incorporar este punto.

1. Abre la terminal integrada y localiza `M05_DDD_CQRS/PROYECTO/5.4-commands-queries`; esa es la etapa completa que vas a continuar.
2. Crea `M05_DDD_CQRS/PROYECTO/5.5-dominio-aceria` y copia en ella el contenido de `5.4-commands-queries`; en Windows puedes usar `xcopy` con `/E /I /H /Y`.
3. Excluye `bin`, `obj` y `TestResults`; abre únicamente la copia de `5.5-dominio-aceria` para conservar intacto el punto anterior.
4. Ejecuta `dir` dentro de `5.5-dominio-aceria` y verifica `AceriaApp.sln`, `README.md`, `src`, `tests`, `docs` y `global.json`.

Verificación concreta. La copia contiene la solución y los 21 proyectos anteriores; todavía no contiene los archivos de 5.5.

Diagnóstico si falla.

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.

PASO 5.5.2 — Abrir la solución copiada en Visual Studio con soporte de .NET 8

Abrir la solución copiada permite trabajar en el punto 5.5 sobre la base del punto 5.4. La compilación verifica que la copia es correcta.

1. Abre `AceriaApp.sln` dentro de `5.5-dominio-aceria` con Visual Studio; en Windows puedes ejecutar `start AceriaApp.sln`.
2. Ejecuta `dotnet sln AceriaApp.sln list` y comprueba los 21 proyectos heredados; todas sus rutas deben existir en esta etapa.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release` para comprobar el punto de partida.
4. Si hay DLL bloqueadas por una API en ejecución, detén esa aplicación y repite la compilación. Continúa cuando no haya errores.

Verificación concreta. La solución muestra 21 proyectos y `dotnet build AceriaApp.sln -c Release` termina con `Build succeeded.`.

Diagnóstico si falla.

- Abrir la solución del punto 5.4 en lugar de la del punto 5.5: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener la API del punto 5.4 en ejecución: el puerto queda ocupado y la compilación falla.

PASO 5.5.3 — Verificar que la carpeta `docs` contiene los documentos del punto 5.4

La nueva etapa preserva 26 documentos y añade su ficha temática y su ADR. Los cambios de integración se aplican únicamente en la copia actual.

1. Expande `docs` dentro de la copia de `5.5-dominio-aceria`.
2. Comprueba los 26 documentos heredados: ADR-010 a ADR-022 y fichas de los puntos anteriores.
3. Compara sus nombres y contenido con `docs` de `5.4-commands-queries`; no modifiques ni elimines documentos anteriores.
4. Confirma que todavía no existen `docs/aplicacion-dominio-aceria.md` ni `docs/ADR-023-aplicacion-dominio-aceria.md`.

Verificación concreta. Existen 26 documentos heredados idénticos y no existen todavía los dos de este punto.

Diagnóstico si falla.

- Modificar alguno de los ADR o documentos existentes: rompe la conservación de la etapa anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 5.4.
- Crear la carpeta `docs` si ya existe: `mkdir` falla.

PASO 5.5.4 — Crear el documento de aplicación al dominio de la acería en `docs`

El documento de aplicación al dominio de la acería es el artefacto de `docs` específico del punto 5.5. Incluye el cierre del módulo 5 con el resumen del estado del proyecto. No modifica los 26 documentos existentes del punto 5.4.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `aplicacion-dominio-aceria.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que el archivo aparece en la carpeta `docs` sin subrayados rojos.

Archivo: `docs/aplicacion-dominio-aceria.md`. **Operación:** Crear.

```
# Aplicación al dominio de la acería — Punto 5.5

## Dominio

El análisis del dominio distingue Producción, Aprovisionamiento, Clientes y Facturación. El catálogo de
elementos de este punto contiene diez ejemplos de los tres primeros; Facturación se menciona como posible
frontera, sin elementos implementados en este ejercicio.

## Contexto de producción en el análisis

- Entidades: OrdenProduccion, LineaProduccion, ProductoTerminado.
- Value objects: Cantidad, Peso, Temperatura.
- Invariantes: cantidad mínima, temperatura de fusión, secuencia.

## Contexto de aprovisionamiento en el análisis

- Entidades: OrdenCompra, Proveedor, Material.
- Value objects: Cantidad, Precio, FechaEntrega.
- Invariantes: cantidad mínima, plazo, condiciones de pago.

## Contexto de clientes en el análisis

- Entidades: Cliente, Contacto.
- Value objects: Direccion, Telefono, Correo.
- Invariantes: razón social, identificador fiscal, dirección completa.

## Agregados del catálogo

- OrdenProduccion: raíz OrdenProduccion con LineaProduccion.
- OrdenCompra: raíz OrdenCompra con LineaCompra.
- Cliente: raíz Cliente con Contacto.

## Lenguaje ubicuo

- Producción: orden de producción, colada, producto terminado.
- Aprovisionamiento: orden de compra, proveedor, materia prima.
- Clientes: cliente, contacto, dirección fiscal.

## Casos de uso del análisis

- Producción: crear orden, asignar material, iniciar colada, finalizar producto.
- Aprovisionamiento: crear orden de compra, recibir material, registrar proveedor.
- Clientes: registrar cliente, actualizar contacto, consultar historial.

## Commands y queries del análisis

- Commands producción: CrearOrdenProduccionCommand, AsignarMaterialCommand.
- Queries producción: ObtenerOrdenProduccionQuery, ListarOrdenesProduccionQuery.
- Commands aprovisionamiento: CrearOrdenCompraCommand, RecibirMaterialCommand.
- Queries aprovisionamiento: ObtenerOrdenCompraQuery, ListarProveedoresQuery.

## Cierre del módulo 5

El proyecto AceriaApp dispone de:

- Conceptos básicos de DDD: contextos delimitados, lenguaje ubicuo y subdominios.
- Contexto Producción ejecutable con entidades y value objects que mantienen invariantes.
```

```
- Casos de uso de Producción con `ResultadoProduccion<T>` y unidad de trabajo.
- Separación entre commands y queries.
- Catálogo de análisis de tres contextos y flujo ejecutable del contexto Producción.
- 208 pruebas acumuladas en 22 proyectos, de los cuales 15 son proyectos de tests.
- Veintitrés puntos completados (1.1 a 1.5, 2.1 a 2.4, 3.1 a 3.4, 4.1 a 4.5 y 5.1 a 5.5).

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Contexto implementado | Producción |
| Agregado implementado | OrdenProduccion con LineaDeOrden |
| Casos de uso implementados | Crear, obtener, planificar, finalizar y añadir líneas |
| Commands ejecutables | Crear, planificar, finalizar y agregar línea en Producción |
| Queries ejecutables | Obtener y listar DTO de Producción |

## Alcance implementado y evidencia

Las listas de entidades, agregados y mensajes de los apartados de análisis describen alternativas de
modelado. El código del paso 16 implementa el contexto Producción y no proporciona un caso de uso de recibir
material ni de registrar clientes. El catálogo usa el nombre LineaProduccion; la entidad hija ejecutable es
`LineaDeOrden`. El agregado permite añadir líneas conforme a su estado y capacidad. La API expone el flujo
bajo `/api/v1/produccion/ordenes`; las pruebas siguen las respuestas HTTP y comprueban la lectura del estado
persistido.
```

Verificación concreta. El archivo `docs/aplicacion-dominio-aceria.md` existe; comprueba que sus títulos, contenido y enlaces coinciden con todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: rompe la conservación de la etapa anterior.
- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

PASO 5.5.5 — Crear la carpeta `DominioAceria` en el proyecto API

La carpeta `DominioAceria` albergará las clases de validación del modelo del dominio de la acería. Se crea como carpeta nueva sin tocar las carpetas existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.Api`.
2. Seleccionar `Add` → `New Folder` y nombrar la carpeta `DominioAceria`.
3. Verificar que la carpeta `DominioAceria` aparece dentro de `AceriaApp.Api` sin afectar a las carpetas existentes.
4. Verificar que no se ha modificado ningún archivo existente del punto 5.4.

Verificación concreta. la carpeta `DominioAceria` existe dentro de `AceriaApp.Api` y las carpetas existentes permanecen intactas.

Diagnóstico si falla.

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: rompe la conservación de la etapa anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

PASO 5.5.6 — Crear la clase `ElementoAceria`

`ElementoAceria` representa una descripción del catálogo, no una orden persistida. Reconoce qué datos pertenecen al contexto Producción y qué nombres de otros contextos siguen siendo parte del mapa conceptual.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `DominioAceria`.
2. Seleccionar `Add` → `Class` y nombrar la clase `ElementoAceria.cs`.

3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/DominioAceria/ElementoAceria.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.DominioAceria;

public class ElementoAceria
{
    public string Nombre { get; init; } = string.Empty;
    public string Contexto { get; init; } = string.Empty;
    public string Tipo { get; init; } = string.Empty;
    public bool TieneInvariantes { get; init; }
    public bool EsRaizDeAgregado { get; init; }
}
```

Verificación concreta. la clase `ElementoAceria` compila sin errores y expone cinco propiedades.

Diagnóstico si falla.

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar las propiedades de tipo `string` con `string.Empty`: el compilador emite advertencias.
- Usar `set` en lugar de `init`: permite modificar la instancia después de crearla.

PASO 5.5.7 — Crear la clase `ValidadorDominioAceria`

`ValidadorDominioAceria` verifica las reglas explícitas de clasificación. El resultado del validador no sustituye los invariantes del agregado ni la prueba del comportamiento HTTP ante una transición prohibida.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `DominioAceria`.
2. Seleccionar `Add` → `Class` y nombrar la clase `ValidadorDominioAceria.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Alcance del validador. Comprueba los indicadores de los ejemplos del catálogo. No analiza por reflexión los tipos de producción ni garantiza por sí solo la arquitectura. Las reglas del modelo ejecutable y sus límites se comprueban en las pruebas funcionales del paso 16.

Archivo: `src/AceriaApp.Api/DominioAceria/ValidadorDominioAceria.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.DominioAceria;

public class ValidadorDominioAceria
{
    private static readonly string[] ContextosPermitidos = { "Produccion", "Aprovisionamiento", "Clientes",
"Facturacion" };
    private static readonly string[] TiposPermitidos = { "Entidad", "ValueObject", "Agregado" };

    public ResultadoDominioAceria Validar(IEnumerable<ElementoAceria> elementos)
    {
        var errores = new List<string>();
        var lista = elementos.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay elementos declarados.");
        }

        foreach (var elemento in lista)
        {
            if (string.IsNullOrWhiteSpace(elemento.Nombre))
            {
                errores.Add("Un elemento no declara el nombre.");
            }
```

```
            if (!ContextosPermitidos.Contains(elemento.Contexto))
            {
                errores.Add($"Contexto no permitido en '{elemento.Nombre}': '{elemento.Contexto}'.");
            }

            if (!TiposPermitidos.Contains(elemento.Tipo))
            {
                errores.Add($"Tipo no permitido en '{elemento.Nombre}': '{elemento.Tipo}'.");
            }

            if (!elemento.TieneInvariantes)
            {
                errores.Add($"El elemento '{elemento.Nombre}' no tiene invariantes.");
            }
        }

        return new ResultadoDominioAceria(errores.Count == 0, errores);
    }
}

public record ResultadoDominioAceria(bool EsValido, IReadOnlyList<string> Errores);
```

Verificación concreta. la clase `ValidadorDominioAceria` compila sin errores y expone un método y un `record`.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<string>` e `IReadOnlyList<string>` no se resuelven.
- Olvidar el `using System.Linq;`: `Contains` y `ToList` no se resuelven.
- No comprobar el caso de lista vacía: el validador devuelve válido sin elementos.

PASO 5.5.8 — Añadir los `using` necesarios en `ValidadorDominioAceria`

Los tipos `List<string>` e `IReadOnlyList<string>` pertenecen a `System.Collections.Generic`. Los métodos `Contains` y `ToList` pertenecen a `System.Linq`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. En el panel Solution Explorer, hacer doble clic en `ValidadorDominioAceria.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; el texto de inserción debe aparecer exactamente una vez.
3. Verificar que no hay subrayados rojos en `List<string>`, `Contains` ni `ToList`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/DominioAceria/ValidadorDominioAceria.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.DominioAceria;
```

```
using System.Collections.Generic;
using System.Linq;
```

Verificación concreta. `ValidadorDominioAceria.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir solo `System.Collections.Generic` y olvidar `System.Linq`: `Contains` y `ToList` no se resuelven.
- Mezclar directivas using con declaraciones de tipos: coloca las directivas antes del namespace para mantener el orden del archivo.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 5.5.9 — Crear la clase `CatalogoAceria`

`CatalogoAceria` reúne ejemplos de elementos del dominio para orientar el razonamiento. Las garantías efectivas se obtienen al ejecutar los casos de uso y comprobar sus efectos persistidos.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `DominioAceria`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoAceria.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/DominioAceria/CatalogoAceria.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.DominioAceria;

public class CatalogoAceria
{
    public static List<ElementoAceria> ElementosPorDefecto()
    {
        return new List<ElementoAceria>
        {
            new() { Nombre = "OrdenProduccion", Contexto = "Produccion", Tipo = "Agregado", TieneInvariantes
= true, EsRaizDeAgregado = true },
            new() { Nombre = "LineaProduccion", Contexto = "Produccion", Tipo = "Entidad", TieneInvariantes =
true, EsRaizDeAgregado = false },
            new() { Nombre = "ProductoTerminado", Contexto = "Produccion", Tipo = "Entidad", TieneInvariantes
= true, EsRaizDeAgregado = false },
            new() { Nombre = "Cantidad", Contexto = "Produccion", Tipo = "ValueObject", TieneInvariantes =
true, EsRaizDeAgregado = false },
            new() { Nombre = "OrdenCompra", Contexto = "Aprovisionamiento", Tipo = "Agregado",
TieneInvariantes = true, EsRaizDeAgregado = true },
            new() { Nombre = "Proveedor", Contexto = "Aprovisionamiento", Tipo = "Entidad", TieneInvariantes
= true, EsRaizDeAgregado = false },
            new() { Nombre = "Precio", Contexto = "Aprovisionamiento", Tipo = "ValueObject", TieneInvariantes
= true, EsRaizDeAgregado = false },
            new() { Nombre = "Cliente", Contexto = "Clientes", Tipo = "Agregado", TieneInvariantes = true,
EsRaizDeAgregado = true },
            new() { Nombre = "Contacto", Contexto = "Clientes", Tipo = "Entidad", TieneInvariantes = true,
EsRaizDeAgregado = false },
            new() { Nombre = "Direccion", Contexto = "Clientes", Tipo = "ValueObject", TieneInvariantes =
true, EsRaizDeAgregado = false }
        };
    }
}
```

Verificación concreta. la clase `CatalogoAceria` compila sin errores y expone un método estático.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<T>` no se resuelve.
- No inicializar las propiedades de los objetos: el compilador emite advertencias.
- Reutilizar una lista estática mutable: cada llamada del catálogo debe construir una lista independiente.

PASO 5.5.10 — Añadir el `using` necesario en `CatalogoAceria`

El tipo `List<T>` pertenece a `System.Collections.Generic`. En proyectos con `ImplicitUsings` desactivado, hay que añadir el using explícito.

1. En el panel Solution Explorer, hacer doble clic en `CatalogoAceria.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; el texto de inserción debe aparecer exactamente una vez.
3. Verificar que no hay subrayados rojos en `List<T>`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/DominioAceria/CatalogoAceria.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.DominioAceria;
```

```
using System.Collections.Generic;
```

Verificación concreta. `CatalogoAceria.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir `using System.Linq;` en lugar de `System.Collections.Generic`: no resuelve `List<T>`.
- Colocar el using después de una clase: las directivas deben preceder las declaraciones de tipos.
- Olvidar el using y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 5.5.11 — Crear el proyecto de tests del punto 5.5

El punto 5.5 añade un proyecto de tests específico para las clases del dominio de la acería. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

1. Crea `tests/AceriaApp.DominioAceria.Tests` y su archivo `AceriaApp.DominioAceria.Tests.csproj` con el XML completo. Si utilizas la plantilla xUnit de Visual Studio, ajusta el archivo generado.
2. Elimina `UnitTest1.cs` si lo creó la plantilla; las pruebas completas se añaden en los pasos 14 y 15.
3. Ejecuta `dotnet sln AceriaApp.sln add tests/AceriaApp.DominioAceria.Tests/AceriaApp.DominioAceria.Tests.csproj` incluso si la solución ya existe.
4. Ejecuta `dotnet sln AceriaApp.sln list` y `dotnet restore tests/AceriaApp.DominioAceria.Tests/AceriaApp.DominioAceria.Tests.csproj`.

Archivo: `tests/AceriaApp.DominioAceria.Tests/AceriaApp.DominioAceria.Tests.csproj`. **Operación:** Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. La solución muestra 22 proyectos, incluido `AceriaApp.DominioAceria.Tests`; el csproj fija .NET 8, SDK de tests 17.12.0, xUnit 2.9.2 y runner 2.8.2.

Diagnóstico si falla.

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 5.4: rompe la conservación de la etapa anterior.
- Usar `dotnet new mstest`: el curso usa xUnit.

PASO 5.5.12 — Añadir la referencia del proyecto de tests a la API

La referencia permite que el proyecto de tests acceda a las clases de `DominioAceria` del proyecto API.

1. Ejecuta `dotnet add tests/AceriaApp.DominioAceria.Tests/AceriaApp.DominioAceria.Tests.csproj reference src/AceriaApp.Api/AceriaApp.Api.csproj` o deja el archivo como el XML completo.
2. Comprueba que la referencia va del proyecto de tests a la API; la API no debe depender de los tests.
3. Abre `tests/AceriaApp.DominioAceria.Tests/AceriaApp.DominioAceria.Tests.csproj` y confirma la ruta `../../src/AceriaApp.Api/AceriaApp.Api.csproj`.
4. Ejecuta `dotnet build tests/AceriaApp.DominioAceria.Tests/AceriaApp.DominioAceria.Tests.csproj -c Release` y verifica que resuelve la API.

Archivo: `tests/AceriaApp.DominioAceria.Tests/AceriaApp.DominioAceria.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. el archivo `AceriaApp.DominioAceria.Tests.csproj` contiene un `ProjectReference` al proyecto API.

Diagnóstico si falla.

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

PASO 5.5.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Ejecuta `dotnet add tests/AceriaApp.DominioAceria.Tests/AceriaApp.DominioAceria.Tests.csproj package FluentAssertions --version 6.12.2` o escribe el XML completo.
2. Ejecuta `dotnet list tests/AceriaApp.DominioAceria.Tests/AceriaApp.DominioAceria.Tests.csproj package` y localiza `FluentAssertions 6.12.2`.
3. Conserva la referencia a la API y las versiones fijadas de xUnit.
4. Ejecuta `dotnet build tests/AceriaApp.DominioAceria.Tests/AceriaApp.DominioAceria.Tests.csproj -c Release`; las aserciones `Should()` estarán disponibles.

Archivo: `tests/AceriaApp.DominioAceria.Tests/AceriaApp.DominioAceria.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
```

```
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. El archivo y `dotnet list package` muestran FluentAssertions 6.12.2; la compilación termina correctamente.

Diagnóstico si falla.

- Instalar una versión preliminar: el curso usa la versión fijada en el archivo de proyecto.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

PASO 5.5.14 — Crear el archivo `ValidadorDominioAceriaTests.cs`

Los tests verifican que el validador acepta el catálogo por defecto y rechaza los elementos que violan las reglas del dominio de la acería.

1. En el panel Solution Explorer, expandir `tests` → `AceriaApp.DominioAceria.Tests`.
2. Crear el archivo `ValidadorDominioAceriaTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.DominioAceria.Tests/ValidadorDominioAceriaTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.DominioAceria;
using FluentAssertions;
using Xunit;

namespace AceriaApp.DominioAceria.Tests;

public class ValidadorDominioAceriaTests
{
    private readonly ValidadorDominioAceria _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoAceria.ElementosPorDefecto());

        resultado.EsValido.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinElementos_DevuelveError()
    {
        var resultado = _validador.Validar(new List<ElementoAceria>());

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay elementos"));
    }

    [Fact]
    public void Validar_ContextoNoPermitido_DevuelveError()
    {
        var elementos = new List<ElementoAceria>
        {
            new() { Nombre = "X", Contexto = "ContextoInventado", Tipo = "Entidad", TieneInvariantes = true }
        };
```

```
        var resultado = _validador.Validar(elementos);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Contexto no permitido"));
    }

    [Fact]
    public void Validar_TipoNoPermitido_DevuelveError()
    {
        var elementos = new List<ElementoAceria>
        {
            new() { Nombre = "X", Contexto = "Produccion", Tipo = "TipoInventado", TieneInvariantes = true }
        };

        var resultado = _validador.Validar(elementos);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Tipo no permitido"));
    }

    [Fact]
    public void Validar_SinInvariantes_DevuelveError()
    {
        var elementos = new List<ElementoAceria>
        {
            new() { Nombre = "X", Contexto = "Produccion", Tipo = "Entidad", TieneInvariantes = false }
        };

        var resultado = _validador.Validar(elementos);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene invariantes"));
    }
}
```

Verificación concreta. el archivo `ValidadorDominioAceriaTests.cs` compila y contiene cinco métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.DominioAceria;`: las clases no se resuelven.
- Usar `Assert.True` en lugar de `Should().BeTrue()`: el curso usa FluentAssertions.
- No cubrir el caso de tipo no permitido: la regla específica no se verifica.

PASO 5.5.15 — Crear el archivo `CatalogoAceriaTests.cs`

Los tests verifican que el catálogo por defecto contiene los elementos esperados y la distribución correcta por contexto y tipo.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.DominioAceria.Tests`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoAceriaTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.DominioAceria.Tests/CatalogoAceriaTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.DominioAceria;
using FluentAssertions;
using Xunit;

namespace AceriaApp.DominioAceria.Tests;

public class CatalogoAceriaTests
{
    [Fact]
    public void ElementosPorDefecto_ContieneDiezElementos()
    {
```

```
        var elementos = CatalogoAceria.ElementosPorDefecto();

        elementos.Should().HaveCount(10);
    }

    [Fact]
    public void ElementosPorDefecto_ContieneTresAgregados()
    {
        var elementos = CatalogoAceria.ElementosPorDefecto();

        elementos.Count(e => e.Tipo == "Agregado").Should().Be(3);
    }

    [Fact]
    public void ElementosPorDefecto_ContieneUnNucleo()
    {
        var elementos = CatalogoAceria.ElementosPorDefecto();

        elementos.Count(e => e.Contexto == "Produccion").Should().Be(4);
    }

    [Fact]
    public void ElementosPorDefecto_TodosTienenInvariantes()
    {
        var elementos = CatalogoAceria.ElementosPorDefecto();

        elementos.Should().OnlyContain(e => e.TieneInvariantes);
    }

    [Fact]
    public void ElementosPorDefecto_RaicesDeAgregadoSonAgregados()
    {
        var elementos = CatalogoAceria.ElementosPorDefecto();

        elementos.Where(e => e.EsRaizDeAgregado).Should().OnlyContain(e => e.Tipo == "Agregado");
    }
}
```

Verificación concreta. el archivo `CatalogoAceriaTests.cs` compila y contiene cinco métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.DominioAceria;`: las clases no se resuelven.
- Olvidar el `using System.Linq;`: `Count`, `Where` y `OnlyContain` no se resuelven.
- No verificar las raíces de agregado: la regla no se comprueba.

PASO 5.5.16 — Completar el flujo de producción y comprobar sus reglas

El cierre recorre la orden de Producción desde HTTP hasta SQL Server: alta, planificación, lectura y finalización, además de los errores esperados. Distingue el fallo de una regla conocida de un error técnico y evita confundir la proyección de una query con la entidad del dominio.

1. Lee cada archivo de integración de este paso y localiza su responsabilidad en las capas del contexto Producción.
2. Crea o modifica los archivos exactamente como se indica; sigue la lectura numerada para conectar contratos, reglas y pruebas.
3. Conserva todos los proyectos de la solución y comprueba las referencias indicadas en cada archivo de proyecto.
4. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`; `dotnet sln AceriaApp.sln list` debe mostrar 22 proyectos.

Integración funcional. Expone el contrato HTTP de Producción con 201 y Location al crear, y errores 400, 404 o 409 según su significado.

Archivo: `src/AceriaApp.Api/Produccion/ProduccionEndpoints.cs`. **Operación:** Reemplazar el contenido completo.

```
using MediatR;
using AceriaApp.Produccion.Aplicacion;
namespace AceriaApp.Api.Produccion;
public sealed record CrearOrdenProduccionRequest(string Cliente, string Material, int Cantidad);
public sealed record AgregarLineaProduccionRequest(string Material, int Cantidad);
public static class ProduccionEndpoints
{
    public static IEndpointRouteBuilder MapProduccion(this IEndpointRouteBuilder app)
    {
        var grupo = app.MapGroup("/api/v1/produccion/ordenes").WithTags("Produccion");
        grupo.MapPost("", async (CrearOrdenProduccionRequest request, ISender sender, CancellationToken ct)
=>
        {
            var resultado = await sender.Send(new CrearOrdenProduccionCommand(request.Cliente,
request.Material, request.Cantidad), ct);
            return resultado.EsExito
                ? Results.Created($"/api/v1/produccion/ordenes/{resultado.Valor}", new { id =
resultado.Valor })
                : ErrorHttp(resultado.Error);
        });
        grupo.MapGet("/{id:guid}", async (Guid id, ISender sender, CancellationToken ct) =>
            Responder(await sender.Send(new ObtenerOrdenProduccionQuery(id), ct)));
        grupo.MapPost("/{id:guid}/planificar", async (Guid id, ISender sender, CancellationToken ct) =>
            Responder(await sender.Send(new PlanificarOrdenProduccionCommand(id), ct)));
        grupo.MapPost("/{id:guid}/finalizar", async (Guid id, ISender sender, CancellationToken ct) =>
            Responder(await sender.Send(new FinalizarOrdenProduccionCommand(id), ct)));
        grupo.MapGet("", async (ISender sender, CancellationToken ct) =>
            Results.Ok(await sender.Send(new ListarOrdenesProduccionQuery(), ct)));
        grupo.MapPost("/{id:guid}/lineas", async (Guid id, AgregarLineaProduccionRequest request, ISender
sender, CancellationToken ct) =>
            Responder(await sender.Send(new AgregarLineaProduccionCommand(id, request.Material,
request.Cantidad), ct)));
        return app;
    }
    private static IResult Responder<T>(ResultadoProduccion<T> resultado) =>
        resultado.EsExito ? Results.Ok(resultado.Valor) : ErrorHttp(resultado.Error);
    private static IResult ErrorHttp(ErrorProduccion error) => Results.Problem(
        title: error.Mensaje,
        statusCode: error.Tipo switch
        {
            TipoErrorProduccion.Validacion => 400,
            TipoErrorProduccion.NoEncontrado => 404,
            TipoErrorProduccion.Conflicto => 409,
            _ => throw new ArgumentOutOfRangeException(nameof(error))
        }, extensions: new Dictionary<string, object?> { ["codigo"] = error.Codigo });
}
```

Mantiene una ruta explícita del contexto nuevo, sin colisionar con las APIs de los módulos anteriores. Devuelve 201 y la URI que permite consultar el recurso recién creado. Envía la escritura por el handler que termina invocando al agregado.

Traduce una transición incompatible con el estado actual a conflicto HTTP.

Integración funcional. Conecta cada command con los casos de uso y cada query con el puerto de lectura; el dominio conserva las invariantes.

Archivo: `src/AceriaApp.Produccion.Aplicacion/ManejadoresProduccion.cs`. **Operación:** Reemplazar el contenido completo.

```
using MediatR;
namespace AceriaApp.Produccion.Aplicacion;
public sealed class CrearOrdenProduccionHandler(OrdenesProduccionCasosUso casos)
    : IRequestHandler<CrearOrdenProduccionCommand, ResultadoProduccion<Guid>>
{
    public Task<ResultadoProduccion<Guid>> Handle(CrearOrdenProduccionCommand request, CancellationToken ct)
=>
        casos.CrearAsync(request.Cliente, request.Material, request.Cantidad, ct);
}
public sealed class PlanificarOrdenProduccionHandler(OrdenesProduccionCasosUso casos)
    : IRequestHandler<PlanificarOrdenProduccionCommand, ResultadoProduccion<OrdenProduccionDto>>
{
```

```
    public Task<ResultadoProduccion<OrdenProduccionDto>> Handle(PlanificarOrdenProduccionCommand request,
CancellationToken ct) =>
        casos.PlanificarAsync(request.Id, ct);
}
public sealed class FinalizarOrdenProduccionHandler(OrdenesProduccionCasosUso casos)
    : IRequestHandler<FinalizarOrdenProduccionCommand, ResultadoProduccion<OrdenProduccionDto>>
{
    public Task<ResultadoProduccion<OrdenProduccionDto>> Handle(FinalizarOrdenProduccionCommand request,
CancellationToken ct) =>
        casos.FinalizarAsync(request.Id, ct);
}
public sealed class ObtenerOrdenProduccionHandler(IOrdenProduccionLecturas lecturas)
    : IRequestHandler<ObtenerOrdenProduccionQuery, ResultadoProduccion<OrdenProduccionDto>>
{
    public async Task<ResultadoProduccion<OrdenProduccionDto>> Handle(ObtenerOrdenProduccionQuery request,
CancellationToken ct)
    {
        var dto = await lecturas.ObtenerAsync(request.Id, ct);
        return dto is null
            ? ResultadoProduccion<OrdenProduccionDto>.Fallo(new("orden.ausente", "La orden de producción no
existe.", TipoErrorProduccion.NoEncontrado))
            : ResultadoProduccion<OrdenProduccionDto>.Exito(dto);
    }
}
public sealed class ListarOrdenesProduccionHandler(IOrdenProduccionLecturas lecturas)
    : IRequestHandler<ListarOrdenesProduccionQuery, IReadOnlyList<OrdenProduccionDto>>
{
    public Task<IReadOnlyList<OrdenProduccionDto>> Handle(ListarOrdenesProduccionQuery request,
CancellationToken ct) =>
        lecturas.ListarAsync(ct);
}

public sealed class AgregarLineaProduccionHandler(OrdenesProduccionCasosUso casos)
    : IRequestHandler<AgregarLineaProduccionCommand, ResultadoProduccion<OrdenProduccionDto>>
{
    public Task<ResultadoProduccion<OrdenProduccionDto>> Handle(AgregarLineaProduccionCommand request,
CancellationToken ct) =>
        casos.AgregarLineaAsync(request.Id, request.Material, request.Cantidad, ct);
}
```

Delega la escritura en el caso de uso y transmite el mismo token de cancelación; el handler no duplica las invariantes del dominio. Pide al caso de uso cargar la raíz, ejecutar la transición y confirmar; no asigna el estado desde el manejador. Conserva la separación de intenciones y deja que el dominio rechace una transición inválida.

El manejador de consulta recibe exclusivamente el puerto de lectura, sin repositorio de escritura ni unidad de trabajo. `var dto = await lecturas.ObtenerAsync(request.Id, ct);` obtiene la proyección sin cargar una entidad con seguimiento para modificarla; la ausencia se resuelve a continuación. Representa la ausencia con el código estable orden.ausente y la categoría NoEncontrado para que la API la traduzca a 404.

La consulta de colección devuelve los DTO del puerto sin ejecutar CommitAsync; una colección vacía es una respuesta normal. Incorpora una nueva escritura reutilizando la coordinación del caso de uso. Las reglas de estado y capacidad siguen perteneciendo a la raíz.

Integración funcional. Declara los commands de escritura y las queries de lectura como contratos MediatR con respuestas explícitas.

Archivo: `src/AceriaApp.Produccion.Aplicacion/MensajesProduccion.cs`. **Operación:** Reemplazar el contenido completo.

```
using MediatR;
namespace AceriaApp.Produccion.Aplicacion;
public sealed record CrearOrdenProduccionCommand(string Cliente, string Material, int Cantidad)
    : IRequest<ResultadoProduccion<Guid>>;
public sealed record PlanificarOrdenProduccionCommand(Guid Id)
    : IRequest<ResultadoProduccion<OrdenProduccionDto>>;
public sealed record FinalizarOrdenProduccionCommand(Guid Id)
    : IRequest<ResultadoProduccion<OrdenProduccionDto>>;
```

```
public sealed record ObtenerOrdenProduccionQuery(Guid Id)
    : IRequest<ResultadoProduccion<OrdenProduccionDto>>;
public sealed record ListarOrdenesProduccionQuery : IRequest<IReadOnlyList<OrdenProduccionDto>>;

public sealed record AgregarLineaProduccionCommand(Guid Id, string Material, int Cantidad)
    : IRequest<ResultadoProduccion<OrdenProduccionDto>>;
```

Transporta la intención de crear una orden y sus datos; la solicitud no ejecuta reglas por sí misma. Expresa una transición mediante identidad, sin permitir asignar directamente el estado de la entidad. Separa la finalización de la planificación para que cada intención tenga un contrato reconocible.

Solicita información sin campos de modificación. Su manejador consulta un puerto de lectura y traduce la ausencia. Solicita la colección y devuelve directamente `IReadOnlyList<OrdenProduccionDto>`. Esta variante demuestra que CQRS no exige envolver toda respuesta en Result. Transporta el identificador de la raíz y los valores de la nueva línea. El agregado comprobará estado y capacidad antes de incorporarla.

Integración funcional. Coordina recuperación, operación del agregado y commit; convierte solo reglas de negocio esperadas en resultados tipados.

Archivo: `src/AceriaApp.Produccion.Aplicacion/OrdenesProduccionCasosUso.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Produccion.Dominio;
namespace AceriaApp.Produccion.Aplicacion;

// La aplicación coordina; las reglas de capacidad y de estado viven en el agregado.
public sealed class OrdenesProduccionCasosUso(
    IOrdenProduccionRepositorio repositorio, IProduccionUnitOfWork unidad)
{
    public async Task<ResultadoProduccion<Guid>> CrearAsync(string cliente, string material, int cantidad,
CancellationToken ct = default)
    {
        try
        {
            var orden = new OrdenProduccion(cliente, material, cantidad);
            await repositorio.AgregarAsync(orden, ct);
            await unidad.CommitAsync(ct);
            return ResultadoProduccion<Guid>.Exito(orden.Id);
        }
        catch (ReglaProduccionException ex) { return ResultadoProduccion<Guid>.Fallo(Traducir(ex)); }
    }
    public async Task<ResultadoProduccion<OrdenProduccionDto>> ObtenerAsync(Guid id, CancellationToken ct =
default)
    {
        var orden = await repositorio.ObtenerAsync(id, ct);
        return orden is null ? Ausente() :
ResultadoProduccion<OrdenProduccionDto>.Exito(OrdenProduccionDto.Desde(orden));
    }
    public Task<ResultadoProduccion<OrdenProduccionDto>> PlanificarAsync(Guid id, CancellationToken ct =
default) =>
        ModificarAsync(id, o => o.Planificar(), ct);
    public Task<ResultadoProduccion<OrdenProduccionDto>> FinalizarAsync(Guid id, CancellationToken ct =
default) =>
        ModificarAsync(id, o => o.Finalizar(), ct);

    public Task<ResultadoProduccion<OrdenProduccionDto>> AgregarLineaAsync(Guid id, string material, int
cantidad, CancellationToken ct = default) =>
        ModificarAsync(id, o => o.AgregarLinea(material, cantidad), ct);

    private async Task<ResultadoProduccion<OrdenProduccionDto>> ModificarAsync(Guid id,
Action<OrdenProduccion> cambio, CancellationToken ct)
    {
        var orden = await repositorio.ObtenerAsync(id, ct);
        if (orden is null) return Ausente();
        try
        {
            cambio(orden);
            await unidad.CommitAsync(ct);
            return ResultadoProduccion<OrdenProduccionDto>.Exito(OrdenProduccionDto.Desde(orden));
        }
        catch (ReglaProduccionException ex) { return
```

```
ResultadoProduccion<OrdenProduccionDto>.Fallo(Traducir(ex)); }
    }
    private static ResultadoProduccion<OrdenProduccionDto> Ausente() =>
        ResultadoProduccion<OrdenProduccionDto>.Fallo(new("orden.ausente", "La orden de producción no
existe.", TipoErrorProduccion.NoEncontrado));
    private static ErrorProduccion Traducir(ReglaProduccionException ex) => new(ex.Codigo, ex.Message,
        ex.Codigo == "estado.invalido" ? TipoErrorProduccion.Conflicto : TipoErrorProduccion.Validacion);
}
```

Delega las invariantes de creación en el dominio. Registra el agregado sin confirmar aún la escritura. Confirma los cambios de la unidad de trabajo del contexto Producción.

Invoca el comportamiento del agregado, sin asignar su estado desde fuera. La nueva escritura reutiliza la coordinación y deja la capacidad en la raíz. Distingue una orden que no existe de una petición con datos inválidos.

Convierte únicamente errores esperados de negocio; los fallos técnicos y la cancelación se propagan.

Integración funcional. Configura las referencias y versiones de este proyecto manteniendo los proyectos y catálogos heredados.

Archivo: `tests/AceriaApp.DominioAceria.Tests/AceriaApp.DominioAceria.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsPackable>false</IsPackable><IsTestProject>true</IsTestProject></PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <ProjectReference Include="../../src/AceriaApp.Produccion.Dominio/AceriaApp.Produccion.Dominio.csproj" />
    <PackageReference Include="Testcontainers.MsSql" Version="3.10.0" />
    <PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="8.0.21" />
    <Compile Include="../Compartido/ProduccionApiFactory.cs" Link="ProduccionApiFactory.cs" />
    <Compile Include="../Compartido/SqlServerPruebas.cs" Link="SqlServerPruebas.cs" />
  </ItemGroup>
</Project>
```

Compila para .NET 8, igual que los proyectos heredados. Fija el cliente de contenedores para SQL Server; el proveedor EF Core SQL Server llega mediante la referencia a la API. Permite ejecutar peticiones HTTP contra el host real de la API.

Compila el helper heredado que crea y elimina únicamente la base aislada de cada prueba, sin añadir otro proyecto a la solución.

Integración funcional. Añade 8 pruebas funcionales del punto 5.5, además del catálogo original; verifica comportamiento real y regresiones heredadas.

Archivo: `tests/AceriaApp.DominioAceria.Tests/DominioAceriaFuncionalesTests.cs`. **Operación:** Crear.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Produccion.Dominio;
using AceriaApp.Produccion.Aplicacion;
using AceriaApp.Produccion.Infraestructura;
using AceriaApp.PruebasProduccion;
using Microsoft.EntityFrameworkCore;
using Xunit;
namespace AceriaApp.DominioAceria.Tests;
public sealed class DominioAceriaFuncionalesTests
{
    private const string Ruta = "/api/v1/produccion/ordenes";
    private static async Task<string> Crear(HttpClient client, int cantidad = 200)
    {
```

```
        var response = await client.PostAsJsonAsync(Ruta, new { Cliente = "Axia", Material = "Acero",
Cantidad = cantidad });
        Assert.Equal(HttpStatusCode.Created, response.StatusCode);
        return response.Headers.Location!.ToString();
    }
    [Fact]
    public async Task Flujo_de_aceria_crea_lineas_planifica_finaliza_y_consulta_el_agregado()
    {
        using var factory = new ProduccionApiFactory(); using var client = factory.CreateClient();
        var ruta = await Crear(client);
        var agregado = await client.PostAsJsonAsync(ruta + "/lineas", new { Material = "Acero inoxidable",
Cantidad = 100 });
        Assert.Equal(HttpStatusCode.OK, agregado.StatusCode);
        var antes = (await agregado.Content.ReadFromJsonAsync<OrdenProduccionDto>())!;
        Assert.Equal(300, antes.CantidadTotal);
        Assert.Equal(2, antes.Lineas.Count);
        Assert.Equal(HttpStatusCode.OK, (await client.PostAsync(ruta + "/planificar", null)).StatusCode);
        Assert.Equal(HttpStatusCode.OK, (await client.PostAsync(ruta + "/finalizar", null)).StatusCode);
        var dto = (await client.GetFromJsonAsync<OrdenProduccionDto>(ruta))!;
        Assert.Equal("Finalizada", dto.Estado);
        Assert.Equal(300, dto.CantidadTotal);
        Assert.Equal(antes.Lineas.Select(x => x.Id).OrderBy(x => x), dto.Lineas.Select(x => x.Id).OrderBy(x
=> x));
        await using var db = factory.NuevoContexto();
        var persistida = await db.Ordenes.Include(o => o.Lineas).SingleAsync();
        Assert.Equal(EstadoOrdenProduccion.Finalizada, persistida.Estado);
        Assert.Equal(2, persistida.Lineas.Count);
        Assert.Equal(300, persistida.CantidadTotal);
    }
    [Fact]
    public async Task Total_excesivo_devuelve_400_y_un_intento_valido_posterior_sigue_funcionando()
    {
        using var factory = new ProduccionApiFactory(); using var client = factory.CreateClient();
        var ruta = await Crear(client, 300);
        var rechazo = await client.PostAsJsonAsync(ruta + "/lineas", new { Material = "Inoxidable", Cantidad
= 201 });
        Assert.Equal(HttpStatusCode.BadRequest, rechazo.StatusCode);
        Assert.Contains("capacidad.excedida", await rechazo.Content.ReadAsStringAsync());
        await using (var control = factory.NuevoContexto())
        {
            var orden = await control.Ordenes.Include(o => o.Lineas).SingleAsync();
            Assert.Single(orden.Lineas);
            Assert.Equal(300, orden.CantidadTotal);
        }
        Assert.Equal(HttpStatusCode.OK, (await client.PostAsJsonAsync(ruta + "/lineas", new { Material =
"Inoxidable", Cantidad = 200 })).StatusCode);
        Assert.Equal(500, (await client.GetFromJsonAsync<OrdenProduccionDto>(ruta))!.CantidadTotal);
    }
    [Fact]
    public async Task Lineas_invalidas_no_llegan_a_la_base_de_datos()
    {
        using var factory = new ProduccionApiFactory(); using var client = factory.CreateClient();
        var ruta = await Crear(client, 20);
        var casos = new[] { (" ", 1), (new string('M', 101), 1), ((string)null!, 1), ("Acero", 0), ("Acero",
501) };
        foreach (var (material, cantidad) in casos)
        {
            var respuesta = await client.PostAsJsonAsync(ruta + "/lineas", new { Material = material,
Cantidad = cantidad });
            Assert.Equal(HttpStatusCode.BadRequest, respuesta.StatusCode);
        }
        await using var db = factory.NuevoContexto();
        var orden = await db.Ordenes.Include(o => o.Lineas).SingleAsync();
        Assert.Single(orden.Lineas);
        Assert.Equal(20, orden.CantidadTotal);
    }
    [Fact]
    public async Task Planificar_cierra_la_edicion_y_finalizar_no_vuelve_a_abrirla()
    {
        using var factory = new ProduccionApiFactory(); using var client = factory.CreateClient();
        var ruta = await Crear(client, 20);
        Assert.Equal(HttpStatusCode.OK, (await client.PostAsync(ruta + "/planificar", null)).StatusCode);
        foreach (var finalizar in new[] { false, true })
        {
            if (finalizar) Assert.Equal(HttpStatusCode.OK, (await client.PostAsync(ruta + "/finalizar",
null)).StatusCode);
```

```
            var rechazo = await client.PostAsJsonAsync(ruta + "/lineas", new { Material = "Inoxidable",
Cantidad = 1 });
            Assert.Equal(HttpStatusCode.Conflict, rechazo.StatusCode);
            Assert.Contains("estado.invalido", await rechazo.Content.ReadAsStringAsync());
        }
        await using var db = factory.NuevoContexto();
        var orden = await db.Ordenes.Include(o => o.Lineas).SingleAsync();
        Assert.Equal(EstadoOrdenProduccion.Finalizada, orden.Estado);
        Assert.Single(orden.Lineas);
    }
    [Fact]
    public async Task Agregar_linea_a_orden_ausente_devuelve_404_sin_crear_otra_orden()
    {
        using var factory = new ProduccionApiFactory(); using var client = factory.CreateClient();
        var respuesta = await client.PostAsJsonAsync($"{Ruta}/{Guid.NewGuid()}/lineas", new { Material =
"Acero", Cantidad = 1 });
        Assert.Equal(HttpStatusCode.NotFound, respuesta.StatusCode);
        Assert.Contains("orden.ausente", await respuesta.Content.ReadAsStringAsync());
        await using var db = factory.NuevoContexto();
        Assert.Equal(0, await db.Ordenes.CountAsync());
    }
    [Fact]
    public async Task Cada_orden_es_un_limite_de_capacidad_independiente()
    {
        using var factory = new ProduccionApiFactory(); using var client = factory.CreateClient();
        var uno = await Crear(client, 300);
        var dos = await Crear(client, 300);
        foreach (var ruta in new[] { uno, dos })
            Assert.Equal(HttpStatusCode.OK, (await client.PostAsJsonAsync(ruta + "/lineas", new { Material =
"Inoxidable", Cantidad = 200 })).StatusCode);
        var lista = (await client.GetFromJsonAsync<OrdenProduccionDto[]>(Ruta))!;
        Assert.Equal(2, lista.Length);
        Assert.All(lista, o => Assert.Equal(500, o.CantidadTotal));
        await using var db = factory.NuevoContexto();
        Assert.Equal(2, await db.Ordenes.CountAsync());
    }
    [Fact]
    public async Task Bordes_validos_se_normalizan_y_conservan_en_el_contrato_http()
    {
        using var factory = new ProduccionApiFactory(); using var client = factory.CreateClient();
        var cliente = new string('C', 100);
        var material = new string('M', 100);
        var creada = await client.PostAsJsonAsync(Ruta, new { Cliente = " " + cliente + " ", Material = "
Acero ", Cantidad = 1 });
        Assert.Equal(HttpStatusCode.Created, creada.StatusCode);
        var linea = await client.PostAsJsonAsync(creada.Headers.Location + "/lineas", new { Material = " " +
material + " ", Cantidad = 499 });
        Assert.Equal(HttpStatusCode.OK, linea.StatusCode);
        var dto = (await client.GetFromJsonAsync<OrdenProduccionDto>(creada.Headers.Location))!;
        Assert.Equal(cliente, dto.Cliente);
        Assert.Contains(dto.Lineas, l => l.Material == material && l.Cantidad == 499);
        Assert.Equal(500, dto.CantidadTotal);
    }
    [Fact]
    public async Task Cancelacion_de_la_escritura_se_propaga_sin_tratarla_como_regla_de_negocio()
    {
        using var factory = new ProduccionApiFactory();
        Guid id;
        await using (var preparacion = factory.NuevoContexto())
        {
            var orden = new OrdenProduccion("Axia", "Acero", 20);
            id = orden.Id;
            preparacion.Ordenes.Add(orden);
            await preparacion.SaveChangesAsync();
        }
        await using var db = factory.NuevoContexto();
        var casos = new OrdenesProduccionCasosUso(new OrdenProduccionRepositorio(db), new
ProduccionUnitOfWork(db));
        using var cancelacion = new CancellationTokenSource();
        cancelacion.Cancel();
        await Assert.ThrowsAnyAsync<OperationCanceledException>(() => casos.AgregarLineaAsync(id,
"Inoxidable", 10, cancelacion.Token));
        await using var control = factory.NuevoContexto();
        Assert.Single((await control.Ordenes.Include(o => o.Lineas).SingleAsync()).Lineas);
    }
}
```

Comprueba el comportamiento observable: flujo de aceria crea lineas planifica finaliza y consulta el agregado. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos. Comprueba el comportamiento observable: total excesivo devuelve 400 y un intento valido posterior sigue funcionando. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos. Comprueba el comportamiento observable: lineas invalidas no llegan a la base de datos. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos.

Comprueba el comportamiento observable: planificar cierra la edicion y finalizar no vuelve a abrirla. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos. Comprueba el comportamiento observable: agregar linea a orden ausente devuelve 404 sin crear otra orden. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos. Comprueba el comportamiento observable: cada orden es un limite de capacidad independiente. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos.

Comprueba el comportamiento observable: bordes validos se normalizan y conservan en el contrato http. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos. Comprueba el comportamiento observable: cancelacion de la escritura se propaga sin tratarla como regla de negocio. Los fallos se detectan mediante aserciones sobre el modelo, las respuestas o los datos persistidos.

Verificación concreta. La solución contiene 22 proyectos, los archivos de integración coinciden con los bloques y la compilación completa termina con cero errores.

Diagnóstico si falla.

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.
- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

PASO 5.5.17 — Ejecutar todos los tests del punto 5.5

Las pruebas de catálogo verifican los datos declarados; las pruebas funcionales ejecutan las reglas y los adaptadores reales del contexto. Ambas evidencias son necesarias.

1. Ejecuta `dotnet test tests/AceriaApp.DominioAceria.Tests/AceriaApp.DominioAceria.Tests.csproj -c Release` desde la raíz de la etapa.
2. Comprueba 18 pruebas aprobadas: 10 de catálogo y 8 funcionales.
3. Exige cero fallos y cero omisiones; revisa los nombres de las pruebas para identificar el contrato comprobado.
4. Si falla una prueba, lee su aserción y corrige el archivo del paso correspondiente antes de continuar.

Verificación concreta. El proyecto ejecuta 18 tests: `Failed: 0, Passed: 18, Skipped: 0, Total: 18`.

Diagnóstico si falla.

- Ejecutar `dotnet test` sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

PASO 5.5.18 — Ejecutar todos los tests de la solución

Ejecutar todos los tests de la solución verifica que los tests de los puntos anteriores y del punto 5.5 pasan conjuntamente.

1. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build` después de la compilación Release.
2. Comprueba que se ejecutan los 15 proyectos de tests, incluidos los heredados de M04.
3. Suma los resultados por proyecto: deben ser 208 pruebas aprobadas, cero fallos y cero omisiones.
4. Conserva la salida como evidencia; si cambias código vuelve a compilar antes de utilizar `--no-build`.

Verificación concreta. Los 15 proyectos ejecutan conjuntamente 208 tests aprobados, cero fallos y cero omisiones.

Diagnóstico si falla.

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Omitir proyectos heredados de la solución: se pierde la comprobación de regresión.
- Eliminar o modificar pruebas anteriores sin una instrucción explícita: se pierde la cobertura heredada.

PASO 5.5.19 — Actualizar el README de la copia 5.5

El README del punto 5.5 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 5.4.

1. Abre el archivo `README.md` copiado de `5.4-commands-queries` dentro de `5.5-dominio-aceria`.
2. Reemplaza su contenido completo por el bloque de este paso; el README de la carpeta anterior permanece intacto.
3. Comprueba el objetivo, el estado, la ejecución, los añadidos, la documentación y la verificación ejecutable.
4. Guarda el archivo y abre su vista previa para comprobar títulos, enlaces, rutas y cifras.

Archivo: `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 5.5 — Aplicación al dominio de la acería

## Objetivo
Aplicar los conceptos de DDD y CQRS al dominio de la acería y añadir al proyecto las clases de validación del modelo del dominio. Cerrar el módulo 5 con el resumen del estado del proyecto.

Completar por HTTP el flujo de órdenes y la incorporación de líneas con sus invariantes.

## Estado del proyecto
- Solución `AceriaApp.sln` con 22 proyectos.
- Documento `docs/aplicacion-dominio-aceria.md` con los contextos, los agregados y el cierre del módulo 5.
- Clases `ElementoAceria`, `ValidadorDominioAceria` y `CatalogoAceria` en `src/AceriaApp.Api/DominioAceria/`.
- 10 tests de catálogo y 8 funcionales nuevos en `tests/AceriaApp.DominioAceria.Tests`.
- Los tests acumulados suman 208 pruebas.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio con soporte de .NET 8.
2. Configurar SQL Server para los adaptadores heredados; las pruebas proporcionan su persistencia de prueba.
3. Configurar las conexiones `AceriaApp`, `AceriaHexagonal` y `AceriaProduccion` en el entorno local si se ejecuta la API con SQL Server.
4. Ejecutar `dotnet run --project src/AceriaApp.Api/AceriaApp.Api.csproj --urls http://localhost:5000` desde la raíz de la etapa.
5. Navegar a `http://localhost:5000/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

El perfil de lanzamiento heredado activa `Development`. En ese entorno, si `SkipDatabaseInitialization` es falso, la API inicializa la base de Producción mediante `EnsureCreatedAsync`; no requiere datos iniciales. Configura `ConnectionStrings:AceriaProduccion` para tu SQL Server antes del arranque.

Para probar el nuevo flujo, envía `POST /api/v1/produccion/ordenes` con
```

```
`{"cliente":"Axia","material":"Acero","cantidad":200}`, sigue su cabecera `Location` y envía los comandos
`POST /{id}/planificar` y `POST /{id}/finalizar` bajo la ruta de órdenes.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/aplicacion-dominio-aceria.md`.
- Carpeta `DominioAceria` en el proyecto API con tres clases nuevas.
- Proyecto `tests/AceriaApp.DominioAceria.Tests` con 10 tests de catálogo y 8 funcionales nuevos.
- Cierre del módulo 5 con el resumen del estado del proyecto.
- Se conserva intacta la carpeta del punto 5.4.

- Completar por HTTP el flujo de órdenes y la incorporación de líneas con sus invariantes. Los bloques
completos están en el paso 16 del manual.

Mientras la orden siga en Borrador, `POST /api/v1/produccion/ordenes/{id}/lineas` con `{"material":"Acero
inoxidable","cantidad":100}` añade una línea antes de planificar.

## Documentación
- [ADR-010](docs/ADR-010-capas-aplicacion.md) a [ADR-022](docs/ADR-022-commands-queries.md) (heredados).
- [Aplicación al dominio de la acería](docs/aplicacion-dominio-aceria.md).

## Verificación ejecutable

El catálogo describe ejemplos de diseño y sus indicadores; no inspecciona automáticamente las clases reales.
Los archivos funcionales del paso 16 y sus tests comprueban el comportamiento ejecutable. Ejecuta `dotnet
test AceriaApp.sln -c Release` y exige cero fallos y cero omisiones.
```

Verificación concreta. El archivo `README.md` existe; comprueba que sus títulos, contenido y enlaces coinciden con todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Modificar el README del punto 5.4: rompe la conservación de la etapa anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 5.4: rompe la separación.

PASO 5.5.20 — Crear el ADR-023 del punto 5.5

El ADR-023 documenta la decisión de aplicar DDD y CQRS al dominio de la acería y de cerrar el módulo 5. Es un artefacto nuevo del punto 5.5 que no modifica los ADR existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `ADR-023-aplicacion-dominio-aceria.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que los ADR-010 a ADR-022 no se han modificado.

Archivo: `docs/ADR-023-aplicacion-dominio-aceria.md`. **Operación:** Crear.

```
# ADR-023: Aplicación de DDD y CQRS al dominio de la acería

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado DDD, entidades, value objects, casos de uso y CQRS. Es necesario aplicar
los conceptos al dominio concreto de la acería y cerrar el módulo 5.

## Decisión
Se cierra el módulo con un flujo ejecutable del contexto Producción. `OrdenProduccion` es la raíz y controla
las instancias de `LineaDeOrden`, los valores `NombreMaterial` y `CantidadProduccion`, la capacidad total de
500 y las transiciones Borrador, Planificada y Finalizada. Un comando permite agregar líneas mediante el
endpoint de la orden; la raíz rechaza cambios incompatibles con su estado. Los casos de uso coordinan
repositorio y unidad de trabajo. Las consultas devuelven proyecciones DTO sobre la misma base de Producción.
Las pruebas HTTP ejecutan el recorrido completo y verifican errores esperados, persistencia y aislamiento de
lectura. Los catálogos describen también Aprovisionamiento y Clientes; no se implementan sus agregados ni
servicios. Facturación e Identidad quedan como fronteras del análisis, fuera de este flujo.

## Alternativas consideradas
- Modelo único para todo el dominio: descartado por mezclar significados.
```

```
- Sin contexto núcleo: descartado por falta de foco.
- Sin CQRS en el dominio: descartado por falta de separación.

## Consecuencias
- Positivas: alineación con el negocio, modelo expresivo, separación clara.
- Negativas: la aplicación requiere análisis por contexto.
- Mitigación: la validación automática y la revisión del modelo.

## Alcance implementado y evidencia

Las listas de entidades, agregados y mensajes de los apartados de análisis describen alternativas de modelado. El código del paso 16 implementa el contexto Producción y no proporciona un caso de uso de recibir material ni de registrar clientes. El catálogo usa el nombre LineaProduccion; la entidad hija ejecutable es `LineaDeOrden`. El agregado permite añadir líneas conforme a su estado y capacidad. La API expone el flujo bajo `/api/v1/produccion/ordenes`; las pruebas siguen las respuestas HTTP y comprueban la lectura del estado persistido.
```

Verificación concreta. El archivo `docs/ADR-023-aplicacion-dominio-aceria.md` existe; comprueba que sus títulos, contenido y enlaces coinciden con todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Modificar los ADR anteriores: rompe la conservación de la etapa anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-023`: rompe la convención.

PASO 5.5.21 — Comparar los documentos heredados con 5.4-commands-queries

La comparación comprueba los documentos heredados byte a byte. Las nuevas clases y los cambios explícitos de integración pertenecen a la copia actual; la carpeta del punto anterior permanece intacta.

1. En la terminal, escribir `cd C:\cursos\AceriaApp` y pulsar Enter.
2. Escribir `for %f in (M05_DDD_CQRS\PROYECTO\5.4-commands-queries\docs\*.md) do fc /L "%f" "M05_DDD_CQRS\PROYECTO\5.5-dominio-aceria\docs\%~nxf"` y pulsar Enter.
3. Verificar que la salida muestra `FC: no differences encountered` para cada archivo.
4. Verificar que el número de comparaciones coincide con el número de documentos del punto 5.4. Si algún archivo muestra diferencias, restaurarlo desde el punto 5.4.

Verificación concreta. Los 26 documentos heredados coinciden con la etapa anterior, sin omisiones ni modificaciones.

Diagnóstico si falla.

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.
- Omitir `/L` cuando se desea una comparación de texto: indica el modo explícitamente y revisa cada resultado.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

PASO 5.5.22 — Verificar la estructura del punto 5.5

La verificación estructural confirma que el punto 5.5 añade archivos nuevos conservando los documentos heredados.

1. Escribir `tree /F M05_DDD_CQRS\PROYECTO\5.5-dominio-aceria` y pulsar Enter.
2. Verificar que la salida muestra `AceriaApp.sln`, `README.md`, `src\AceriaApp.Api\DominioAceria` con las tres clases nuevas, `tests\AceriaApp.DominioAceria.Tests` con los dos archivos de tests de catálogo y las pruebas funcionales y `docs` con el documento de aplicación y el ADR-023. Comprueba también los proyectos y archivos del contexto Producción indicados en el paso 16.
3. Verificar que las carpetas existentes del punto 5.4 permanecen intactas.

4. Verificar que no existe ninguna carpeta `labs` ni ningún archivo `metaprompt.md` ni `verificacion.md`.

Verificación concreta. la estructura contiene los archivos nuevos y conserva los del punto 5.4.

Diagnóstico si falla.

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 5.4: viola la regla de no modificar.
- Omitir la carpeta `DominioAceria`: los tests no encuentran las clases.

PASO 5.5.23 — Verificar el `.gitignore` del punto 5.5

El `.gitignore` se hereda del punto 5.4 sin modificaciones. La verificación confirma que no se ha tocado.

1. Verificar que el archivo `.gitignore` existe en la carpeta del punto 5.5 (copiado del punto 5.4).
2. Abrir el archivo `.gitignore` con Visual Studio con soporte de .NET 8.
3. Verificar que contiene las líneas `**/bin/`, `**/obj/`, `.vs/`, `*.user`, `TestResults/` y `*.trx`.
4. Verificar que no se ha modificado respecto al punto 5.4. Pulsar `Ctrl+S` si no hay cambios y cerrar el archivo.

Verificación concreta. el `.gitignore` existe y contiene las seis líneas sin modificaciones.

Diagnóstico si falla.

- Modificar el `.gitignore` copiado del punto 5.4: rompe la conservación de la etapa anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el `.gitignore` con otro nombre: el archivo no se reconoce.

PASO 5.5.24 — Hacer el commit del punto 5.5 y verificar el estado final del módulo 5

El commit registra la etapa completa. Si existe uno por punto desde 1.1, el recorrido suma 23 commits de puntos; otros commits legítimos pueden aumentar el historial.

1. Ejecuta `git diff --stat` y revisa los archivos y entregables de esta etapa.
2. Ejecuta `git add .` después de comprobar que no se incluyen binarios, resultados temporales ni credenciales.
3. Ejecuta `git commit -m "Punto 5.5: Aplicación al dominio de la acería"` con `user.name` y `user.email` configurados.
4. Comprueba `git log --oneline` y `git status`; deben registrar este punto y no dejar cambios pendientes de esta actividad.

Verificación concreta. El historial incluye el commit de 5.5 y `git status` no muestra cambios pendientes de esta actividad; no se impone una longitud rígida al historial.

Diagnóstico si falla.

- No tener configurado `user.name` y `user.email`: `git commit` falla.
- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

Reto de ampliación (optativo)

Trabaja en una copia independiente de la etapa obligatoria, por ejemplo `5.5-dominio-aceria-reto`. Conserva intacta la carpeta obligatoria y sus documentos. Los tres fragmentos añaden un contrato, un cálculo y una prueba al proyecto del punto.

Reto: añadir un método `Resumen` a `ValidadorDominioAceria` que devuelva el número total de elementos, el número de agregados, el número de entidades, el número de value objects y el número de raíces de agregado, y un test que verifique el cálculo con el catálogo por defecto.

Paso 1: En `ValidadorDominioAceria.cs`, añadir el siguiente `record` al final del archivo:

Archivo: `src/AceriaApp.Api/DominioAceria/ValidadorDominioAceria.cs`. **Operación:** Modificar.

Punto de inserción: después de este texto exacto, que debe aparecer una sola vez:

```
public record ResultadoDominioAceria(bool EsValido, IReadOnlyList<string> Errores);
```

```
public record ResumenDominioAceria(
    int Total,
    int Agregados,
    int Entidades,
    int ValueObjects,
    int RaicesDeAgregado);
```

Paso 2: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public record ResumenDominioAceria(` declara un contrato inmutable. Número total de elementos. Número de agregados. Número de entidades. Número de value objects. Número de raíces de agregado.

Paso 3: En `ValidadorDominioAceria.cs`, añadir el siguiente método estático dentro de la clase:

Archivo: `src/AceriaApp.Api/DominioAceria/ValidadorDominioAceria.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}

public record ResultadoDominioAceria(bool EsValido, IReadOnlyList<string> Errores);
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase indicado por el texto de inserción.

```
public static ResumenDominioAceria Resumen(IEnumerable<ElementoAceria> elementos)
{
    var lista = elementos.ToList();
    var agregados = lista.Count(e => e.Tipo == "Agregado");
    var entidades = lista.Count(e => e.Tipo == "Entidad");
    var valueObjects = lista.Count(e => e.Tipo == "ValueObject");
    var raices = lista.Count(e => e.EsRaizDeAgregado);
    return new ResumenDominioAceria(lista.Count, agregados, entidades, valueObjects, raices);
}
```

Paso 4: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public static ResumenDominioAceria Resumen(IEnumerable<ElementoAceria> e` declara el método. Materializa la lista. Cuenta los agregados. Cuenta las entidades. Cuenta los value objects. Cuenta las raíces. Devuelve el resumen.

Paso 5: Guardar el archivo con `Ctrl+S`.

Paso 6: En `ValidadorDominioAceriaTests.cs`, añadir el siguiente test:

Archivo: `tests/AceriaApp.DominioAceria.Tests/ValidadorDominioAceriaTests.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase indicado por el texto de inserción.

```
[Fact]
public void Resumen_ConCatalogoPorDefecto_CalculaTotales()
{
    var resumen = ValidadorDominioAceria.Resumen(CatalogoAceria.ElementosPorDefecto());

    resumen.Total.Should().Be(10);
    resumen.Agregados.Should().Be(3);
    resumen.Entidades.Should().Be(4);
    resumen.ValueObjects.Should().Be(3);
    resumen.RaicesDeAgregado.Should().Be(3);
}
```

Paso 7: Explicar línea por línea:

Marca el método como test de xUnit. Declara el test. Invoca el método. Verifica el total. Verifica los agregados. Verifica las entidades. Verifica los value objects. Verifica las raíces.

Paso 8: Ejecuta `dotnet test tests/AceriaApp.DominioAceria.Tests/AceriaApp.DominioAceria.Tests.csproj -c Release` en la copia optativa. Deben pasar 19 tests del proyecto, incluido el nuevo resumen. Ejecuta después `dotnet test AceriaApp.sln -c Release` y exige 209 pruebas aprobadas.

Paso 9: Comprueba que solo cambian `ValidadorDominioAceria.cs` y `ValidadorDominioAceriaTests.cs` en la copia optativa; la etapa obligatoria `5.5-dominio-aceria` y el punto anterior permanecen idénticos.

Paso 10: Hacer commit con `git commit -m "Reto 5.5: método Resumen en ValidadorDominioAceria"`.

Verificación: `Passed: 19` en el proyecto del punto y 209 pruebas aprobadas en la solución de la copia, cero fallos y cero omisiones.

Resultado verificable

Al finalizar este punto, el alumno debe tener:

- La carpeta `M05_DDD_CQRS/PROYECTO/5.5-dominio-aceria/` con la solución `AceriaApp.sln` que contiene 22 proyectos.
- El documento `docs/aplicacion-dominio-aceria.md` con los contextos, los agregados y el cierre del módulo 5.
- El documento `docs/ADR-023-aplicacion-dominio-aceria.md` con la decisión sobre la aplicación al dominio.
- Las clases `ElementoAceria`, `ValidadorDominioAceria` y `CatalogoAceria` en `src/AceriaApp.Api/DominioAceria/`.
- El proyecto `tests/AceriaApp.DominioAceria.Tests` añade 10 tests de catálogo y 8 funcionales; el reto añade uno más.
- `dotnet test AceriaApp.sln -c Release` ejecuta 208 pruebas en la etapa obligatoria y 209 en una copia con su reto; ninguna falla ni se omite.
- Los ADR-010 a ADR-022 del punto 5.4 sin modificar.
- Un commit revisado para 5.5, con el historial previo conservado.
- El módulo 5 cerrado con el resumen del estado del proyecto en `docs/aplicacion-dominio-aceria.md`.

Continuidad del aprendizaje

El punto 5.5 ha cerrado el módulo 5 con la aplicación concreta de DDD y CQRS al dominio de la acería. El proyecto AceriaApp dispone ahora de catálogos de varios contextos y de un contexto Producción ejecutable con agregado, lenguaje ubicuo, casos de uso y separación entre commands y queries. El módulo 5 queda cerrado con el resumen del estado del proyecto en `docs/aplicacion-dominio-aceria.md`. Esta base permite avanzar hacia el módulo 6, donde se estudiará la comunicación entre microservicios: comunicación síncrona mediante HTTP, comunicación asíncrona basada en eventos y criterios para elegir cada enfoque.